



ספר פרויקט לעבודת גמר

י"ד הנדסאי תוכנה (שאלון 714918)

בנושא

שרת FTP לאחסון והורדת קבצים



שם הסטודנט: אוריאל טוטיאשוילי

ת"ז: 215750779

מנחה: אילן פרץ

תאריך הגשה: י"ח סיוון תשפ"ו 03/06/2026

שם המכללה: כנפי רוח קרית נוער ירושלים (סמל מוסד: 140129)

תוכן עניינים

4	1. הצעת הפרויקט שאושרה ע"י משרד החינוך
5	2. מבוא
5	2.1 הרקע לפרויקט
6	2.2 תהליך המחקר
7	2.3 רקע תיאורטי וסקירת ספרות מקצועית
8	2.4 אתגרים מרכזיים במחקר
9	3. מטרות ויעדים לפיתוח המערכת
9	4. אתגרים בבניית המערכת
10	5. הגדרת מדדי הצלחה למערכת
10	6. תיאור פתרונות קיימים לבעיה וניתוח חלופות
12	7. תיאור החלופה הנבחרת ונימוקים לבחירתה
13	8. אפיון המערכת המוצעת
13	8.1 ניתוח הדרישות מהמערכת
14	8.2 מודלי המערכת
16	8.3 אפיון פונקציונלי וביצועים עיקריים
17	8.4 אילוצי המערכת
19	9. תיאור הארכיטקטורה של המערכת
19	9.1 ארכיטקטורת המערכת
21	9.2 מודל שרת לקוח
22	9.3 תהליכים של מע' ההפעלה
23	9.4 תיאור פרוטוקולי תקשורת
25	9.5 שירותים חיצוניים
27	10. ניתוח תרחישים וזרימת המידע במערכת המוצעת
27	10.1 תרשימי תרחיש Use case Diagram
28	10.2 תרשימי רצף Sequence Diagram
33	11. תיאור מסכים וממשק משתמש
33	11.1 תרשים היררכיית המסכים
36	11.2 תיאור המסכים
41	12. תיאור התוכנה
41	12.1 סביבת הפיתוח של התוכנה
42	12.2 תיאור מבנה הפרויקט והמחלקות
90	12.3 מבני נתונים בהם נעשה שימוש
92	12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים
98	13. תיאור מסד הנתונים
99	14. מדריך למשתמש
103	15. בדיקות והערכה

105.....	16. מסקנות
105.....	17. פיתוחים עתידיים
106.....	18. ביבליוגרפיה

1. הצעת הפרויקט שאושרה ע"י משרד החינוך

להלן הצעת הפרויקט שאושרה ע"י משרד החינוך בקצרה.

תיאור הנושא הפרויקט

המערכת היא אתר אינטרנט לאחסון ולהעברת קבצים, המיועד למשתמשים פרטיים הזקוקים לפתרון פשוט ונגיש לשמירת קבצים ולהעברתם בין מכשירים שונים. המערכת מאפשרת למשתמש לשמור קבצים על שרת מרוחק, ולגשת אליהם מכל מקום ובכל זמן באמצעות דפדפן אינטרנט, ללא צורך בהתקנת תוכנה או שימוש באמצעי אחסון פיזיים. השימוש במערכת מתבצע בצורה אינטואיטיבית: המשתמש נרשם לאתר, מתחבר לחשבון האישי שלו, מעלה קבצים מהמחשב האישי, וצופה ברשימת הקבצים השמורים עבורו. בכל שלב המשתמש יכול להוריד קבצים חזרה למחשב או למחוק קבצים שאינם נחוצים עוד.

הגדרת הבעיה האלגוריתמית

בתחום העברת ואחסון קבצים ברשתות מחשבים קיימת בעיה מרכזית הנוגעת לנפח הנתונים המועברים והמאוחסנים. קבצים דיגיטליים, ובפרט קבצים גדולים כגון סרטונים, מסמכים מורכבים וארכיונים, דורשים משאבי רשת ואחסון משמעותיים. העברת קבצים אלו כפי שהם, ללא עיבוד מוקדם, מובילה להעמסת רוחב הפס, להארכת זמני העברה ולניצול לא יעיל של משאבי המערכת. הבעיה האלגוריתמית המוגדרת בפרויקט היא הפחתת נפח המידע של קבצים דיגיטליים לצורך העברה ואחסון ברשת, תוך שמירה מלאה על תוכן הקובץ המקורי ואפשרות לשחזור מדויק שלו. באופן פורמלי, מדובר בבעיה מסוג דחיסת נתונים חסרת אובדן (LossLess Data Compression) שבה האלגוריתם מקבל קלט של רצף נתונים ומחזיר ייצוג דחוס יותר, כך שקיים תהליך הפוך המאפשר שחזור מלא של הנתונים ללא כל אובדן מידע. בעיה זו היא בעיה אלגוריתמית מובהקת, שכן היא עוסקת בזיהוי תבניות וחזרות בנתונים, בניהול מבני נתונים מתאימים ובביצוע טרנספורמציה של הקלט לפלט יעיל יותר מבחינת נפח. הגדרת הבעיה האלגוריתמית נובעת ישירות מהאתגרים המרכזיים שתוארו בסעיף תיאור הנושא:

- הצורך ביעול העברת קבצים ברשתות מחשבים
- מגבלות רוחב פס וזמני העברה ארוכים
- דרישה לשימוש חסכוני במשאבי אחסון

2. מבוא

2.1 הרקע לפרויקט

בעידן הדיגיטלי המודרני, תחום העברת ואחסון קבצים ברשתות מחשבים מהווה נדבך מרכזי ובלתי נפרד מתשתיות המידע העולמיות. כתוצאה מהתפתחותן המהירה של מערכות ארגוניות, שירותי ענן, מערכות גיבוי ושיתוף מידע, נפחי המידע הדיגיטלי המיוצרים והמועברים מדי יום גדלים בקצב מואץ. על כן, ניהול יעיל, אמין וחסכוני של מידע זה הפך לאחד האתגרים ההנדסיים המשמעותיים ביותר בעולמות של מערכות תקשורת, רשתות מחשבים וניהול מידע דיגיטלי.

המטרה המרכזית העומדת בבסיסו של תחום העברת ואחסון הקבצים היא לאפשר זרימת מידע חלקה, מהירה ורציפה בין גורמים ומערכות קצה שונות. זרימה זו חייבת להתבצע תוך הבטחת שלושה עקרונות יסודיים: שמירה מוחלטת על שלמות הנתונים, שמירה על זמני תגובה מהירים וניצול אופטימלי של משאבי הרשת והחומרה הקיימים במערכת.

עם זאת, לצד החשיבות המכריעה של תשתיות אלו, פיתוח מערכות מודרניות להעברת ואחסון קבצים כרוך בהתמודדות עם מגבלות פיזיות ולוגיות מורכבות של תשתיות התקשורת ורוחב הפס. האתגרים המרכזיים המאפיינים את התחום כיום כוללים:

- **צריכת רוחב פס גבוהה ועומסי רשת:** עבודה עם קבצים בנפחים משתנים, ובפרט קבצים גדולים מאוד (כגון סרטוני וידאו, מסמכים מורכבים וארכיונים), דורשת משאבי רשת עצומים. העברת קבצים אלו במצבם המקורי ("הגולמי"), ללא עיבוד מוקדם, יוצרת עומס כבד על רוחב הפס הציבורי והארגוני, דבר המוביל להאטת שירותים חיוניים אחרים ברשת.
- **זמני העברה ארוכים ואי-יציבות בתקשורת:** בסביבות תקשורת בעלות חיבורים איטיים, מוגבלים או לא יציבים, זמני העברת הקבצים מתארכים משמעותית, מה שפוגע ישירות בחוויית המשתמש וביעילות העבודה הארגונית.
- **רגישות לשגיאות תקשורת ואיבוד נתונים:** במהלך העברת מידע ברשת, המערכות נדרשות להתמודד דרך קבע עם תופעות של איבוד חבילות מידע, (Packet Loss) ניתוקי תקשורת פתאומיים ושגיאות שידור, הדורשים מנגנוני התאוששות ובקרה קפדניים מבלי לפגוע בשלמות ובדיוק של הקובץ.
- **ניהול וניצול משאבי מערכת:** אחסון והעברה של קבצים רבים ומגוונים דורשים מהשרתים הקצאה דינמית ויעילה של משאבי חומרה חיוניים, כגון זיכרון עבודה, כוח עיבוד (CPU) ונפח אחסון דיסק פיזי.

כדי להתמודד בהצלחה עם אתגרים אלו, נוצר צורך הנדסי מובהק בפתרונות תוכנה מתקדמים שאינם מסתפקים רק בהעברת קבצים פסיבית, אלא משלבים שיטות חכמות לניהול תעבורה, חלוקת מידע והקטנת נפחי נתונים.

במסגרת פרויקט זה, הפתרון המוצע מתמקד בפיתוח שרת FTP (File Transfer Protocol) ייעודי לאחסון והורדת קבצים, המשלב בתוכו מנגנון אלגוריתמי מובנה של דחיסת נתונים חסרת אובדן (Lossless Data Compression) שילוב זה מאפשר להקטין באופן משמעותי את נפח הקבצים בזמן העלאתם לשרת ולפרוס אותם חזרה בעת הורדתם, ובכך הוא מספק מענה הנדסי ישיר לבעיית רוחב הפס ומגבלות האחסון, תוך הבטחה מלאה שהקובץ המשוחזר יהיה זהה לחלוטין לקובץ המקור.

2.2 תהליך המחקר

תהליך המחקר בפרויקט זה התמקד בבחינת האתגרים המרכזיים בתחום העברת ואחסון קבצים ברשתות מחשבים, תוך דגש על יעול נפח הנתונים המועברים ושיפור ביצועי המערכת. המחקר בוצע בשלבים, במטרה לבחור פתרון הנדסי מתאים אשר יספק מענה לבעיית רוחב הפס וניצול משאבי האחסון.

בשלב הראשון בוצע מחקר בתחום מערכות העברת הקבצים ופרוטוקולי התקשורת המקובלים בעולם התוכנה והרשתות. במסגרת זו נלמדו עקרונות הפעולה של פרוטוקול FTP, מבנה מודל שרת-לקוח, ואופן ניהול תהליכי העלאה והורדה של קבצים דרך רשת האינטרנט. בנוסף נבחנו מגבלות נפוצות במערכות מסוג זה, כגון עומסי רשת, זמני העברה גבוהים וניצול לא יעיל של שטחי אחסון.

בשלב השני בוצעה סקירה של שיטות שונות לדחיסת נתונים ללא אובדן מידע (Lossless Data Compression). במהלך המחקר נבדקו מספר אלגוריתמים מרכזיים בתחום, ביניהם Huffman Coding, LZ77, LZ78 ו-LZW. עבור כל אלגוריתם נבחנו עקרונות הפעולה, מורכבות החישוב, יעילות הדחיסה, התאמה להעברת קבצים ברשת ויכולת השחזור של המידע המקורי ללא אובדן נתונים.

לאחר ניתוח האלגוריתמים בוצעה השוואה בין הפתרונות השונים בהתאם לצורכי המערכת. נמצא כי האלגוריתם LZ78 מספק איזון מתאים בין פשטות המימוש, יעילות הדחיסה ויכולת העבודה עם קבצים מסוגים שונים. בנוסף, מבנה הפעולה ההדרגתי של האלגוריתם מאפשר שילוב נוח בתהליכי העלאה והורדה של קבצים בשרת.

בשלב הבא נבחנה הארכיטקטורה המתאימה למערכת. במסגרת זו נבדקו מודלים שונים של מערכות מבוזרות, ולבסוף נבחרה ארכיטקטורת שלוש שכבות (Three-Tier Architecture), המאפשרת הפרדה בין שכבת התצוגה, שכבת הלוגיקה היישומית ושכבת הנתונים. בחירה זו מאפשרת תחזוקה נוחה יותר של המערכת, הרחבה עתידית ושיפור אבטחת המידע.

בנוסף למחקר התאורטי, בוצע גם מחקר טכנולוגי שכלל בחינת טכנולוגיות Web, שרתי FTP, מנגנוני REST API ושיטות לניהול קבצים בשרת. מטרת המחקר הייתה לבחור פתרונות יציבים, מודולריים ויעילים אשר יתאימו למימוש מערכת אמינה להעברת ואחסון קבצים.

תהליך המחקר כולו אפשר להבין לעומק את הבעיה ההנדסית, לנתח פתרונות קיימים בתחום ולבחור בגישה המתאימה ביותר למטרות הפרויקט.

2.3 רקע תיאורטי וסקירת ספרות מקצועית

תחום דחיסת הנתונים מהווה נדבך מרכזי במדעי המחשב המודרניים, ומהווה רכיב בסיסי במערכות תקשורת, אחסון מידע ועיבוד נתונים. בעידן הדיגיטלי הנוכחי, בו כמויות המידע גדלות באופן מעריכי, עולה הצורך בפתרונות יעילים לצמצום נפח הנתונים תוך שמירה מלאה על שלמותם המקורית. דחיסה חסרת אובדן (Lossless Compression) מאפשרת ייצוג יעיל של מידע באופן כזה שניתן לשחזר את הנתונים המקוריים בצורה מדויקת לחלוטין לאחר תהליך הפירוק.

הבסיס המדעי לתחום זה נשען על תורת המידע (Information Theory), אשר פותחה על ידי קלוד שאנון. תיאוריה זו מגדירה את מושג האנטרופיה כמדד לכמות המידע הממוצעת במקור הסתברותי. האנטרופיה מתארת את גבול היעילות התיאורטי של דחיסת מידע, ומהווה חסם תחתון לכל שיטת קידוד ללא אובדן. בהתאם לכך, כל אלגוריתם דחיסה אינו יכול, במוצא, לעבור את גבול האנטרופיה של מקור המידע, אלא להתקרב אליו בלבד.

על בסיס תיאוריה זו פותחו לאורך השנים מודלים שונים לדחיסת מידע, אשר ניתן לחלק באופן כללי לשתי גישות מרכזיות: גישות הסתברותיות (Statistical Methods) וגישות מבוססות מבנה (Structure-Based Methods). גישות הסתברותיות מתבססות על שכיחות הופעת סמלים בקלט, בעוד שגישות מבוססות מבנה מתמקדות בזיהוי תבניות, חזרות ותלות פנימית בין רכיבי המידע.

סקירה היסטורית של התחום מצביעה על מעבר הדרגתי משיטות סטטיות, אשר דורשות ניתוח מוקדם של התפלגות הנתונים, לשיטות אדפטיביות המסוגלות ללמוד את מבנה הנתונים בזמן אמת. שינוי זה היווה התקדמות משמעותית, שכן במערכות מידע מודרניות לרוב אין ידע מוקדם על אופי הנתונים, והם עשויים להיות מגוונים מאוד (טקסט, קוד, מדיה, נתוני מערכת ועוד).

אחד העקרונות המרכזיים בתחום הדחיסה הוא עקרון החיזוי המקומי, לפיו ניתן לנצל תלות סטטיסטית בין רכיבי מידע סמוכים לצורך ייצוג קצר יותר של הנתונים. עיקרון זה עומד בבסיסם של מרבית האלגוריתמים המודרניים לדחיסה חסרת אובדן, והוא מאפשר זיהוי רצפים חוזרים, תבניות חוזרות ומבנים תחביריים בתוך זרם הנתונים.

בנוסף, התפתחות התחום הובילה לשילוב בין שיטות דחיסה לבין מערכות זמן אמת ותקשורת מחשבים. כיום דחיסת נתונים אינה רק כלי לאחסון יעיל, אלא רכיב מרכזי בפרוטוקולי תקשורת, מערכות ענן, שירותי סטרימינג ומערכות מבוזרות. שילוב זה מחייב איזון בין רמת הדחיסה לבין מורכבות החישוב וזמן הביצוע, שכן במערכות רשת קיימת מגבלה משמעותית של זמן ההשהיה (Latency) ורוחב פס.

מהבחינה המחקרית, ספרות מקצועית בתחום מצביעה על כך שאין פתרון אוניברסלי יחיד המתאים לכל סוגי הנתונים. מחקרים רבים מראים כי ביצועי אלגוריתם דחיסה תלויים באופן ישיר באופי הנתונים, ברמת החזרתיות שלהם, ובמבנה הפנימי של המידע. בהתאם לכך, תחום הדחיסה נחשב לתחום הסתגלותי (Adaptive Field), שבו פתרונות שונים מתאימים לסביבות שונות.

בנוסף, קיימת חשיבות הנדסית גבוהה לשיקולים של סיבוכיות חישובית ויעילות זיכרון. מערכות דחיסה מודרניות נדרשות לפעול בזמן ליניארי או קרוב לכך, תוך שימוש מבוקר במשאבי זיכרון, במיוחד בסביבות בעלות מגבלות חומרה או עבודה בזמן אמת. לפיכך, רבות מהמערכות המעשיות משלבות בין רעיונות תיאורטיים לבין התאמות הנדסיות שמטרתן לשפר ביצועים בפועל.

לסיכום, תחום דחיסת הנתונים חסרת האובדן מבוסס על יסודות מתמטיים עמוקים מתורת המידע, ומתפתח באופן מתמיד בהתאם לצורכי מערכות המחשוב המודרניות. שילוב בין תיאוריה הסתברותית, זיהוי תבניות ומגבלות הנדסיות הופך את התחום לאחד מאבני היסוד של מערכות מידע ותקשורת עכשוויות.

2.4 אתגרים מרכזיים במחקר

תחום העברת ואחסון הקבצים ברשתות מחשבים כולל מספר אתגרים הנדסיים משמעותיים, הנובעים מהצורך להתמודד עם כמויות גדולות של מידע דיגיטלי, מגבלות רוחב פס, זמני העברה ארוכים ושמירה על שלמות הנתונים במהלך ההעברה והאחסון.

במסגרת המחקר והפיתוח של הפרויקט נבחנו האתגרים המרכזיים בתחום, תוך התמקדות בייעולת תהליך העברת הקבצים וצמצום נפח הנתונים המועברים ברשת באמצעות דחיסת נתונים ללא אובדן מידע. בנוסף, נדרש לבחון פתרונות אלגוריתמיים שונים ולבחור בגישה שתאפשר איזון בין יעילות הדחיסה, מהירות הביצוע ויכולת שחזור מלאה של הקבצים המקוריים.

2.4.1 התמודדות עם הבעיה

במהלך המחקר והפיתוח של הפרויקט עלו מספר אתגרים מרכזיים הנוגעים לייעול העברת ואחסון קבצים ברשתות מחשבים. האתגר המרכזי היה הצורך בצמצום נפח הנתונים המועברים מבלי לפגוע בשלמות המידע, שכן כל פגיעה או אובדן נתונים עלולים לגרום לשחזור שגוי של הקובץ המקורי.

אתגר נוסף היה בחירת אלגוריתם דחיסה המתאים לעבודה עם מגוון סוגי קבצים, כולל קבצים גדולים ומורכבים, תוך שמירה על יעילות חישובית סבירה וזמן עיבוד קצר יחסית. בנוסף, היה צורך למצוא פתרון המאפשר שילוב חלק של תהליך הדחיסה בתוך מערכת הפועלת בסביבת רשת, כך שלא יפגע בתהליך העברת הקבצים או בחוויית המשתמש.

כמו כן, נדרש להתמודד עם האיזון בין רמת הדחיסה לבין מהירות הביצוע, שכן אלגוריתמים מורכבים יותר עשויים להציע דחיסה טובה יותר אך במחיר של זמן עיבוד גבוה.

2.4.2 הסיבות והמוטיבציה לבחירת הנושא

בחרתי לפתח שרת FTP המשלב אלגוריתם דחיסה משום שעניין אותי לחבר בין שני תחומים מרכזיים: תקשורת ורשתות מחשבים מצד אחד, ואלגוריתמים ועיבוד נתונים מצד שני. רציתי ליצור מערכת שמשלבת עבודה עם תקשורת בין מחשבים יחד עם מנגנון שמייעל את העברת הנתונים ומשפר את ביצועי המערכת.

המוטיבציה המרכזית שלי בפרויקט הייתה להבין בצורה מעמיקה כיצד מתבצעת העברת קבצים ברשת וכיצד פועל שרת תקשורת אמיתי. פיתוח שרת ה-FTP ועבודה ישירה מול Sockets אפשרו לי להיחשף לתהליך המלא של יצירת חיבורים בין מחשבים, ניהול תקשורת בין שרת ללקוח, שליחת נתונים בזמן אמת וטיפול במספר משתמשים במקביל.

בנוסף, שילוב אלגוריתם דחיסה בפרויקט הוסיף רובד נוסף של חשיבה אלגוריתמית וייעול ביצועים. באמצעות הדחיסה ניתן לצמצם את נפח המידע המועבר ברשת, לשפר את יעילות ההעברה ולהפחית זמני שליחה והורדה של קבצים.

הפרויקט היווה עבורי אתגר משמעותי משום שהוא משלב מספר תחומים שונים בתוך מערכת אחת מלאה ועובדת, ודורש תכנון ארכיטקטורה, עבודה בצד השרת, ניהול מידע ופתרון בעיות תקשורת בזמן אמת.

2.4.3 על איזה צורך הפרויקט עונה ?

הפרויקט נותן מענה לצורך ממשי בהעברה ואחסון יעיל של קבצים דיגיטליים ברשת. משתמשים רבים נדרשים כיום להעביר קבצים גדולים בין מכשירים שונים, כאשר מגבלות רוחב פס, זמני העברה ונפח אחסון מהווים בעיה משמעותית.

המערכת המוצעת בפרויקט פותרת בעיה זו באמצעות שילוב של שרת אחסון קבצים יחד עם מנגנון דחיסת נתונים חסרת אובדן, המאפשר צמצום נפח הקבצים לפני אחסונם והעברה מהירה יותר שלהם ברשת. בצורה זו ניתן לשפר את ניצול משאבי המערכת, להפחית עומסים על הרשת, ולהבטיח שמירה מלאה על שלמות המידע.

בנוסף, הפרויקט נותן מענה לצורך בנגישות גבוהה לקבצים מכל מקום ובכל זמן, ללא תלות באמצעי אחסון פיזיים, ובכך תורם ליעילות ונוחות השימוש במערכות ניהול קבצים מודרניות.

3. מטרות ויעדים לפיתוח המערכת

מטרת הפרויקט היא פיתוח מערכת אינטרנטית לאחסון והעברת קבצים, המבוססת על שרת FTP ומנגנון דחיסת נתונים חסרת אובדן, אשר תאפשר למשתמשים ניהול קבצים אישי הכולל העלאה, הורדה ומחיקה בצורה יעילה, מאובטחת ונגישה מכל מקום באמצעות דפדפן.

המערכת נועדה להתמודד עם בעיית העברת קבצים בנפחים גדולים ברשתות מחשבים, תוך שימוש בדחיסת נתונים לשם צמצום נפח המידע, שיפור זמני ההעברה והקטנת העומס על משאבי האחסון והרשת.

בנוסף, הפרויקט שם דגש על חוויית משתמש פשוטה ואינטואיטיבית, כך שגם משתמשים ללא ידע טכני יוכלו לבצע פעולות בסיסיות במערכת בצורה קלה וברורה.

יעדי המערכת כוללים פיתוח מערכת יציבה ואמינה להעברת קבצים, מימוש מנגנון דחיסה חסרת אובדן לשיפור יעילות האחסון והעברת הנתונים, תמיכה בניהול משתמשים וקבצים באופן אישי ומבודל, שמירה על הרשאות גישה ואבטחת מידע בסיסית, ופיתוח ממשק משתמש נוח ונגיש.

המערכת מתוכננת להיות מודולרית וגמישה, כך שתאפשר הרחבות עתידיות כגון שיפור אלגוריתם הדחיסה, הוספת שירותי ענן נוספים, ותמיכה בסוגי קבצים נוספים.

4. אתגרים בבניית המערכת

במהלך פיתוח המערכת להעברת ואחסון קבצים עלו מספר אתגרים הנדסיים אשר השפיעו על תכנון המערכת ועל אופן מימוש הפתרון. אתגרים אלו נבעו מהצורך לשלב בין מערכת מבוססת שרת-לקוח, פרוטוקול FTP ומנגנון דחיסת נתונים חסרת אובדן, תוך שמירה על ביצועים גבוהים ואמינות מלאה של המידע.

אחד האתגרים המרכזיים היה שילוב מנגנון הדחיסה בתהליך העלאת והורדת הקבצים. מצד אחד, הדחיסה נועדה לצמצם את נפח הנתונים ולשפר את יעילות ההעברה, אך מצד שני היא מוסיפה שלב עיבוד נוסף אשר עלול להשפיע על זמני התגובה של המערכת. לכן נדרש איזון בין רמת הדחיסה לבין ביצועי המערכת בזמן אמת.

אתגר נוסף היה שמירה על שלמות הנתונים (Data Integrity) לאורך כל תהליך ההעברה. מאחר והמערכת עושה שימוש בדחיסה ללא אובדן, היה צורך לוודא שתהליך הפריסה (Decompression) משחזר את הקובץ בדיוק מוחלט, ללא פגיעה במידע המקורי.

בנוסף, נדרש להתמודד עם עבודה עם קבצים גדולים במיוחד, אשר עלולים לגרום לעומס על זיכרון המערכת ועל משאבי הרשת. לשם כך היה צורך לתכנן את תהליך העיבוד כך שיתבצע בצורה יעילה והדרגתית ככל האפשר.

אתגר נוסף היה שילוב בין רכיבי המערכת השונים, המערכת משלבת בין שני פרוטוקולי תקשורת, כאשר לכל אחד תפקיד מוגדר וברור.

שרת ה-HTTP המשמש כשכבת תיווך בין המשתמש לבין המערכת, ואחראי על קבלת בקשות והצגת מידע, בעוד ששרת ה-FTP אחראי על ניהול הקבצים.

לבסוף, עליה גם נושא אבטחת המידע וניהול ההרשאות, במטרה להבטיח שכל משתמש יוכל לגשת אך ורק לקבצים השייכים לו, ולמנוע גישה לא מורשית למידע המאוחסן במערכת.

5. הגדרת מדדי הצלחה למערכת

הצלחת המערכת תימדד על פי מספר מדדים מרכזיים, המשקפים את איכות הביצוע, היעילות והאמינות של תהליך העברת ואחסון הקבצים.

ראשית, נבחן את תקינות פעולות ההעלאה וההורדה. מערכת תיחשב כמוצלחת אם כל קובץ אשר הועלה לשרת ניתן להורדה מלאה ושחזור מדויק ללא פגיעה בנתונים, תוך שמירה על שלמות הקובץ המקורי.

מדד נוסף הוא יעילות הדחיסה. המערכת תיבחן לפי יחס הדחיסה (Compression Ratio), כלומר הפחתת נפח הקובץ לאחר תהליך הדחיסה בהשוואה לגודלו המקורי. ככל שיחס הדחיסה גבוה יותר, כך ניצול המשאבים יעיל יותר.

בנוסף, תיבחן ביצועי המערכת מבחינת זמן תגובה. המערכת תיחשב יעילה אם זמני ההעלאה וההורדה של קבצים, כולל תהליך הדחיסה והפריסה, נשארים סבירים ואינם פוגעים בחוויית המשתמש.

מדד חשוב נוסף הוא יציבות המערכת תחת עומס. המערכת צריכה לתמוך במספר משתמשים בו זמנית ובפעולות מקבילות של העלאה והורדה, מבלי לגרום לקריסות או לאיבוד נתונים.

לבסוף, תיבחן נוחות השימוש במערכת, כך שהמשתמש יוכל לבצע פעולות בסיסיות כגון התחברות, העלאת קובץ והורדתו בצורה פשוטה, ברורה ואינטואיטיבית ללא צורך בידע טכני מתקדם.

6. תיאור פתרונות קיימים לבעיה וניתוח חלופות

דחיסת נתונים – היבט אלגוריתמי

העברת קבצים גדולים ברשת יוצרת עומס על רוחב הפס ודורשת זמן רב. לפיכך נעשה שימוש בדחיסת נתונים, המתמקדת בצמצום נפח המידע תוך שמירה על תוכנו המקורי. בפרויקט נעשה שימוש בדחיסה ללא אובדן מידע, שבה הקובץ המשוחזר זהה לחלוטין למקור. קיימים מספר אלגוריתמים מרכזיים בתחום זה.

אלגוריתמים מרכזיים לדחיסה ללא אובדן

Huffman Coding הוא אלגוריתם דחיסה ללא אובדן מידע המבוסס על ניתוח תדירות הופעת התווים בקלט. האלגוריתם בונה עץ בינארי שבו תווים בעלי תדירות גבוהה מקבלים קוד בינארי קצר יותר,

בעוד שתווים נדירים מקבלים קוד ארוך יותר. שיטה זו מבטיחה קידוד אופטימלי מבחינת אורך ממוצע של הקוד, אך אינה מנצלת חזרות של רצפי תווים אלא פועלת ברמת התו הבודד.

LZ77 הוא אלגוריתם דחיסה המבוסס על זיהוי חזרות מקומיות בזרם הנתונים. האלגוריתם עושה שימוש בחלון הזזה (Sliding Window) ומחליף רצפים חוזרים בהפניות לאזורים קודמים בקלט, המיוצגות על ידי מרחק ואורך. שיטה זו מתאימה במיוחד לזרמי נתונים רציפים, אך יעילותה תלויה בגודל החלון ובמרחק בין החזרות.

LZ78 פועל על ידי בניית מילון של רצפים חדשים בזמן קריאת הקלט. בכל פעם שמופיע רצף שאינו קיים במילון, הוא מתווסף אליו ומיוצג באמצעות אינדקס למילון ותו נוסף. גישה זו מאפשרת ייצוג קומפקטי של חזרות ארוכות ומגוונות, תוך שמירה על תהליך דחיסה ופענוח פשוטים וברורים.

LZW הוא שיפור של LZ78, שבו אין צורך לשלוח את התו הנוסף בכל שלב, אלא רק אינדקסים למילון המתפתח. שיטה זו מיעלת את תהליך הקידוד ומקטינה את נפח הפלט, במיוחד בקבצים טקסטואליים. עם זאת, האלגוריתם מורכב יותר ליישום ולניתוח תאורטי בהשוואה ל-LZ78.

שם האלגוריתם	יתרונות	חסרונות	זמן ריצה
Huffman code	קידוד אופטימלי לפי תדירות, פשוט מבחינה מתמטית, מבטיח קידוד מינימלי	אינו מזהה חזרות, דורש מעבר מלא על הנתונים לבניית עץ, פחות יעיל לבד	$O(n \log n)$
LZ77	אינו דורש מילון קבוע מראש, יעיל בזיהוי חזרות סמוכות, מתאים לזרימת נתונים רציפה	פחות יעיל כאשר החזרות מרוחקות, צורך חלון חיפוש, ביצועים תלויים בגודל החלון	$O(n)$
LZ78	בניית מילון דינמית, מתאים למגוון רחב של קבצים, פשוט להבנה ולשחזור, דחיסה יציבה	צורך אחסון מילון, יעילות נמוכה יחסית בקבצים קצרים מאוד	$O(n)$
LZW	אינו דורש שליחת מילון בנפרד, יעיל מאוד לקבצים טקסטואליים, נפוץ בתעשייה	מורכב יותר מ-LZ78, בעיות רישוי היסטוריות, פחות שקוף להסבר תאורטי	$O(n)$

7. תיאור החלופה הנבחרת ונימוקים לבחירתה

האלגוריתם הנבחר:

האלגוריתם **LZ78** נבחר כאלגוריתם הדחיסה המרכזי במערכת, מאחר שהוא מספק מענה ישיר לדרישות הבעיה האלגוריתמית: צמצום נפח נתונים לצורך העברה ואחסון, תוך שמירה מלאה על שלמות המידע. האלגוריתם מבצע דחיסה ללא אובדן, ולכן מאפשר שחזור מדויק של הקובץ המקורי לאחר ההעברה.

LZ78 פועל על ידי קריאת הקלט משמאל לימין ובניית מילון דינמי של רצפים חוזרים המופיעים בנתונים. בכל שלב, האלגוריתם מזהה את הרצף הארוך ביותר שכבר קיים במילון, ומוסיף רצף חדש המורכב ממנו ומתו נוסף. כל רצף מיוצג באמצעות זוג הכולל אינדקס למילון ותו, ובכך מתקבל ייצוג קומפקטי יותר של הנתונים. גישה זו מאפשרת ניצול יעיל של חזרות במידע, גם כאשר מבנה הקובץ אינו ידוע מראש, ומאפשרת עיבוד הדרגתי של הנתונים – תכונה חשובה בעבודה עם קבצים גדולים ובהעברה ברשת.

אופן פעולת האלגוריתם

האלגוריתם פועל לפי השלבים הבאים:

1. אתחול מילון ריק, שבו האינדקס 0 מייצג רצף ריק.
 2. קריאת הקלט מהתו הראשון ועד סופו.
 3. בכל שלב, האלגוריתם מחפש את הרצף הארוך ביותר מהקלט הנוכחי שכבר קיים במילון.
 4. לאחר זיהוי הרצף, האלגוריתם קורא את התו הבא בקלט שעדיין לא נכלל ברצף.
 5. נוצר פלט בצורת זוג:
 - אינדקס הרצף הקיים במילון
 - התו החדש שנוסף
 6. הרצף החדש (הרצף הקיים + התו החדש) מתווסף למילון עם אינדקס חדש.
 7. האלגוריתם ממשיך לקרוא את הקלט מנקודת הסיום של הרצף שנמצא.
 8. התהליך נמשך עד לסיום הקלט.
- באופן זה, במקום לשלוח או לאחסן כל תו בנפרד, האלגוריתם מייצג רצפים שלמים באמצעות זוגות קצרים, ובכך מצמצם את נפח הנתונים הכולל.

הפעולה של האלגוריתם ניתנת לתיאור מילולי באופן הבא:

התחל עם מילון ריק.

קרא את הקלט משמאל לימין.

כל עוד נותרו תווים בקלט:

מצא את הרצף הארוך ביותר שכבר מופיע במילון ומתחיל במיקום הנוכחי בקלט.

קרא את התו הבא שאינו חלק מהרצף שנמצא.

פלט זוג המורכב מהאינדקס של הרצף במילון ומהתו החדש.

הוסיף למילון רצף חדש המורכב מהרצף שנמצא ומהתו החדש.

המשך לקרוא את הקלט מהמיקום הבא.

סיים כאשר כל הקלט עובד.

תיאור זה מדגיש את אופי העבודה ההדרגתית של האלגוריתם ואת בניית המילון תוך כדי תהליך הדחיסה.

דוגמה להמחשה:

עבור המחזורת: ABABABA

האלגוריתם יבנה בהדרגה מילון של רצפים כגון:

- A •
- B •
- AB •
- ABA •

במקום לייצג כל תו בנפרד, הרצפים החוזרים מוחלפים בייצוג קצר המבוסס על אינדקס מהמילון ותו נוסף, וכך נפח המידע הכולל קטן.

8. אפיון המערכת המוצעת

8.1 ניתוח הדרישות מהמערכת

המערכת המוצעת היא מערכת Web לניהול, אחסון והעברת קבצים באמצעות שרת FTP, הכוללת מנגנון דחיסה חסרת אובדן המבוצע בצד השרת. מטרת המערכת היא לאפשר למשתמשים העלאה, הורדה וניהול קבצים בצורה מאובטחת, יעילה ונגישה דרך דפדפן אינטרנט. דרישות פונקציונליות:

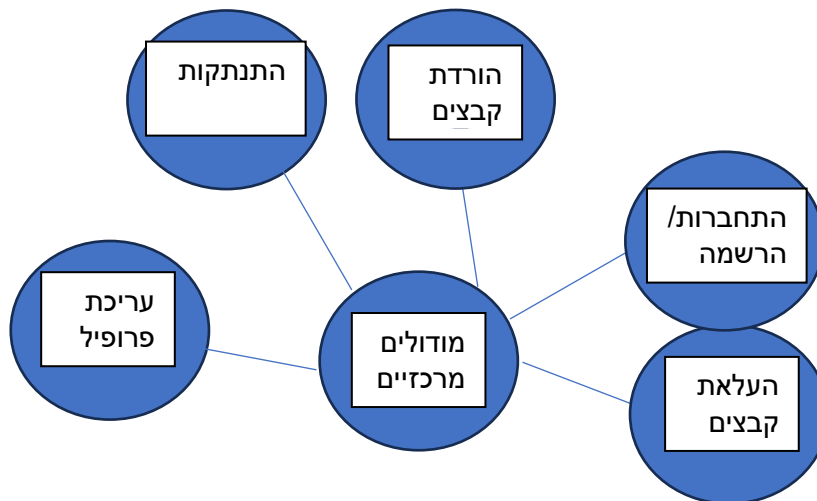
- הרשמה והתחברות של משתמשים למערכת
- ניהול משתמשים והרשאות גישה לקבצים
- העלאת קבצים מהלקוח לשרת
- הורדת קבצים מהשרת ללקוח
- מחיקת קבצים מהשרת
- דחיסה אוטומטית של קבצים בעת העלאה
- פריסת קבצים (Decompression) בעת הורדה
- שמירת נתוני קבצים ומידע משתמשים במסד נתונים

דרישות לא פונקציונליות:

- תמיכה בריבוי משתמשים במקביל (Concurrency)
- זמני תגובה מהירים בפעולות העלאה והורדה
- יעילות בשימוש בזיכרון ואחסון

- אמינות גבוהה ושמירה על שלמות הנתונים
- אבטחת מידע בסיסית (אימות משתמשים והרשאות)
- עבודה יציבה לאורך זמן ללא קריסות מערכת

8.2 מודלי המערכת



1. יצירת משתמש

מקרה שימוש זה מתאר את תהליך ההרשמה למערכת.

המשתמש מזין שם משתמש, כתובת אימייל וסיסמה, והמערכת מבצעת בדיקות תקינות עבור הנתונים שהוזנו, כגון תקינות מבנה האימייל, חוזק הסיסמה וזמינות שם המשתמש או כתובת האימייל במערכת.

במידה וכל הנתונים תקינים והאימייל אינו קיים במערכת, נוצר חשבון חדש ונשמר במסד הנתונים. לאחר השלמת ההרשמה, המשתמש יכול להתחבר למערכת באמצעות פרטי החשבון שיוצר.

2. התחברות למערכת

מקרה שימוש זה מתאר את תהליך הכניסה למערכת.

המשתמש מזין את שם המשתמש והסיסמה שלו, והמערכת מבצעת אימות מול הנתונים השמורים במסד הנתונים.

במידה והאימות מצליח, המערכת יוצרת סשן פעיל עבור המשתמש ומעניקה לו גישה לאזור האישי ולפעולות המערכת השונות.

3. התנתקות מהמערכת

מקרה שימוש זה מתאר את תהליך היציאה מהמערכת.

כאשר המשתמש בוחר להתנתק, המערכת סוגרת את הסשן הפעיל ומבטלת את הרשאות הגישה הנוכחיות, במטרה לשמור על אבטחת המידע ולמנוע גישה לא מורשית לחשבון.

4. עריכת פרופיל משתמש

מקרה שימוש זה מתאר עדכון פרטי משתמש במערכת.

המשתמש יכול לערוך פרטים אישיים כגון שם משתמש, כתובת אימייל או סיסמה. לפני שמירת הנתונים החדשים, המערכת מבצעת בדיקות תקינות ואימות נתונים כדי לוודא שהמידע שהוזן תקין ואינו מתנגש עם משתמשים אחרים במערכת.

לאחר השלמת הבדיקות, הנתונים המעודכנים נשמרים במסד הנתונים.

5. העלאת קובץ

מקרה שימוש זה מתאר את תהליך העלאת הקבצים לשרת.

המשתמש בוחר קובץ מהמחשב המקומי ושולח אותו לשרת באמצעות המערכת. במהלך התהליך, המערכת מאמתת את הרשאות המשתמש ושומרת את פרטי המשתמש שהעלה את הקובץ לצורך זיהוי וניהול עתידי.

לאחר קבלת הקובץ, המערכת מפעילה את אלגוריתם הדחיסה לצורך הקטנת נפח הנתונים לפני האחסון. בסיום התהליך הקובץ נשמר בשרת ומוצגת למשתמש הודעה המאשרת שהפעולה הושלמה בהצלחה.

6. הורדת קובץ

מקרה שימוש זה מתאר את תהליך הורדת הקבצים מהשרת.

המשתמש בוחר קובץ מתוך רשימת הקבצים הזמינים עבורו, והמערכת בודקת האם קיימת לו הרשאת גישה לקובץ בהתאם לפרטי המשתמש המחוברים.

לאחר אימות ההרשאות, המערכת מבצעת תהליך פריסה (Decompression) המשחזר את הקובץ המקורי מהמידע הדחוס. בסיום התהליך הקובץ נשלח למשתמש ומורד למחשב המקומי שלו.

8.3 אפיון פונקציונלי וביצועים עיקריים

המערכת פותחה כך שתבצע תהליך מלא של העברת קבצים, דחיסה, אחסון ושליפה בצורה אוטומטית ושקופה למשתמש.

הארכיטקטורה של המערכת מבוססת על הפרדה בין שכבת התקשורת, שכבת עיבוד הנתונים ושכבת האחסון, במטרה לשמור על מבנה מודולרי, יעיל ונוח להרחבה.

רכיבי המערכת המרכזיים:

ניהול משתמשים והרשאות

המערכת כוללת מנגנון התחברות בסיסי המאפשר זיהוי משתמשים ובקרת גישה לקבצים. מנגנון זה נועד להבטיח שרק משתמשים מורשים יוכלו לבצע פעולות העלאה, הורדה וצפייה בקבצים.

העלאת קבצים (Upload)

כאשר המשתמש מעלה קובץ, הנתונים מועברים מהלקוח אל השרת באמצעות חיבור Socket. הקובץ מועבר בצורה רציפה כזרם נתונים (Data Stream), דבר המאפשר טיפול יעיל גם בקבצים גדולים יחסית.

דחיסת נתונים אוטומטית

לאחר שהקובץ מתקבל בשרת, מופעל עליו אלגוריתם הדחיסה LZ78. תהליך הדחיסה מתבצע בזמן אמת במהלך קבלת הנתונים, ללא צורך להמתין לסיום ההעברה המלאה של הקובץ. גישה זו מאפשרת לחסוך בזיכרון ולייעל את תהליך האחסון.

אחסון הקבצים

לאחר הדחיסה, הקובץ נשמר במערכת האחסון יחד עם מידע נוסף הנדרש לצורך זיהוי ושחזור הקובץ בעתיד, כגון שם הקובץ, גודל, סוג הקובץ ופרטי המשתמש שהעלה אותו.

פריסת נתונים (Decompression)

בעת בקשת הורדה, המערכת מבצעת תהליך פריסה המשחזר את המידע המקורי מתוך הנתונים הדחוסים. תהליך זה מבטיח שחזור מלא של הקובץ ללא אובדן מידע.

הורדת קבצים (Download)

לאחר תהליך הפריסה, הקובץ נשלח בחזרה ללקוח בצורה רציפה דרך חיבור התקשורת, כך שהמשתמש מקבל את הקובץ המקורי כפי שהועלה למערכת.

ביצועים עיקריים של המערכת

יעילות זמן ריצה

אלגוריתם הדחיסה והפריסה מבוסס על מעבר רציף על הנתונים, ולכן סיבוכיות הזמן של התהליך היא בקירוב ליניארית: $O(n)$

כאשר n מייצג את גודל הקלט.

יחס דחיסה

יעילות הדחיסה תלויה בסוג המידע המועבר. קבצי טקסט, קוד מקור וקבצים בעלי חזרתיות גבוהה מגיעים ליחס דחיסה טוב יחסית. לעומת זאת, קבצים שכבר דחוסים מראש, כמו ZIP או תמונות בפורמט JPEG, מציגים שיפור קטן יותר.

שימוש בזיכרון

המערכת משתמשת במילון דינמי המתעדכן במהלך תהליך הדחיסה. ככל שמספר הרצפים הייחודיים בקלט גדל, כך גם צריכת הזיכרון של האלגוריתם גדלה בהתאם.

זמן תגובה

המערכת מבצעת את תהליך הדחיסה תוך כדי העברת הנתונים, ולכן אין צורך להמתין לקבלת הקובץ המלא לפני תחילת העיבוד. גישה זו מאפשרת עבודה רציפה ומשפרת את זמן התגובה הכולל של המערכת.

התמודדות עם עומסים

המערכת תוכננה לעבוד עם קבצים בגדלים שונים ובמספר חיבורים במקביל. עם זאת, ביצועי הדחיסה וההעברה עשויים להשתנות בהתאם לעומס על השרת, מהירות הרשת וסוג הנתונים המועברים.

8.4 אילוצי המערכת

במהלך פיתוח המערכת הוגדרו מספר אילוצים טכנולוגיים ותפעוליים אשר משפיעים על אופן המימוש, ביצועי המערכת ויכולת ההרחבה שלה. אילוצים אלו נלקחו בחשבון במהלך תכנון הארכיטקטורה והפיתוח, במטרה לשמור על יציבות המערכת ופעולה תקינה תחת עומסים שונים.

מגבלת גודל קבצים

המערכת מוגבלת להעלאת קבצים בגודל של עד 7 GB. מגבלה זו נובעת משיקולי ביצועים, זמני עיבוד וניהול משאבי מערכת. טיפול בקבצים גדולים במיוחד דורש שימוש משמעותי בזיכרון ובמשאבי עיבוד, דבר שעלול להשפיע על יציבות השרת ועל זמני התגובה של המערכת.

מגבלת זיכרון ומשאבי חומרה

השרת פועל בסביבה הכוללת זיכרון RAM בנפח של GB16, ולכן קיימת מגבלה על כמות הפעולות שניתן לבצע במקביל. כאשר מספר משתמשים מעלים או מורידים קבצים בו־זמנית, צריכת המשאבים של המערכת גדלה באופן משמעותי, במיוחד בזמן ביצוע תהליכי דחיסה ופריסה. מסיבה זו היה צורך לנהל את הזיכרון בצורה יעילה ולשחרר משאבים באופן מבוקר לאחר סיום כל תהליך.

תלות בחיבור רשת יציב

פעולת המערכת מבוססת על תקשורת רציפה בין הלקוח לשרת, ולכן איכות חיבור הרשת משפיעה באופן ישיר על תהליך העלאת והורדת הקבצים. ניתוק או הפרעה במהלך ההעברה עלולים לגרום להפסקת התהליך, במיוחד כאשר מדובר בקבצים גדולים הדורשים זמן העברה ממושך.

מגבלות ביצועים בזמן העברת קבצים גדולים

בעת העלאת או הורדת של קבצים גדולים, זמן הביצוע של התהליך עשוי להתארך בהתאם למהירות הרשת, עומס השרת וגודל הקובץ. הסיבה לכך היא שהמערכת מבצעת מספר פעולות במקביל - קריאת הנתונים, דחיסה או פריסה, וכתיבה לאחסון - וכל אחת מהפעולות צורכת משאבי עיבוד וזיכרון.

התאמת ממשק המשתמש

הממשק הגרפי של המערכת פותח בעיקר עבור סביבת Desktop, ולכן חוויית השימוש מותאמת למסכים גדולים ולעבודה באמצעות מחשב אישי. שימוש דרך מכשירים ניידים או מסכים קטנים עשוי להיות פחות נוח מבחינת תצוגה ותפעול.

תלות במשאבי צד השרת

כל תהליכי הדחיסה, הפריסה וניהול הקבצים מתבצעים בצד השרת בלבד. כתוצאה מכך, ביצועי המערכת תלויים באופן ישיר בכוח העיבוד ובמשאבי החומרה של השרת. ככל שעומס המשתמשים גדל, כך עולה גם העומס על השרת ועל זמני העיבוד של המערכת.

9. תיאור הארכיטקטורה של המערכת

9.1 ארכיטקטורת המערכת

בסעיף זה מתואר המבנה הטכני של המערכת והאופן שבו רכיביה פועלים יחד לצורך מימוש תהליכי העברת ואחסון קבצים. התיאור נעשה מנקודת מבט הנדסית, ומתמקד בארכיטקטורת המערכת, בזרימת המידע בין רכיביה, בשילוב בין פרוטוקולי התקשורת ובאופן יישום הפתרון האלגוריתמי שנבחר. סעיף זה אינו עוסק בחוויית משתמש או בממשק גרפי, אלא בתכנון ובפעולה הפנימית של המערכת.

ארכיטקטורת שלוש השכבות (Three-Tier Architecture)

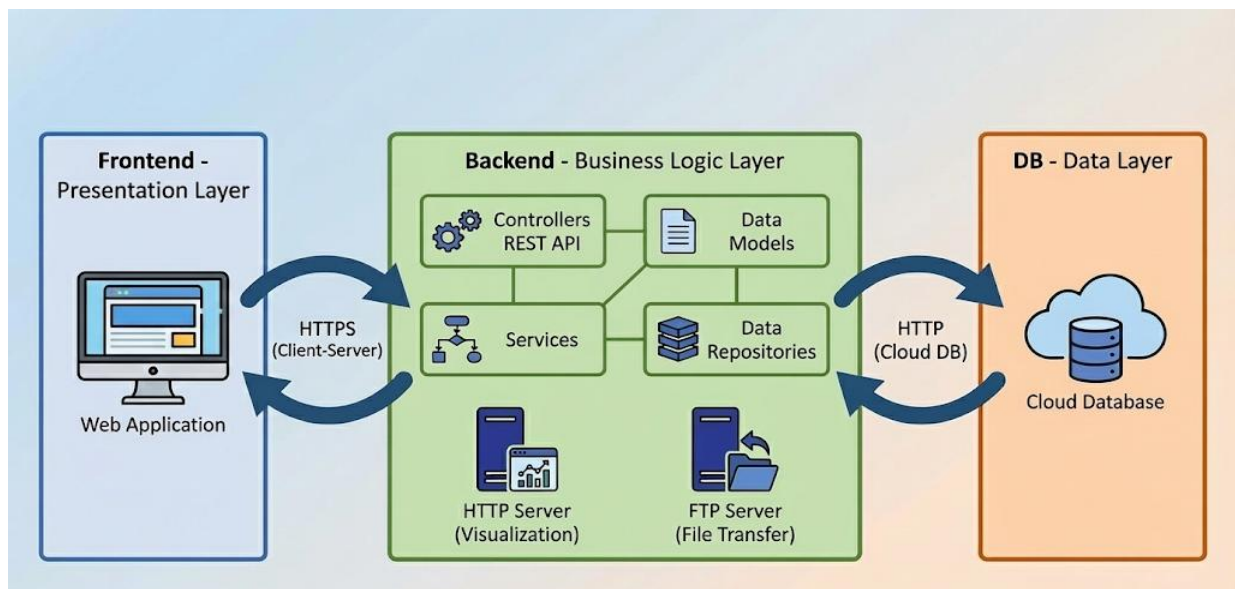
המערכת מבוססת על ארכיטקטורת שלוש שכבות, המאפשרת הפרדה ברורה של תחומי אחריות בין רכיבי המערכת. ארכיטקטורה זו נפוצה במערכות שרת-לקוח ומספקת סדר הנדסי, תחזוקה נוחה והפרדה בין שכבות העיבוד השונות.

ארכיטקטורת שלוש השכבות כוללת:

1. שכבת תצוגה (Presentation Layer)

2. שכבת לוגיקה יישומית (Application / Business Logic Layer)

3. שכבת נתונים (Data Layer)



חלוקה זו מאפשרת לכל שכבה לפעול בתחום אחריות מוגדר, תוך תקשורת מבוקרת עם שאר השכבות.

להלן הסבר על התרשים:

1. שכבת התצוגה (Presentation Layer)

שכבת התצוגה אחראית על יצירת נקודת הגישה למערכת ועל שליחת בקשות לפעולות כגון התחברות, העלאת קבצים והורדתם. שכבה זו ממומשת באמצעות ממשק Web ופועלת מעל פרוטוקול HTTP. מבחינה טכנית, שכבה זו משמשת כמתווך בלבד ואינה מבצעת עיבוד נתונים, דחיסה או ניהול אחסון, אלא שולחת בקשות לשכבת הלוגיקה היישומית ומציגה את התשובות המתקבלות ממנה.

2. שכבת הלוגיקה היישומית (Application / Business Logic Layer)

שכבת הלוגיקה היישומית מהווה את הליבה ההנדסית של המערכת. שכבה זו אחראית על ניהול תהליכי העבודה, קבלת החלטות וביצוע הפעולות המרכזיות של המערכת. במסגרת זו מתבצע התיאום בין שכבת התצוגה, שרת ה-FTP, שרת ה-HTTP ושכבת הנתונים. במערכת נעשה שימוש בשרת ה-HTTP המובנה Apache Tomcat, המשולב במסגרת Spring Boot ומשמש לניהול תקשורת ה-Web של האפליקציה. שרת זה אחראי על קבלת בקשות HTTP מהמשתמשים, ניהול סשנים, טעינת מסכי המערכת, טיפול בניווט בין עמודים, והעברת הבקשות אל שכבת הלוגיקה היישומית לצורך עיבוד הנתונים וביצוע הפעולות הנדרשות.

בשכבה זו מתבצעים:

- ניהול חיבורי משתמשים ואימות זהות
 - בדיקת הרשאות וגישה לקבצים
 - ניהול תהליכי העלאה והורדה
 - הפעלת תהליך דחיסת קבצים לפני אחסון
 - הפעלת תהליך פריסה (Decompression) לפני הורדה
 - ניהול בקשות HTTP ותקשורת Web מול המשתמשים
 - ניהול סשנים וניווט בין מסכי המערכת
- מנגנון הדחיסה משולב כחלק בלתי נפרד משכבת הלוגיקה היישומית, ופועל כתהליך פנימי התומך בפעולת שרת ה-FTP ומאפשר ייעול של נפח הנתונים המועברים והמאוחסנים.

3. שכבת הנתונים (Data Layer)

שכבת הנתונים אחראית על אחסון הקבצים והמידע הנלווה אליהם. בפרויקט זה, שכבה זו כוללת את מאגר הקבצים המאוחסנים בשרת, כאשר הקבצים נשמרים לאחר שעברו תהליך דחיסה. בנוסף, נשמר מידע הדרוש לניהול הקבצים ולשחזורם, כגון מבנה תיקיות ונתונים הנדרשים לתהליך הפריסה. שכבה זו אינה עוסקת בלוגיקה יישומית או בדחיסה, אלא מתמקדת באחסון אמין, מסודר ויעיל של המידע.

9.2 מודל שרת לקוח

המערכת מבוססת על מודל ארכיטקטוני מסוג שרת-לקוח (Client-Server), שהוא מודל נפוץ במערכות מבוזרות ובפרט במערכות ניהול קבצים והעברת מידע ברשת. במודל זה קיימת הפרדה ברורה בין הלקוח, אשר אחראי על שליחת בקשות בלבד, לבין השרת, אשר מרכז את כל עיבוד הנתונים, ניהול הקבצים והלוגיקה העסקית של המערכת.

הפרדה זו מאפשרת שליטה מלאה על המידע בצד השרת, שיפור אבטחת המידע, וניהול מרכזי של משאבי המערכת כגון אחסון, דחיסה והרשאות גישה.

צד הלקוח (Client Side)

צד הלקוח במערכת משמש כממשק אינטראקציה מול המשתמש. תפקידו העיקרי הוא לאפשר למשתמש לבצע פעולות כגון:

- התחברות למערכת
- העלאת קבצים
- הורדת קבצים
- קבלת מידע על מצב פעולות (כגון התקדמות העלאה)

הלקוח אינו מבצע עיבוד כבד של נתונים ואינו מבצע פעולות דחיסה או פריסה. כל פעולה לוגית או חישובית מועברת לביצוע בצד השרת. באופן זה נשמרת קלות תפעולית בצד הלקוח והמערכת נשארת נגישה ופשוטה לשימוש.

בנוסף, מאחר והמערכת מבוססת Web, צד הלקוח תלוי בביצועי הדפדפן ובחיבור הרשת, ולכן קיימת התאמה טובה יותר לשימוש במחשבים לעומת מכשירים ניידים.

צד השרת (Server Side)

צד השרת מהווה את הליבה המרכזית של המערכת ומרכז את כל הפעולות הקריטיות. הוא אחראי על:

- ניהול משתמשים והרשאות גישה
- קבלת קבצים מהלקוח
- הפעלת אלגוריתם הדחיסה (LZ78)
- שמירת הקבצים הדחוסים במערכת הקבצים
- ביצוע פריסה (Decompression) בעת הורדת קובץ
- ניהול משאבי מערכת ומעקב אחר עומסים

בפרויקט זה, השרת מהווה רכיב מרכזי במיוחד, שכן כל פעולת דחיסה והעברת קבצים מתבצעת בו בלבד. המשמעות היא שהעומס החישובי, כולל שימוש בזיכרון (RAM) ומשאבי CPU, מרוכז בצד השרת ולא בצד הלקוח.

9.3 תהליכים של מע' ההפעלה

המערכת פועלת מעל מערכת הפעלה סטנדרטית ומסתמכת על מנגנוני הרשת והניהול של מערכת ההפעלה לצורך הפעלת שרתים, תקשורת עם לקוחות וביצוע פעולות עיבוד קבצים. מערכת ההפעלה אחראית על ניהול תהליכים (Processes), הקצאת משאבי חומרה כגון זיכרון (RAM) ומעבד (CPU), וניהול תקשורת רשת באמצעות מנגנון Sockets.

במערכת זו פועלים שני שרתים מרכזיים במקביל:

- שרת FTP הפועל על פורט 2121
- שרת HTTP הפועל על פורט 8080

כל אחד מהשירותים מאזין (Listen) על פורט ייעודי, ומקבל בקשות נכנסות מצד לקוחות שונים.

שרת FTP (פורט 2121)

שרת ה-FTP משמש להעברת קבצים בין הלקוח לשרת. הוא מטפל בבקשות העלאה והורדה של קבצים, ומנהל את תהליך הדחיסה והפריסה כחלק מהעיבוד בצד השרת. התקשורת מתבצעת באמצעות חיבור Socket קבוע, המאפשר העברת נתונים בצורה רציפה עד סיום פעולת הקובץ. מערכת ההפעלה מנהלת את חיבורי ה-FTP כתהליכים נפרדים או חוטים (Threads), ומאפשרת טיפול במספר משתמשים במקביל בהתאם למשאבי המערכת.

שרת HTTP (פורט 8080)

שרת ה-HTTP משמש כממשק גישה למערכת דרך דפדפן אינטרנט. דרכו מתבצעות פעולות כגון התחברות למערכת, שליחת בקשות, וקבלת מידע על קבצים. שרת זה פועל באמצעות פרוטוקול HTTP מעל TCP, ומאזין על פורט 8080.

תפקיד מערכת ההפעלה הוא לנהל את הבקשות הנכנסות מהדפדפן, לפתוח חיבורים חדשים, ולהעביר את המידע לשכבת הלוגיקה היישומית של המערכת.

מנגנון תקשורת (Sockets ו-TCP)

התקשורת בין הלקוח לשרתים מתבצעת באמצעות Sockets, כאשר כל חיבור מבוסס על פרוטוקול TCP המבטיח העברת נתונים אמינה ושמירה על סדר המידע. כל בקשה נכנסת יוצרת תהליך או Thread נפרד המטופל על ידי מערכת ההפעלה.

תפקידי מערכת ההפעלה במערכת

מערכת ההפעלה אחראית על מספר פעולות מרכזיות במערכת:

- ניהול תהליכי שרת FTP ושרת HTTP במקביל
- הקצאת משאבי זיכרון ו-CPU לכל חיבור משתמש
- ניהול חיבורי רשת באמצעות כרטיס הרשת (Network Interface Card)
- פתיחת וניהול סוקטי תקשורת לכל משתמש מחובר
- תמיכה בהרצת מספר משתמשים בו-זמנית (Concurrency)

באופן זה, מערכת ההפעלה מהווה שכבת תשתית קריטית המאפשרת את פעולת המערכת בזמן אמת, תוך שמירה על יציבות, חלוקת עומסים ותקשורת תקינה בין כל רכיבי המערכת.

9.4 תיאור פרוטוקולי תקשורת

פרוטוקולי תקשורת מהווים את הבסיס להעברת מידע ברשתות מחשבים. כל פרוטוקול מגדיר כללים ברורים לאופן שבו נתונים נשלחים, מתקבלים ומטופלים בין רכיבים שונים במערכת. בפרויקטים של העברת ואחסון קבצים נעשה שימוש במספר פרוטוקולים, כאשר כל אחד מהם מותאם לסוג פעולה שונה.

• HTTP / HTTPS (HyperText Transfer Protocol)

HTTP הוא פרוטוקול תקשורת נפוץ הפועל במודל של בקשה-תגובה, שבו הלקוח שולח בקשה לשרת והשרת מחזיר תגובה מתאימה. הפרוטוקול משמש בעיקר לתקשורת מבוססת דפדפן ולהעברת מידע לוגי כגון טקסט, נתונים ופעולות ניהול. הגרסה המאובטחת, HTTPS, עושה שימוש בהצפנה על מנת להגן על המידע המועבר בין הלקוח לשרת ולמנוע יירוט או שינוי של הנתונים במהלך התקשורת. בפרויקט זה HTTP/HTTPS משמש כשכבת הגישה למערכת, דרכה המשתמש מבצע פעולות ניהול כגון התחברות, צפייה ברשימת קבצים והפעלת בקשות לוגיות, מבלי לעסוק ישירות בהעברת הקבצים עצמם.

• FTP (File Transfer Protocol)

FTP הוא פרוטוקול ייעודי להעברת קבצים ברשת, המאפשר פעולות כגון העלאה, הורדה, מחיקה וניהול תיקיות על גבי שרת מרוחק. הפרוטוקול פועל באמצעות שני ערוצי תקשורת נפרדים: ערוץ פיקוד (Control Channel) להעברת פקודות וערוץ נתונים (Data Channel) להעברת הקבצים בפועל. FTP תומך במספר מצבי עבודה, כגון Active ו-Passive. וכן במצבי העברת נתונים שונים, כאשר מטרתו העיקרית היא העברה אמינה ומבוקרת של קבצים.

בפרויקט זה FTP משמש כפרוטוקול הליבה להעברת הקבצים, והוא אחראי על ניהול האחסון, אכיפת הרשאות והעברת קבצים בין השרת ללקוח בצורה יעילה ומדויקת.

• **TCP/IP (Transmission Control Protocol / Internet Protocol)**

TCP/IP מהווה את שכבת התשתית של התקשורת ברשת. TCP אחראי על תקשורת אמינה, בקרת זרימה, שמירה על סדר החבילות ושליחה חוזרת במקרה של אובדן נתונים IP. אחראי על ניתוב המידע בין נקודות שונות ברשת. שילוב זה מאפשר העברת מידע יציבה ושלמה גם במצבים של עומס רשת או העברת קבצים גדולים.

בפרויקט זה TCP/IP משמש כתשתית שעליה פועלים פרוטוקולי HTTP ו-FTP, ומבטיח העברת נתונים אמינה בין רכיבי המערכת.

REST API (Representational State Transfer)

REST הוא סגנון ארכיטקטוני לתכנון ממשקי תקשורת בין מערכות. REST API מבוסס על שליחת בקשות HTTP לפעולות לוגיות, כאשר כל פעולה מיוצגת על ידי נקודת קצה (Endpoint).

ה-API משמש להעברת מידע בלבד, ללא תלות בממשק גרפי או במבנה דפי Web. נקודות קצה אופייניות מאפשרות פעולות כגון:

אימות משתמשים

קבלת מידע על משאבים

הפעלת תהליכים לוגיים

גישה זו מאפשרת תקשורת ברורה, מודולרית וסטנדרטית בין רכיבי מערכת שונים.

• **פורמט התקשורת- JSON**

JSON (JavaScript Object Notation) הוא פורמט טקסטואלי קל משקל לייצוג נתונים. הפורמט מבוסס על מבנה של זוגות מפתח-ערך ומאפשר ייצוג נוח של מידע מובנה כגון רשימות, אובייקטים ופרמטרים. JSON נפוץ מאוד בשימוש עם REST API בשל קריאותו, פשטותו והתאמתו להעברת מידע בין לקוח לשרת בצורה יעילה.

בפרויקט זה JSON משמש כפורמט להעברת מידע ניהולי ונתונים לוגיים בין רכיבי המערכת.

9.5 שירותים חיצוניים

במערכת נעשה שימוש בספריות קוד חיצוניות ובממשקי API לצורך מימוש תקשורת תקנית עם שרת FTP, משיכת נתונים דינמיים מרשת האינטרנט, וכן לשיפור היציבות, הפשטות והיעילות של פעולות עיבוד המידע באפליקציה.

שימוש ב- FTPClient - Apache Commons Net

בפרויקט נעשה שימוש בספריית Apache Commons Net, ובפרט ברכיב FTPClient, המאפשר עבודה עם פרוטוקול FTP בצורה מובנית וסטנדרטית. ספרייה זו מספקת פונקציות מוכנות לביצוע פעולות מרכזיות כגון: התחברות לשרת FTP באמצעות שם משתמש וסיסמה העלאת קבצים לשרת (Upload) הורדת קבצים מהשרת (Download) ניהול תיקיות וקבצים בשרת ניתוק בטוח מהשרת וניהול תקלות תקשורת באמצעות שימוש ב- FTPClient, נמנע הצורך במימוש ידני של פרוטוקול FTP ברמת ה-Socket, ונחסכת מורכבות רבה בפיתוח שכבת התקשורת של המערכת.

שילוב ה-API במערכת

ה- FTPClient משמש כשכבת תקשורת מרכזית בין ממשק המשתמש לבין שרת ה-FTP במערכת. כל פעולת העלאת, הורדה או שליפת קבצים מתבצעת דרך פונקציות הספרייה, אשר מנהלות את פתיחת החיבור, שליחת הנתונים וקבלת התשובות מהשרת בצורה שקופה למפתח.

שימוש ב- HttpClient - Open-Meteo

בפרויקט נעשה שימוש בשירות הרשת הציבורי Open-Meteo ובמנגנון HttpClient המובנה של שפת Java, המאפשרים עבודה עם פרוטוקול HTTP לצורך קבלת נתוני מזג אוויר בזמן אמת בצורה מובנית וסטנדרטית.

רכיבים אלו מספקים יכולות מוכנות לביצוע פעולות מרכזיות כגון:

שליחת בקשות HTTP GET לשרת חיצוני לפי קווי אורך ורוחב

משיכת נתוני טמפרטורה עדכניים בזמן אמת (Real-Time Weather Data)

קבלת תשובות בפורמט JSON לעיבוד מהיר במערכת

עבודה מול שירות REST API ללא צורך במפתחות אבטחה (No Authentication)

באמצעות שימוש ב- HttpClient, נמנע הצורך בתחזוקת מסד נתונים מקומי עבור נתוני אקלים, ונחסכת תקורת פיתוח רבה בהבאת מידע חיצוני ודינמי למערכת.

שילוב ה-API במערכת

ה- HttpClient משמש כצינור תקשורת ישיר בין המערכת לבין שרתי Open-Meteo. כאשר המשתמש בוחר עיר או אזור מסוים בממשק, המערכת שולחת בקשת HTTP אל שירות ה-API, מקבלת תשובת JSON, ומעבדת את הנתונים לצורך הצגת הטמפרטורה והמידע העדכני בממשק המשתמש.

שימוש ב־org.json

בפרויקט נעשה שימוש בספריית org.json, ובפרט ברכיבים JSONObject ו-JSONArray, המאפשרים פענוח, עיבוד וקריאת נתונים במבנה JSON בצורה מובנית וסטנדרטית. ספרייה זו מספקת פונקציות מוכנות לביצוע פעולות מרכזיות כגון: המרת מחרוזות JSON לאובייקטים מובנים בזיכרון חילוץ נתונים מתוך מבנים היררכיים בצורה פשוטה ויעילה קריאת מערכים, אובייקטים וערכים בודדים מתוך תשובות שרת גישה נוחה ומהירה לשדות מידע ללא צורך במנגנוני Parsing ידניים באמצעות שימוש ב־org.json, נמנע הצורך בכתיבה ידנית ומסורבלת של קוד לפענוח מחרוזות JSON, ונחסכת מורכבות רבה בעיבוד הנתונים המתקבלים מה-API החיצוני.

שילוב ה-API במערכת

ה־org.json משמש כשכבת עיבוד מידע בין תשובת שרת Open-Meteo לבין ממשק המשתמש במערכת. כל תשובת JSON שמתקבלת מהרשת מפוענחת באמצעות הספרייה, ולאחר מכן הנתונים הרלוונטיים מועברים להצגה במסכים השונים באפליקציה.

10. ניתוח תרחישים וזרימת המידע במערכת המוצעת

10.1 תרשימי תרחיש Use case Diagram

להלן תרשימי תרחיש Use Case של התהליכים המרכזיים במערכת התוכנה:

1. יצירת משתמש

מקרה שימוש זה מתאר את תהליך ההרשמה למערכת.

המשתמש מזין פרטים בסיסיים הנדרשים לצורך יצירת חשבון חדש, כגון שם משתמש וסיסמה. לאחר הזנת הנתונים, המערכת מבצעת בדיקות תקינות ומוודאת ששם המשתמש אינו קיים כבר במערכת.

במידה והנתונים תקינים ותהליך האימות מסתיים בהצלחה, המערכת יוצרת חשבון חדש ושומרת את פרטי המשתמש. לאחר מכן המשתמש יכול להתחבר למערכת באמצעות פרטי ההזדהות שהוגדרו.

2. התחברות

מקרה שימוש זה מתאר את תהליך הכניסה למערכת. המשתמש מזין שם משתמש וסיסמה, והמערכת בודקת את התאמת הנתונים למידע הקיים במערכת. במידה והפרטים נכונים, המשתמש מקבל גישה לאזור האישי שלו ויכול להשתמש בפונקציות המרכזיות של המערכת, כגון העלאת והורדת קבצים.

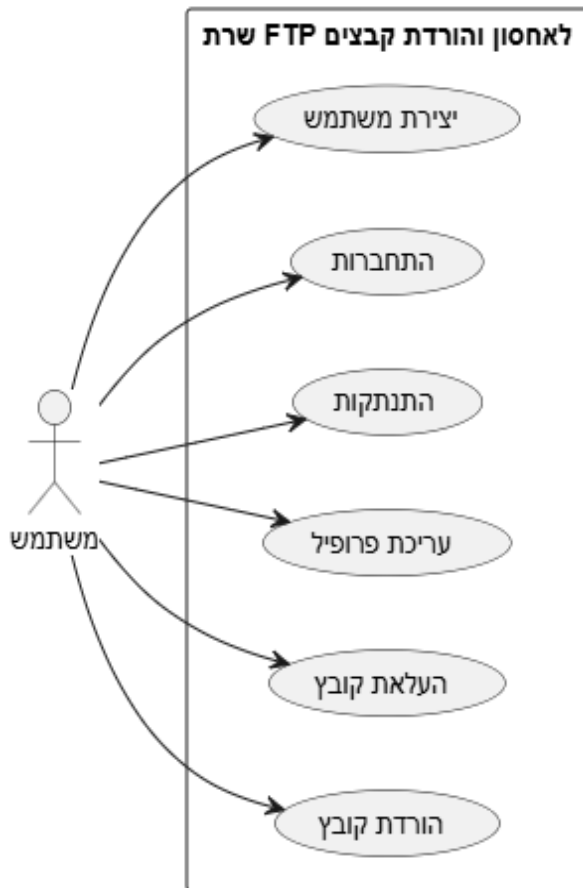
בנוסף, לאחר התחברות מוצלחת המערכת יוצרת ששן פעיל המאפשר זיהוי רציף של המשתמש במהלך העבודה.

3. התנתקות

מקרה שימוש זה מתאר את סיום העבודה של המשתמש במערכת. כאשר המשתמש בוחר להתנתק, המערכת סוגרת את החיבור הפעיל ומסיימת את הסשן הנוכחי בצורה מסודרת. פעולה זו נועדה לשמור על אבטחת המידע ולמנוע גישה לא מורשית לחשבון לאחר סיום השימוש במערכת.

4. עריכת פרופיל

מקרה שימוש זה מתאר עדכון פרטי משתמש במערכת. המשתמש יכול לשנות ולעדכן פרטים אישיים שונים, כגון סיסמה או פרטים מזהים אחרים. לאחר ביצוע השינויים, המערכת בודקת את תקינות הנתונים החדשים ושומרת אותם במערכת. תהליך זה מאפשר למשתמש לנהל ולעדכן את פרטי החשבון שלו בצורה נוחה ומאובטחת.



5. העלאת קובץ

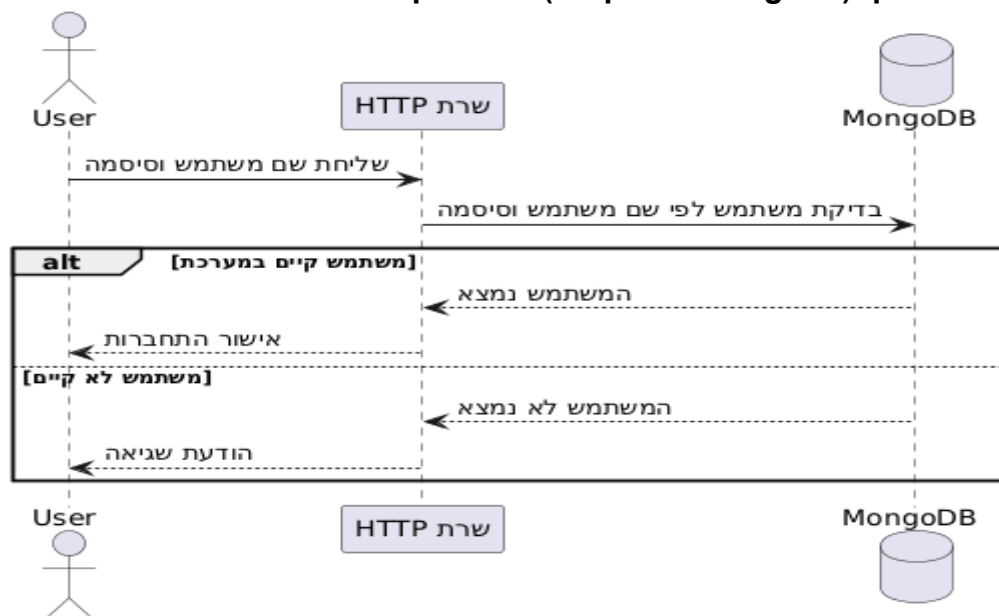
מקרה שימוש זה מתאר את תהליך העלאת הקבצים לשרת. המשתמש בוחר קובץ מהמחשב האישי ומבקש להעלות אותו למערכת. לאחר קבלת הבקשה, המערכת בודקת שלמשתמש קיימת הרשאה לבצע את הפעולה. בשלב הבא הקובץ מועבר לשרת ועובר תהליך דחיסה באמצעות אלגוריתם הדחיסה של המערכת, במטרה לצמצם את נפח המידע לפני האחסון. לאחר השלמת תהליך הדחיסה, הקובץ נשמר בשרת והמערכת מציגה למשתמש הודעה המאשרת שהפעולה הסתיימה בהצלחה.

6. הורדת קובץ

מקרה שימוש זה מתאר את תהליך הורדת הקבצים מהמערכת אל מחשב המשתמש. המשתמש בוחר קובץ מתוך רשימת הקבצים הזמינים עבורו ומבקש להוריד אותו. המערכת בודקת שלמשתמש קיימת הרשאת גישה לקובץ המבוקש, ולאחר מכן שולפת את הקובץ מהאחסון. לפני שליחת הקובץ למשתמש, המערכת מבצעת תהליך פריסה (Decompression) המשחזר את הקובץ למצבו המקורי. בסיום התהליך הקובץ מועבר למחשב המשתמש בצורה מלאה ותקינה.

10.2 תרשימי רצף Sequence Diagram

1. להלן תרשים רצף (Sequence Diagram) של תהליך ההתחברות למערכת:



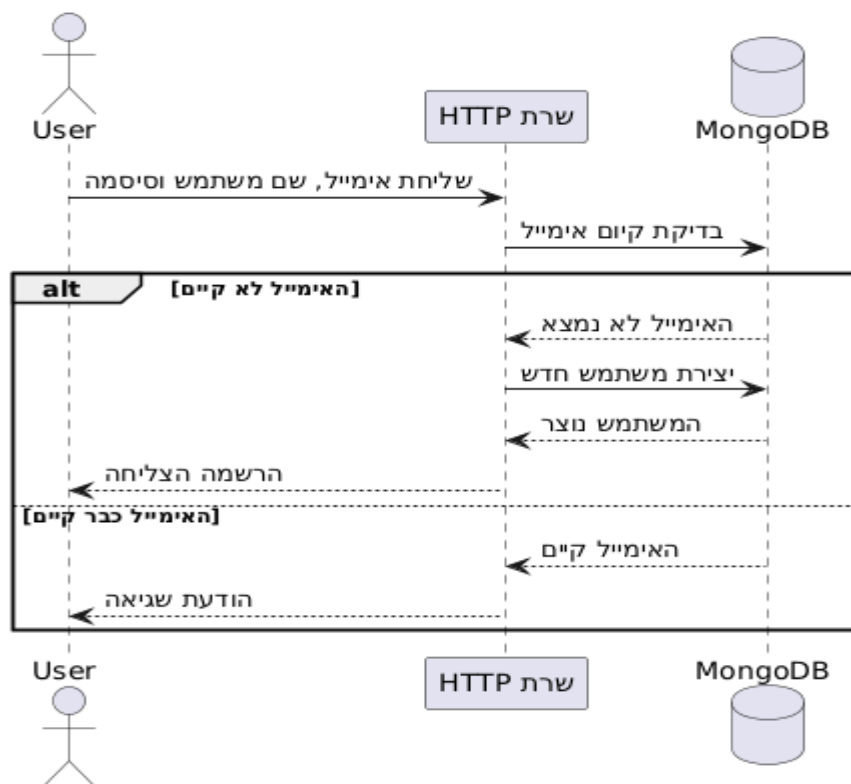
תהליך ההתחברות מתחיל כאשר המשתמש מזין את שם המשתמש והסיסמה שלו דרך ממשק המערכת ושולח בקשת התחברות לשרת. לאחר קבלת הבקשה, שרת ה-HTTP מקבל את פרטי ההתחברות ומעביר שאלתה למסד הנתונים לצורך בדיקת קיום משתמש בעל פרטים תואמים.

מסד הנתונים מבצע חיפוש אחר המשתמש ובודק האם שם המשתמש והסיסמה תואמים לנתונים השמורים במערכת.

במידה והנתונים תקינים ונמצא משתמש מתאים, מסד הנתונים מחזיר לשרת תשובת אימות חיובית. לאחר מכן השרת יוצר סשן התחברות פעיל ומחזיר למשתמש הודעת התחברות מוצלחת, המאפשרת גישה לאזור האישי ולשירותי המערכת.

במקרה שבו לא נמצא משתמש תואם או שהוזנו פרטים שגויים, מסד הנתונים מחזיר תשובת כישלון לשרת, והמערכת מציגה למשתמש הודעת שגיאה מתאימה המודיעה כי פרטי ההתחברות אינם תקינים. תרשים זה מדגים את זרימת המידע בין רכיבי המערכת ואת אופן ניהול תהליך האימות בצורה מסודרת ומאובטחת.

2. להלן תרשים רצף (Sequence Diagram) של תהליך ההרשמה למערכת:



תהליך ההרשמה מתחיל כאשר המשתמש מזין את כתובת האימייל, שם המשתמש והסיסמה שלו דרך ממשק המערכת ושולח בקשת הרשמה לשרת.

לאחר קבלת הבקשה, שרת ה-HTTP מקבל את פרטי המשתמש ומבצע בדיקה מול מסד הנתונים כדי לוודא שלא קיים כבר משתמש עם אותה כתובת אימייל או אותם פרטי זיהוי.

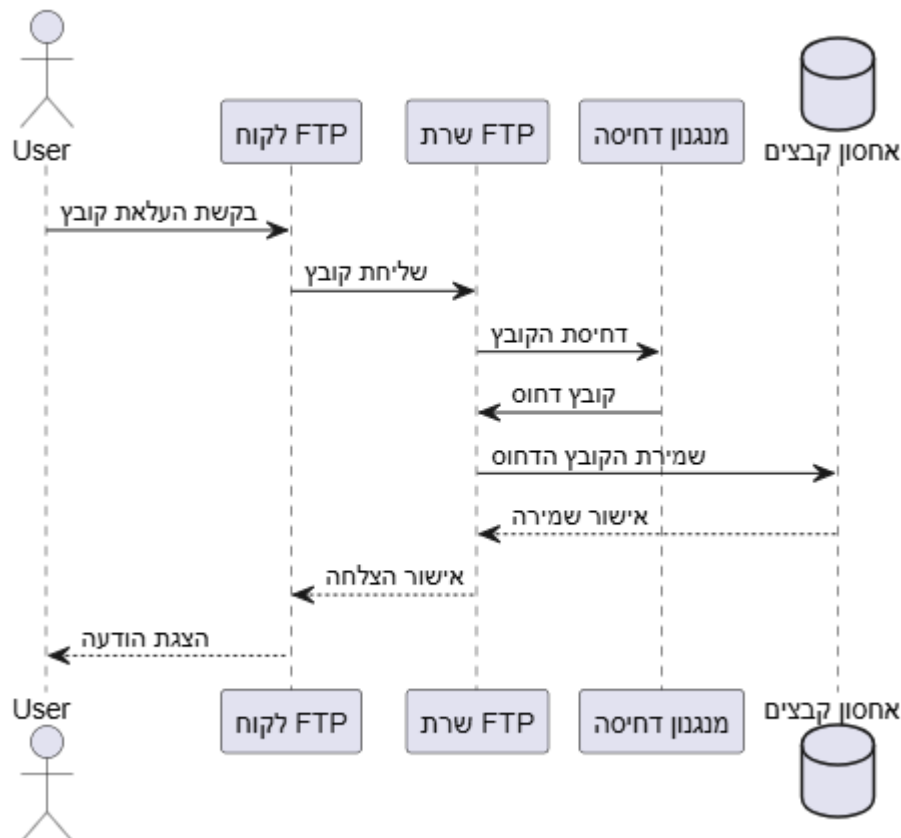
מסד הנתונים מחפש משתמש קיים בהתאם לנתונים שהתקבלו ומחזיר תשובה לשרת.

במידה שלא נמצא משתמש בעל אותה כתובת אימייל, השרת יוצר משתמש חדש ושומר את פרטי החשבון במסד הנתונים. לאחר השלמת הפעולה, מוחזרת למשתמש הודעת הרשמה מוצלחת המאשרת שהחשבון נוצר בהצלחה.

לעומת זאת, אם נמצא משתמש קיים עם אותה כתובת אימייל, מסד הנתונים מחזיר תשובת כישלון לשרת, והמערכת מציגה למשתמש הודעת שגיאה מתאימה המודיעה כי האימייל כבר קיים במערכת ולכן לא ניתן להשלים את תהליך ההרשמה.

תרשים זה מדגים את אופן ניהול תהליך יצירת המשתמש במערכת ואת זרימת המידע בין רכיבי המערכת השונים במהלך תהליך ההרשמה.

3. להלן תרשים רצף (Sequence Diagram) של תהליך העלאת קובץ למערכת:



תרשים הרצף מתאר את רצף הפעולות בתהליך העלאת קובץ למערכת, ואת האינטראקציה בין הרכיבים השונים: משתמש, לקוח, שרת, FTP, מנגנון הדחיסה ושכבת האחסון.

תהליך העלאת הקובץ מתחיל כאשר המשתמש בוחר קובץ מהמחשב האישי ושולח בקשת העלאה באמצעות לקוח ה-FTP.

לקוח ה-FTP יוצר חיבור מול שרת ה-FTP ומתחיל להעביר את הקובץ כזרם נתונים רציף. לאחר קבלת הנתונים, שרת ה-FTP מעביר את הקובץ למנגנון הדחיסה לצורך הקטנת נפח המידע לפני שלב האחסון.

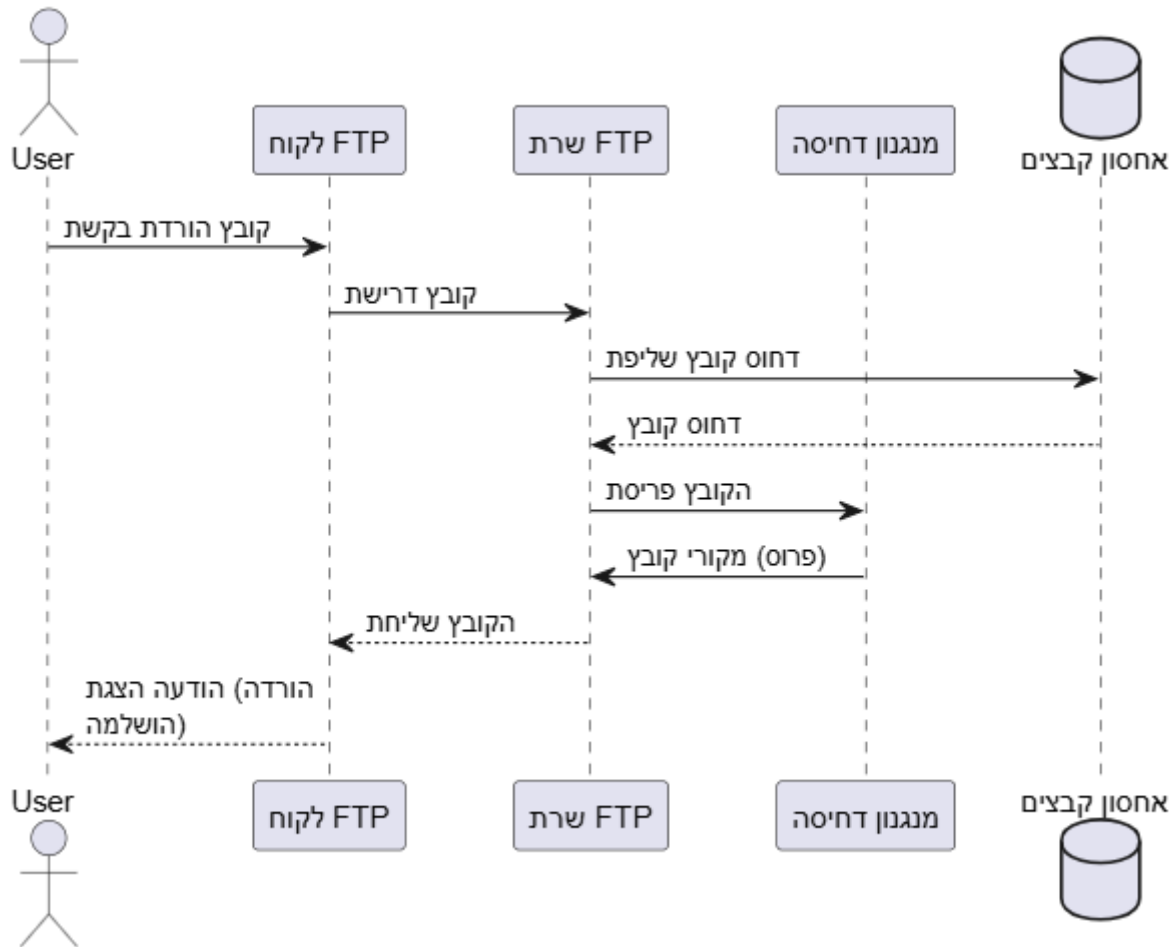
מנגנון הדחיסה מבצע עיבוד לקובץ באמצעות אלגוריתם הדחיסה שנבחר במערכת ומחזיר לשרת את המידע לאחר תהליך הדחיסה.

לאחר השלמת תהליך הדחיסה, השרת שומר את הקובץ הדחוס בשכבת האחסון יחד עם המידע הנדרש לניהול הקובץ ולשחזורו בעתיד.

בסיום התהליך שכבת האחסון מחזירה לשרת אישור על שמירה תקינה של הקובץ, והשרת מחזיר הודעת הצלחה ללקוח ה-FTP. לאחר מכן המשתמש מקבל הודעה המאשרת שהקובץ הועלה ונשמר במערכת בהצלחה.

תרשים זה מדגים את זרימת המידע בין רכיבי המערכת ואת השילוב של מנגנון הדחיסה כחלק מרכזי בתהליך העלאת הקבצים ואחסונם.

4. להלן תרשים רצף (Sequence Diagram) של תהליך הורדת קובץ למערכת:



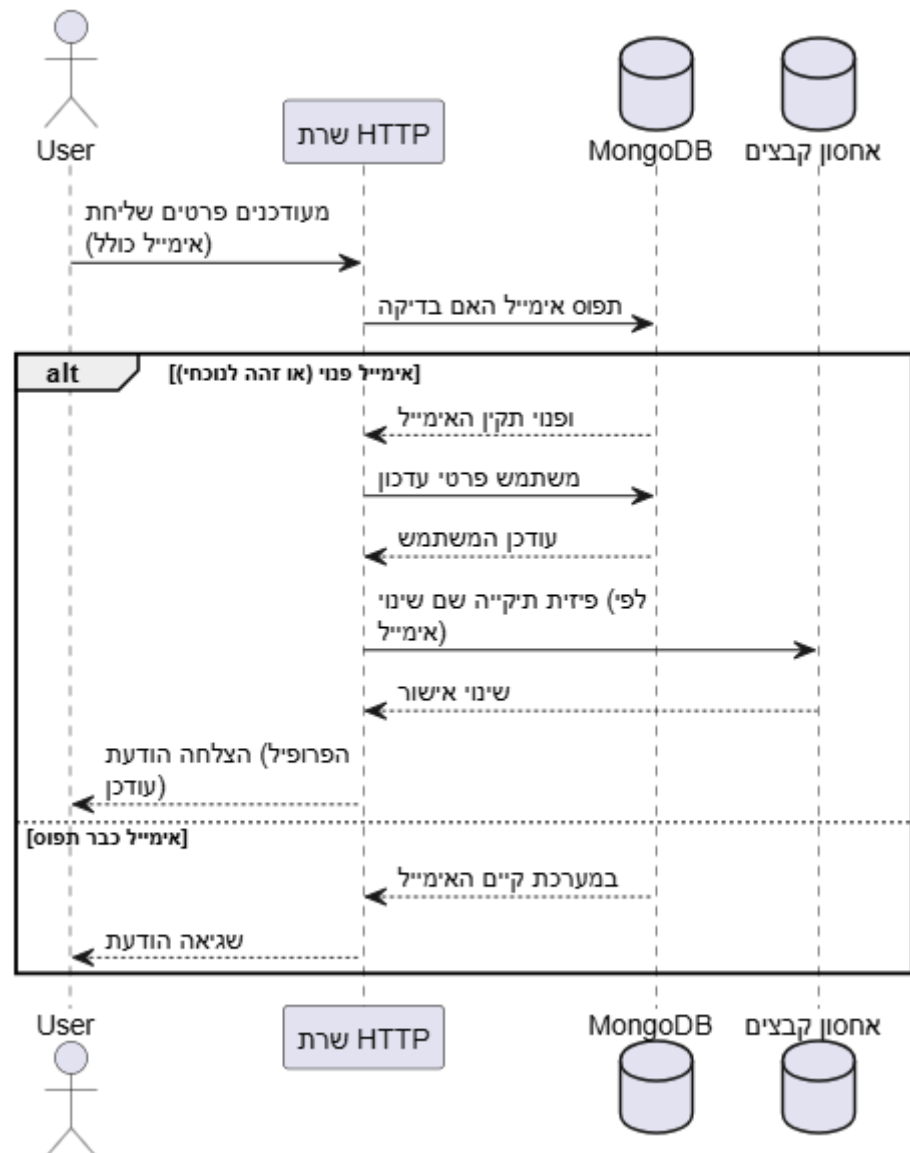
תרשים הרצף מתאר את תהליך הורדת הקובץ מהמערכת ואת זרימת המידע בין רכיבי המערכת השונים: המשתמש, לקוח ה-FTP, שרת ה-FTP, מנגנון הדחיסה ושכבת האחסון.

התהליך מתחיל כאשר המשתמש שולח בקשה להורדת קובץ דרך לקוח ה-FTP. לקוח ה-FTP מעביר את בקשת ההורדה לשרת ה-FTP, אשר פונה לשכבת האחסון לצורך שליפת הקובץ הדחוס השמור במערכת.

לאחר קבלת הקובץ הדחוס, שרת ה-FTP מעביר אותו למנגנון הדחיסה לצורך ביצוע תהליך פריסה (Decompression) ושחזור הקובץ המקורי. לאחר השלמת תהליך הפריסה, הקובץ המקורי מוחזר לשרת ה-FTP ונשלח בחזרה ללקוח ה-FTP.

בסיום התהליך המשתמש מקבל את הקובץ המלא ומוצגת הודעה המאשרת כי ההורדה הושלמה בהצלחה.

5. להלן תרשים רצף (Sequence Diagram) של תהליך עריכת פרופיל:



תרשים הרצף מתאר את תהליך עדכון פרופיל המשתמש במערכת ואת זרימת המידע בין המשתמש, שרת ה-HTTP, מסד הנתונים MongoDB ושכבת אחסון הקבצים.

התהליך מתחיל כאשר המשתמש שולח פרטים מעודכנים למערכת, כגון שם משתמש או כתובת אימייל חדשה. שרת ה-HTTP מקבל את הבקשה ופונה למסד הנתונים כדי לבדוק האם כתובת האימייל החדשה כבר קיימת במערכת.

במידה שהאימייל פנוי, או שמדובר באותו האימייל הקיים של המשתמש, מסד הנתונים מחזיר אישור לשרת והמערכת מעדכנת את פרטי המשתמש. לאחר מכן מתבצע גם שינוי של שם תיקיית האחסון

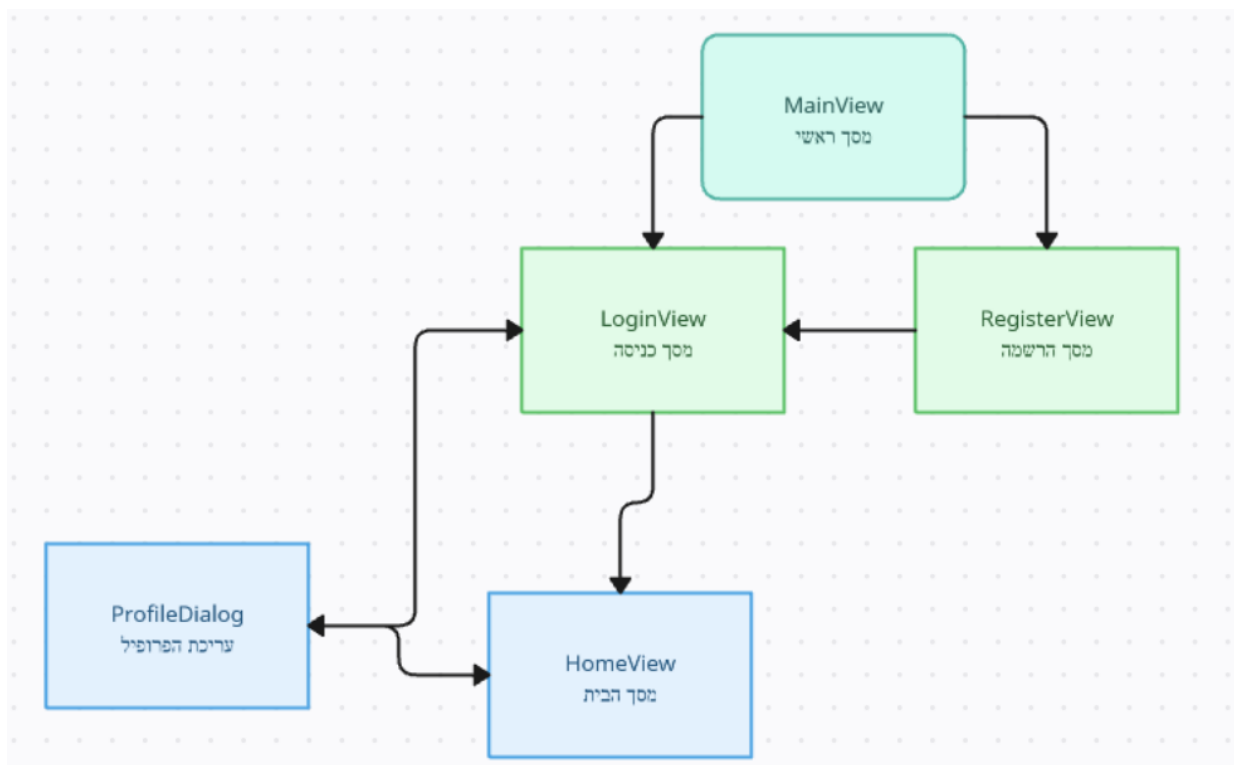
הפיזית המשוּיכת למשתמש, כדי לשמור על התאמה בין נתוני המשתמש לבין הקבצים השמורים במערכת.

לאחר השלמת כל פעולות העדכון, המערכת מחזירה למשתמש הודעת הצלחה המאשרת כי הפרופיל עודכן בהצלחה.

לעומת זאת, אם כתובת האימייל כבר קיימת במערכת, מסד הנתונים מחזיר תשובת כישלון, והמערכת מציגה למשתמש הודעת שגיאה המודיעה שלא ניתן לעדכן את הפרופיל משום שהאימייל כבר תפוס.

11. תיאור מסכים וממשק משתמש

11.1 תרשים היררכיית המסכים



התרשים מציג את מבנה הניווט במערכת ואת מנגנון בקרת הגישה (Authentication Flow) בין מסכי האפליקציה.

המערכת מחולקת לשני אזורים מרכזיים:

אזור ציבורי (Public Area) - נגיש לכל משתמש גם ללא התחברות.

אזור פרטי (Protected Area) - נגיש רק למשתמשים שעברו אימות בהצלחה.

התרשים מתאר את מעבר המשתמש בין המסכים ואת תהליך האימות מול מסד הנתונים.

MainView / StartView - מסך הפתיחה

זהו המסך הראשי והראשון אליו מגיע כל משתמש בעת כניסה למערכת.

מסך זה משמש כ- Landing Page של האפליקציה ומציג מידע כללי על המערכת, מטרת הפרויקט ואפשרויות ניווט למסכי ההתחברות וההרשמה.

מתוך מסך זה המשתמש יכול:

- לעבור למסך ההתחברות (LoginView)
- לעבור למסך ההרשמה (RegisterView)

LoginView - מסך התחברות

במסך זה המשתמש מזין את פרטי ההתחברות שלו (שם משתמש וסיסמה).

לאחר לחיצה על כפתור ההתחברות, המערכת שולחת בקשה לבדיקת תקינות הנתונים מול שכבת הנתונים (UserRepository).

UserRepository - מנגנון אימות משתמשים

היהלום בתרשים מייצג נקודת החלטה לוגית (Decision Node) במערכת.

בשלב זה המערכת בודקת האם פרטי ההתחברות שהוזנו תואמים למשתמש קיים במסד הנתונים.

התהליך מתפצל לשני תרחישים:

התחברות מוצלחת – "אם קיים משתמש"

אם פרטי ההתחברות תקינים:

- המערכת מאמתת את המשתמש
- נוצר Session פעיל עבורו
- המשתמש מועבר לאזור המאובטח של המערכת (HomeView)

התחברות נכשלה – "אם לא קיים משתמש"

אם פרטי ההתחברות אינם תקינים או שהמשתמש אינו קיים במערכת:

- המשתמש הולך למסך ההרשמה (RegisterView)
- שם ניתן ליצור חשבון חדש

RegisterView - מסך הרשמה

מסך זה מאפשר למשתמש ליצור חשבון חדש במערכת באמצעות הזנת פרטים כגון:

- שם משתמש
- כתובת אימייל
- סיסמה

לאחר הרשמה מוצלחת, המשתמש מועבר בחזרה למסך ההתחברות (LoginView) כדי להתחבר עם החשבון החדש שיוצר.

HomeView - האזור האישי של המשתמש

זהו המסך המרכזי והמאובטח של המערכת, אליו ניתן לגשת רק לאחר אימות מוצלח.

במסך זה המשתמש יכול לבצע פעולות ניהול קבצים כגון:

- העלאת קבצים
- הורדת קבצים
- מחיקת קבצים
- צפייה בטבלת הקבצים האישית שלו

מסך זה מהווה את סביבת העבודה המרכזית של מערכת אחסון הענן.

ProfileDialog - חלון פרופיל משתמש

ProfileDialog הוא חלון קופץ (Dialog / Modal Window) הנפתח מתוך HomeView.

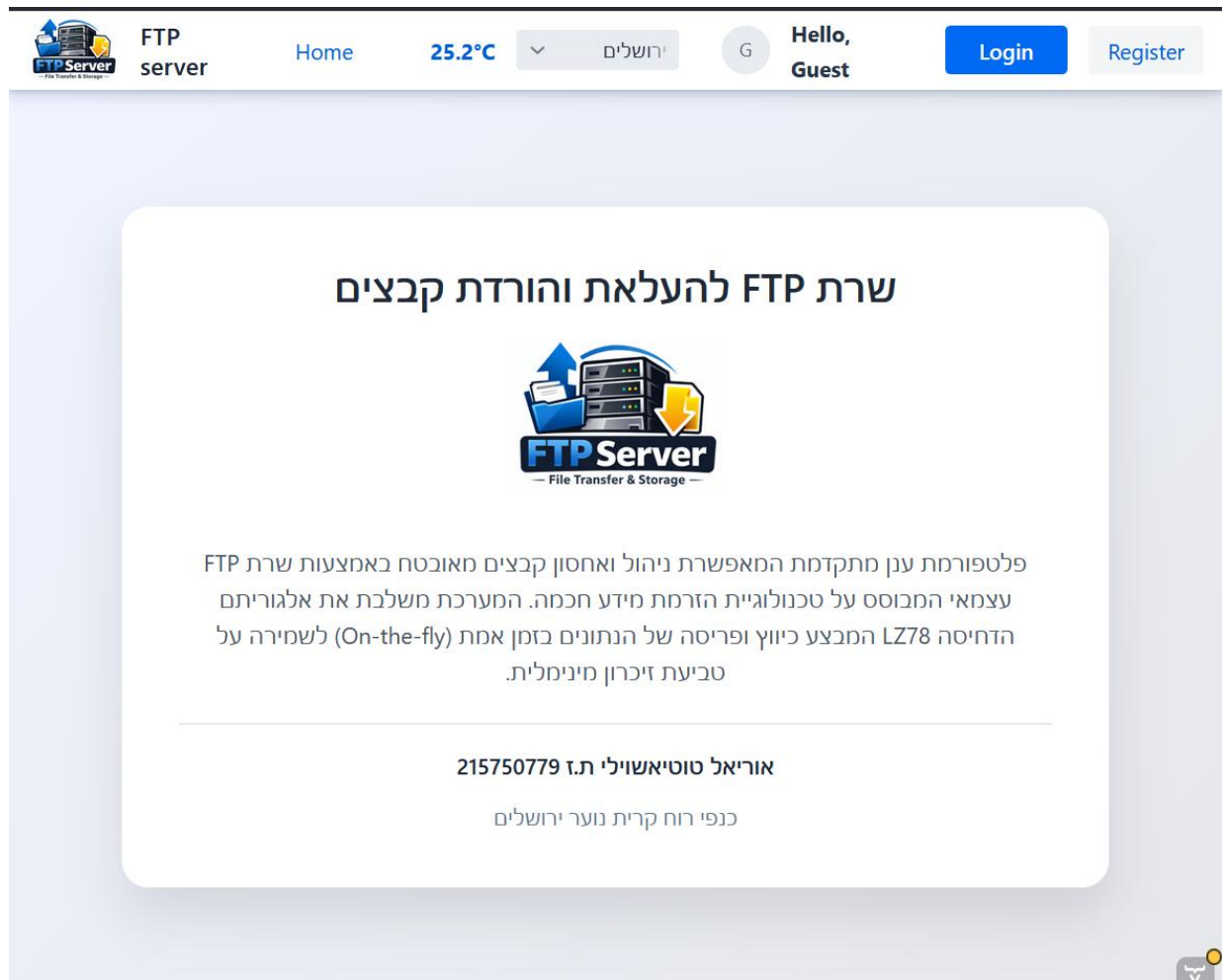
חלון זה מאפשר למשתמש:

- לצפות בפרטי החשבון שלו
- לערוך מידע אישי
- לבצע פעולות הקשורות לפרופיל המשתמש

החץ הדו-כיווני בין HomeView ל-ProfileDialog מציין כי מדובר בחלון משני הנפתח מעל המסך הראשי ונסגר מבלי לצאת מהאזור האישי של המשתמש או לאבד את מצב העבודה הנוכחי.

11.2 תיאור המסכים

מסך הראשי MainView:



המסך משמש כדף הבית הראשי ונקודת הכניסה למערכת עבור משתמשים שאינם מחוברים. מטרתו להציג בצורה ברורה את האפליקציה (אחסון ענן חכם המבוסס על כיווץ LZ78) ולהפנות את המשתמש להתחברות או הרשמה.

רכיבי UI ותפקידם:

תמונת לוגו מציגה את סמל המערכת ומחזקת את המיתוג. כותרת ראשית ותת-כותרת מציגות את שם הפרויקט ותיאור קצר על מה הפרויקט. אזור מאפיינים מציג את יכולות המערכת כמו שימוש ב-LZ78 וניהול קבצים דרך FTP.

הודעות ודיאלוגים:

במסך זה אין הודעות קופצות או דיאלוגים, מכיוון שהוא משמש כמסך פתיחה בלבד לניווט ראשוני במערכת.

מסך כניסה LoginView:

 FTP server

Home

25.2°C

▼ ירושלים

G

Hello, Guest

Login

Register

התחברות

Log in

Username •

Password •



Log in

[Forgot password](#)

המסך משמש לאימות זהות (Authentication) של משתמשים רשומים במערכת. הוא בודק שהשם משתמש והסיסמה שהוזנו תואמים לנתונים השמורים במסד הנתונים (MongoDB), כדי לאפשר כניסה מאובטחת לאזור האיש.

רכיבי UI ותפקידם:

כותרת H1 מציגה "התחברות" להכוונת המשתמש. טופס התחברות כולל שדה שם משתמש, שדה סיסמה מאובטח וכפתור כניסה שמבצע את תהליך האימות מול השרת.

הודעות ודיאלוגים:

במקרה של שגיאה בהזנת פרטים מוצגת הודעת שגיאה בתוך הטופס (באדום) המציינת שהנתונים אינם נכונים. במקרה של הצלחה מוצגת הודעת הצלחה קצרה, ולאחריה המשתמש מועבר אוטומטית למסך הראשי (HomeView).

מסך הרשמה RegisterView:

 FTP server

Home

25.2°C

▼ ירושלים

G

Hello, Guest

Login

Register

Create Account

Username •

Email •

Password •

Register

[Already have an account? Log in](#)



המסך משמש ליצירת חשבון משתמש חדש במערכת. הוא קולט פרטי משתמש בסיסיים (שם, אימייל וסיסמה), בודק שהאימייל אינו קיים כבר במערכת, ושומר את המשתמש החדש במסד הנתונים (MongoDB).

רכיבי UI ותפקידם:

כותרת H1 מציגה "הרשמה" לזיהוי הדף. שדה אימייל מאפשר הזנת כתובת דוא"ל עם בדיקת תקינות מובנית, שדה שם משתמש לקליטת שם תצוגה, ושדה סיסמה מאובטח שמסתיר את התווים. כפתור "הרשם" מפעיל את תהליך הבדיקה והשמירה מול ה-UserService, וכפתור "חזור" מחזיר למסך ההתחברות.

הודעות ודיאלוגים:

במקרה של שגיאה (אימייל קיים או שדות חסרים) מוצגת הודעת אזהרה בטופס או כ-Notification. במקרה של הצלחה מוצגת הודעה קצרה "נרשמת בהצלחה", ולאחר מכן המשתמש מועבר אוטומטית למסך ההתחברות.

 FTP server

[Main](#) [My Cloud](#)

25.2°C

ירושלים

u

Hello, uriel

[Profile](#)

[Logout](#)

My FTP Storage

[Upload Files...](#)  Drag files here

File Name	Type	Size	Upload Date	Actions
Batman_29_icon.jpg	JPG	32 KB	2026-05-27T10:51:14.989	Download Delete
Pictures.rar	RAR	32 KB	2026-05-27T10:51:43.883	Download Delete

זהו לוח הבקרה הראשי (Dashboard) של המשתמש לאחר ההתחברות. המסך משמש כאזור העבודה המרכזי בענן, ומאפשר ניהול קבצים: העלאה (עם דחיסה אוטומטית ב-LZ78), צפייה בקבצים קיימים, הורדה למחשב (עם פריסה בזמן אמת) ומחיקה.

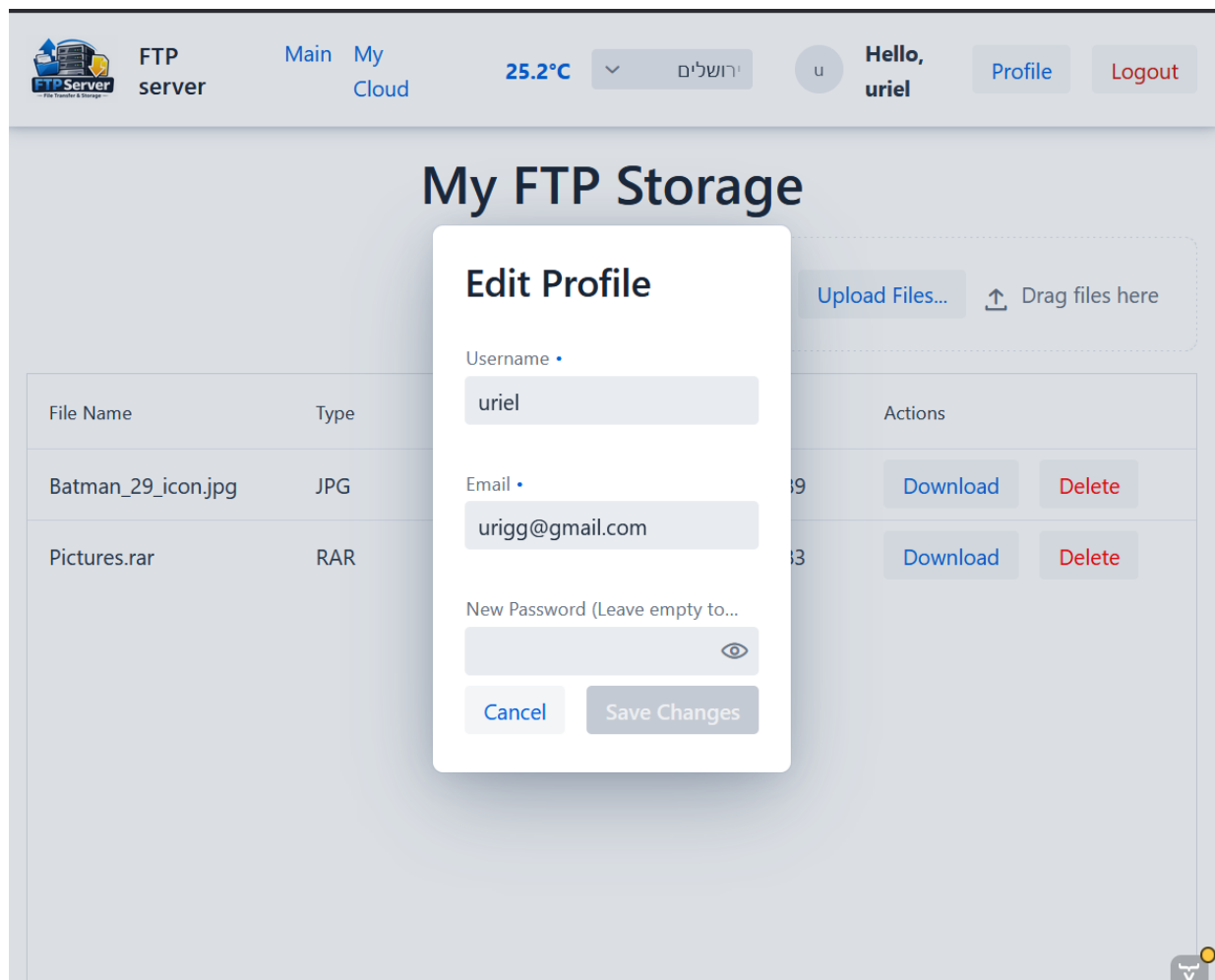
רכיבי UI ותפקידם:

כותרת H1 מציגה את אזור האחסון של המשתמש. רכיב העלאת קבצים מאפשר בחירה או גרירה של קבצים ומעביר אותם לשרת לצורך עיבוד. טבלת קבצים מציגה את כלל הקבצים של המשתמש עם פרטים כמו שם, סוג, גודל ותאריך העלאה. בכל שורה קיימים כפתורי הורדה שמוריד את הקובץ דרך בקשת HTTP, וכפתור מחיקה שמסיר את הקובץ מהשרת ומהמסד נתונים.

הודעות ודיאלוגים:

בעת ניסיון להעלות קובץ עם שם קיים מתקבלת הודעת אזהרה. לאחר העלאה מוצגת הודעת הצלחה והטבלה מתעדכנת. בעת מחיקה מוצגת הודעת סטטוס המאשרת הצלחה או כישלון של הפעולה.

מסך עריכת פרופיל ProfileDialog:



זהו חלון קופץ מודאלי (Modal Dialog) שנפתח ממסך הבית כאשר המשתמש בוחר באפשרות הפרופיל. מטרתו לאפשר עדכון מאובטח של פרטי החשבון (שם משתמש, אימייל וסיסמה), תוך שליטה מלאה על שמירת השינויים.

רכיבי UI ותפקידם:

שדה שם משתמש מאפשר עריכה של השם הקיים, ושדה אימייל מאפשר עדכון כתובת דוא"ל עם בדיקת תקינות מובנית. כפתור שינוי סיסמה מציג או מסתיר את שדה הסיסמה החדשה לפי צורך. שדה הסיסמה עצמו מוצג בצורה מוסתרת ומאפשר הזנת סיסמה חדשה. כפתור שמירה מבצע עדכון מול השרת ומופעל רק כאשר זוהי שינוי בפרטים, וכפתור ביטול סוגר את החלון.

הודעות ודיאלוגים:

אם המשתמש מנסה לצאת ללא שמירת שינויים מופיעה אזהרה למניעת איבוד מידע. במקרה של שגיאה (שדות ריקים או נתונים קיימים במערכת) מוצגת הודעת שגיאה. לאחר עדכון מוצלח מופיעה הודעת הצלחה המאשרת שהפרטים נשמרו.

12. תיאור התוכנה

12.1 סביבת הפיתוח של התוכנה

12.1.1 מערכת הפעלה

פיתוח והרצת המערכת בוצעו בסביבת עבודה המבוססת על מערכת ההפעלה Windows 11.

12.1.2 סביבות הפיתוח (IDEs)

VsCode 1.108

Visual Studio Code (VS Code) היא סביבת פיתוח קלה וגמישה, התומכת בפיתוח Java באמצעות הרחבות ייעודיות. הסביבה מאפשרת כתיבת קוד, ניפוי שגיאות, ניהול פרויקטים ועבודה עם מערכות ניהול גרסאות בצורה נוחה ויעילה.

12.1.3 מסגרות לפיתוח התוכנה (Frameworks)

Spring Boot 3.5.8 היא מסגרת עבודה לפיתוח יישומי צד שרת מבוססי Java, המספקת תצורה אוטומטית ותשתית מוכנה לפיתוח שירותי Web ו-API. המסגרת מאפשרת מימוש רכיבי Backend כגון שרת HTTP מובנה, לוגיקה יישומית בצד השרת, שכבת גישה לנתונים ובקרים לטיפול בבקשות HTTP. השימוש ב-Spring Boot מפשט את תהליך הפיתוח ומאפשר בנייה מסודרת של שכבת הלוגיקה והתקשורת במערכת.

Vaadin 24.9.9

Vaadin היא מסגרת עבודה לפיתוח ממשקי Web מבוססי Java בצד השרת, המאפשרת יצירת ממשק משתמש אינטראקטיבי ללא כתיבה ישירה של קוד Frontend. הסנכרון בין השרת לדפדפן מתבצע באופן אוטומטי באמצעות WebSocket ו-Push.

12.1.4 שפות תכנות

Java 21

Java 21 היא שפת תכנות עילית מונחית עצמים (Object-Oriented Programming), המתאימה לפיתוח מערכות תוכנה מורכבות. השפה מאפשרת מימוש של רכיבי צד שרת ולוגיקה יישומית, וכן שילוב עם מסגרות עבודה המאפשרות פיתוח מערכת מלאה מקצה לקצה (Full-Stack).

12.1.5 ספריות/מערכות/מנגנוני עזר נוספים

במהלך הפיתוח נעשה שימוש בהרחבות וכלי עזר, כגון PlantUML ליצירת תרשימי ארכיטקטורה ותרשימי רצף, וכן הרחבות לפיתוח Java, Spring ו-MongoDB.

בפרויקט נעשה שימוש במספר ספריות מרכזיות, בהן Spring Web לפיתוח שירותי Web, Vaadin לפיתוח ממשק משתמש מבוסס Web, Spring Data MongoDB לניהול הנתונים ו־Spring Boot DevTools לצורכי פיתוח וטעינה מחדש.

12.2 תיאור מבנה הפרויקט והמחלקות

12.2.1 עץ המודולים של הפרויקט

```

C:.\
├── project_ftp_v1
│   ├── AppConfig.java
│   ├── ProjectFtpV1Application.java
│   ├── controllers
│   │   └── FileDownloadController.java
│   ├── datamodels
│   │   ├── FileDocument.java
│   │   ├── UploadedFileDTO.java
│   │   └── User.java
│   ├── repositories
│   │   ├── FileRepository.java
│   │   └── UserRepository.java
│   ├── services
│   │   ├── FtpClientHandler.java
│   │   ├── FtpUiService.java
│   │   ├── LZ78Service.java
│   │   ├── MyFtpServer.java
│   │   ├── UserService.java
│   │   └── WeatherService.java
│   ├── ui
│   │   ├── AppNavbarLayout.java
│   │   ├── HomeView.java
│   │   ├── LoginView.java
│   │   ├── ProfileDialog.java
│   │   ├── RegisterView.java
│   │   └── MainView.java
│   └── utils
│       ├── RouteHelper.java
│       └── SessionHelper.java

```

קבצי השורש (Root Files):

קבצי הליבה של המערכת, כגון ProjectFtpV1Application ו־AppConfig, האחראיים על אתחול המערכת, הפעלת שרת ה-Spring Boot וטעינת ההגדרות הראשוניות של הפרויקט.

controllers:

שכבת הבקרים (Controllers) של המערכת. שכבה זו אחראית על טיפול בבקשות HTTP חיצוניות וחשיפת נקודות קצה (Endpoints) לצורך תקשורת עם הדפדפן או עם שירותים חיצוניים, כגון הורדת קבצים דרך הדפדפן.

:datamodels

שכבת מודלי הנתונים של המערכת. מכילה את המחלקות המייצגות את מבנה המידע הנשמר במסד הנתונים, כגון משתמשים, קבצים ונתונים נוספים הקשורים למערכת.

:repositories

שכבת הגישה למסד הנתונים. שכבה זו כוללת ממשקי Spring Data האחראיים על ביצוע פעולות מול MongoDB, כגון שמירה, שליפה, עדכון ומחיקה של נתונים.

:services

שכבת הלוגיקה העסקית של המערכת ומהווה את הליבה המרכזית של הפרויקט. בשכבה זו מתבצעות הפעולות המרכזיות כגון ניהול משתמשים, תהליכי העלאה והורדה של קבצים, הפעלת שרת ה-FTP ומימוש אלגוריתם הדחיסה LZ78.

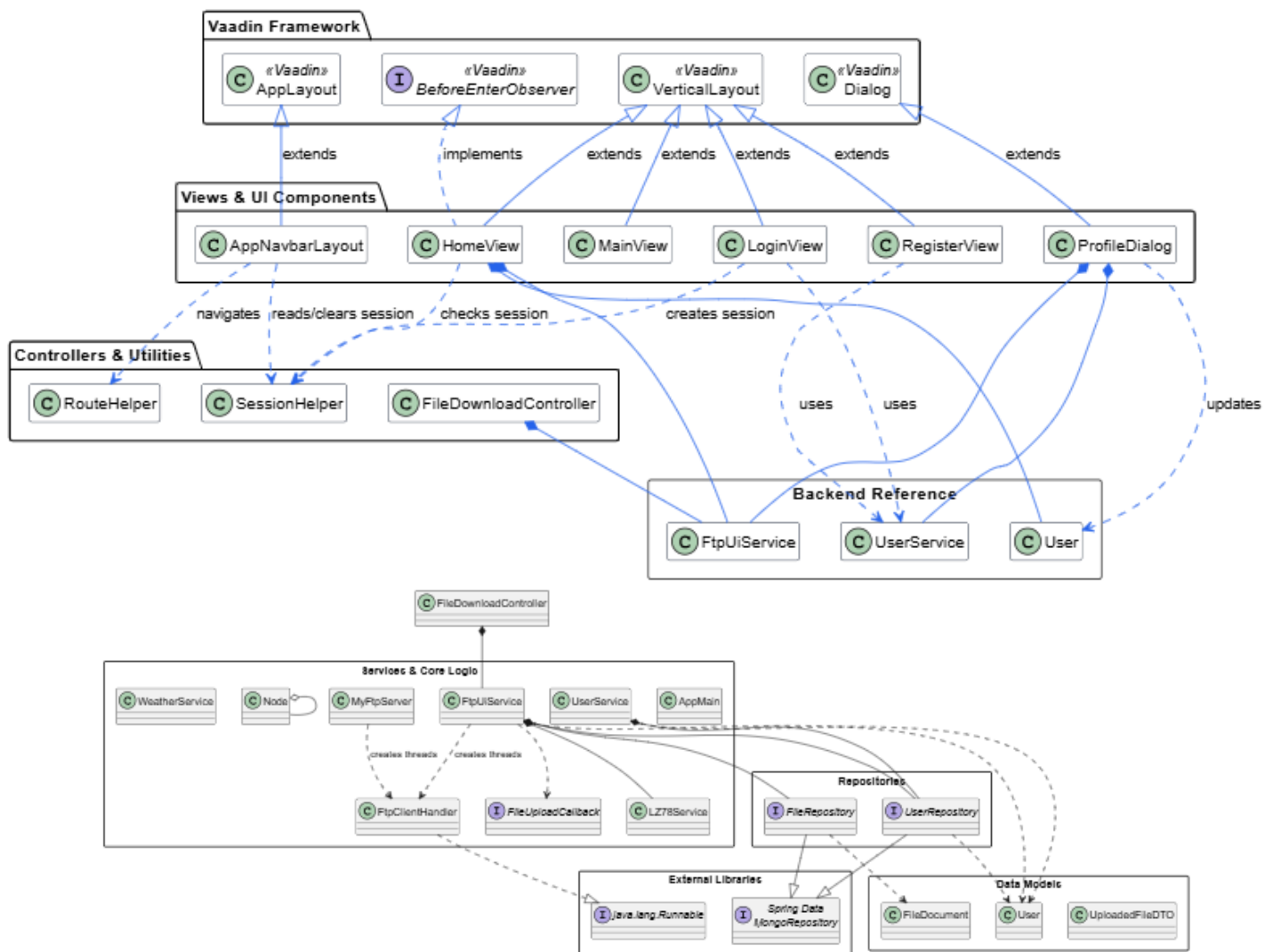
:ui

שכבת ממשק המשתמש (UI) המבוססת על Vaadin. שכבה זו כוללת את כל המסכים, הטפסים, התפריטים והדיאלוגים המוצגים למשתמש ומאפשרים אינטראקציה עם המערכת.

:utils

שכבת מחלקות העזר של המערכת. מכילה פונקציות וכלים כלליים המסייעים לפעילות המערכת, כגון ניהול ניווט בין מסכים, ניהול Session של המשתמש המחובר ופונקציות עזר נוספות.

12.2.2 תרשים המחלקות UML Class Diagram



מבנה המערכת כולל את השכבות הבאות:

- שכבת הנתונים (Data Models)
- שכבת מסד הנתונים (Repositories)
- שכבת השירותים והלוגיקה (Services)
- שכבת הבקרים (Controllers)
- שכבת ממשק המשתמש (UI / Views)
- שכבת עזר (Utilities)

1. שכבת הנתונים (Data Models)

שכבה זו מייצגת את המידע המרכזי של המערכת ואת האובייקטים הנשמרים במסד הנתונים.

User - מחלקה המייצגת משתמש במערכת.

המחלקה שומרת נתונים כגון:

- שם משתמש
- כתובת אימייל
- סיסמה
- רשימת קבצים השייכים למשתמש

FileDocument - מחלקה המייצגת קובץ שהועלה לשרת.

היא שומרת מידע על הקובץ כגון:

- שם הקובץ
- המשתמש שהעלה אותו
- תאריך העלאה
- גודל מקורי
- גודל לאחר דחיסה

UploadedFileDTO - מחלקת DTO המשמשת להעברת מידע לממשק המשתמש בצורה נוחה להצגה.

המחלקה אינה נשמרת במסד הנתונים, אלא משמשת להצגת נתונים בטבלאות ובמסכים.

2. שכבת מסד הנתונים (Repositories)

שכבה זו אחראית על התקשורת הישירה מול MongoDB.

UserRepository - מחלקה המטפלת בכל פעולות המשתמשים במסד הנתונים:

- שמירת משתמשים
- חיפוש משתמשים
- עדכון נתונים
- בדיקת התחברות

FileRepository - מחלקה האחראית על ניהול נתוני הקבצים:

- שמירת מידע על קבצים
- שליפת קבצים
- מחיקת קבצים

- חיפוש קבצים לפי משתמש

3. שכבת השירותים והלוגיקה (Services)

שכבה זו מהווה את הליבה המרכזית של המערכת ובה מתבצעת רוב הלוגיקה העסקית.

UserService - מחלקה האחראית על ניהול המשתמשים במערכת.

תפקידי המחלקה:

- הרשמה
- התחברות
- עדכון פרופיל
- בדיקות תקינות והרשאות

FtpUiService - מחלקת השירות המרכזית של המערכת.

היא מקשרת בין ממשק המשתמש לבין שרת ה-FTP.

המחלקה אחראית על:

- העלאת קבצים
- הורדת קבצים
- מחיקת קבצים
- שמירת מידע במסד הנתונים
- הפעלת מנגנון הדחיסה והפריסה

LZ78Service - מחלקה המממשת את אלגוריתם הדחיסה LZ78.

המחלקה אחראית על:

- דחיסת קבצים
- שחזור קבצים
- עבודה עם זרמי נתונים (Streams)

MyFtpServer - מחלקה המייצגת את שרת ה-FTP של המערכת.

השרת:

- מאזין לחיבורים נכנסים
- מנהל את תהליך התקשורת
- מטפל בבקשות העברת קבצים

FtpClientHandler - מחלקה המטפלת בכל חיבור FTP פעיל.

לכל משתמש נפתח Thread נפרד המטפל ב:

- העלאת קבצים
- הורדת קבצים
- העברת נתונים לשרת

גישה זו מאפשרת תמיכה במספר משתמשים במקביל.

4. שכבת הבקרים (Controllers)

שכבה זו אחראית על תקשורת HTTP ו-REST API.

FileDownloadController - בקר האחראי על הורדת קבצים דרך הדפדפן.

הבקר:

- מקבל בקשת הורדה
- מזהה את המשתמש
- שולף את הקובץ מהשרת
- מעביר את הקובץ לדפדפן

הבקר משתמש ב-FtpUiService לצורך ביצוע פעולות ההורדה.

5. שכבת ממשק המשתמש (UI / Views)

שכבה זו כוללת את כל המסכים שמולם המשתמש עובד.

StartView

עמוד הפתיחה הראשי של המערכת.

LoginView

מסך התחברות למשתמשים קיימים.

RegisterView

מסך הרשמה ליצירת משתמש חדש.

HomeView

מסך העבודה המרכזי של המשתמש.

במסך זה ניתן:

- להעלות קבצים
- להוריד קבצים
- לצפות בקבצים השמורים

AppNavbarLayout

תפריט הניווט העליון של המערכת.

ProfileDialog

חלון לעריכת פרטי המשתמש:

- שינוי שם
- שינוי אימייל
- שינוי סיסמה

6. שכבת עזר (Utilities)

מחלקות עזר המשמשות את המערכת בפעולות כלליות.

-SessionHelper מחלקה האחראית על ניהול נתוני המשתמש המחובר במערכת.

-RouteHelper מחלקת עזר לניווט בין מסכי המערכת.

הקשרים בין רכיבי המערכת

רכיבי המערכת עובדים יחד בצורה מסודרת לפי חלוקת השכבות.

דוגמאות לקשרים מרכזיים:

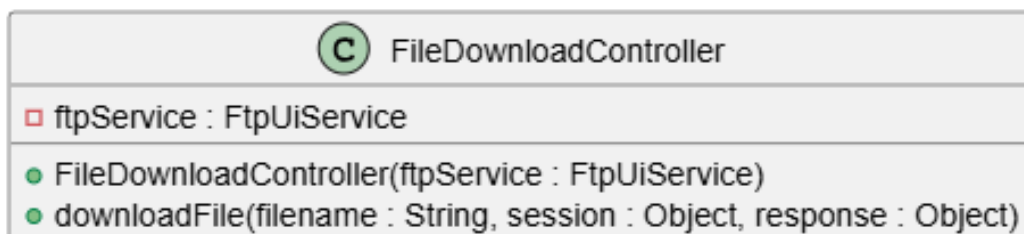
- UserService משתמש ב- UserRepository
- FtpUiService משתמש ב- FileRepository וב- LZ78Service
- HomeView מתקשר עם FtpUiService
- FileDownloadController משתמש ב- FtpUiService לצורך הורדת קבצים
- MyFtpServer יוצר מופעים של FtpClientHandler עבור כל חיבור חדש

מבנה זה מאפשר הפרדה ברורה בין:

- ממשק המשתמש
- הלוגיקה העסקית
- ניהול הנתונים
- תהליכי התקשורת והדחיסה

וכך מתקבלת מערכת יציבה, מודולרית ונוחה להרחבה ותחזוקה.

12.2.3 תיאור מחלקות מפורט



תפקיד המחלקה FileDownloadController:

המחלקה משמשת כבקר תקשורת (REST Controller) במערכת Spring Boot. תפקידה הוא לאפשר הורדת קבצים דרך בקשות HTTP מהדפדפן באמצעות כתובת ייעודית מסוג GET (/download/{filename}).

הבקר מקשר בין ממשק המשתמש לבין שכבת הלוגיקה של המערכת, ומאפשר למשתמש להוריד קבצים המאוחסנים בשרת בצורה תקינה ומאובטחת. תכונות ופעולות משמעותיות

ftpService (תכונה):

אובייקט המקשר את הבקר לשכבת הלוגיקה העסקית של המערכת. דרכו מתבצעת שליפת הקבצים מהשרת והעברת הנתונים אל המשתמש.

downloadFile (פעולה):

הפונקציה המרכזית האחראית על הורדת קבצים.

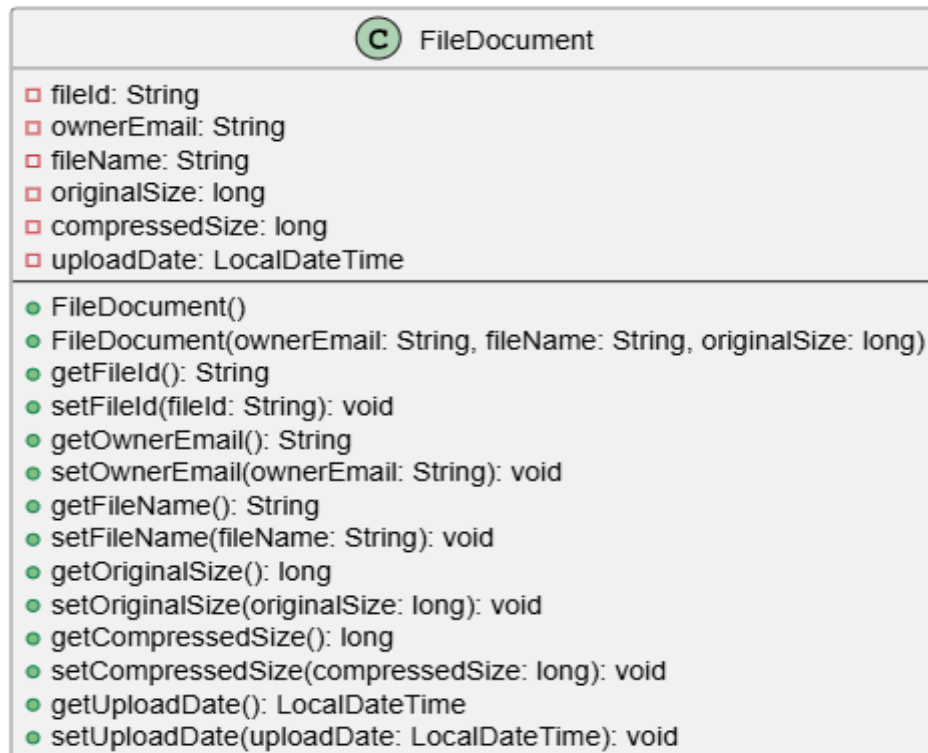
במהלך הפעולה:

המערכת בודקת האם המשתמש מחובר ומורשה לבצע הורדה.

במקרה שהמשתמש אינו מחובר, מוחזרת שגיאת הרשאה (401 Unauthorized).

אם המשתמש מאומת, הקובץ נשלף מהשרת ומוזרם ישירות לדפדפן דרך חיבור HTTP.

במהלך ההורדה הקובץ נשלח בצורה רציפה (Streaming), ללא טעינה מלאה לזיכרון השרת, דבר המשפר את יעילות העבודה עם קבצים גדולים.



תפקיד המחלקה FileDocument:

המחלקה FileDocument מייצגת קובץ במערכת ומשמשת כיישות מידע (Entity) האחראית על שמירת נתוני המטא־דאטה של קבצים במסד הנתונים MongoDB. המחלקה אינה שומרת את תוכן הקובץ עצמו, אלא מידע תיאורי עליו, כגון:

- שם הקובץ
- בעל הקובץ
- גודל לפני ואחרי דחיסה
- תאריך העלאה

באמצעות מידע זה המערכת מסוגלת לנהל קבצים, לבצע בקרה על אחסון, להציג נתונים למשתמשים, ולעקוב אחר יעילות תהליך הדחיסה. בנוסף, המחלקה מהווה חלק מרכזי בקישור בין מערכת ניהול המשתמשים לבין מנגנון ניהול הקבצים.

תכונות ופעולות משמעותיות

fileId (תכונה):

מזהה ייחודי של הקובץ במערכת.

שדה זה משמש כמפתח ראשי (Primary Key) במסד הנתונים ומאפשר זיהוי חד־חד ערכי של כל קובץ הנשמר במערכת.

ownerEmail (תכונה):

שומר את כתובת האימייל של המשתמש שהעלה את הקובץ.

באמצעות שדה זה המערכת מסוגלת:

- לקשר קבצים למשתמשים
- לבצע בדיקות הרשאה
- להציג לכל משתמש את הקבצים האישיים שלו בלבד

fileName (תכונה):

מייצג את שם הקובץ כפי שנבחר או הועלה על ידי המשתמש.

המערכת משתמשת בשם זה לצורך:

הצגה בממשק המשתמש

הורדת קבצים

זיהוי קבצים בתהליכי ניהול ואחסון

originalSize (תכונה):

שומר את גודל הקובץ המקורי לפני ביצוע דחיסה.

נתון זה מאפשר לחשב את יחס הדחיסה ואת כמות החיסכון בנפח האחסון.

compressedSize (תכונה):

שומר את גודל הקובץ לאחר תהליך הדחיסה באמצעות אלגוריתם LZ78.

באמצעות השוואה בין originalSize לבין compressedSize, המערכת מסוגלת להציג למשתמש את יעילות הדחיסה.

uploadDate (תכונה):

שומר את התאריך והשעה שבהם הקובץ הועלה למערכת.

המידע משמש לצורך:

- ניהול היסטוריית העלאות
- מיון קבצים
- מעקב אחר פעילות משתמשים

FileDocument (פעולה):

בנאי המחלקה האחראי על יצירת אובייקט חדש מסוג FileDocument.

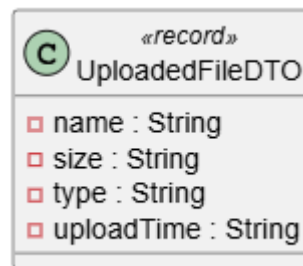
בעת יצירת האובייקט מתבצע אתחול ראשוני של נתוני הקובץ לפני שמירתו במסד הנתונים.

Getter / Setter Methods (פעולות):

המחלקה כוללת פעולות Getter ו-Setter עבור כלל השדות.

פעולות אלו מאפשרות:

- גישה מבוקרת לנתוני האובייקט
- עדכון ערכים בצורה מאורגנת
- שמירה על עקרונות הכימוס (Encapsulation) בתכנות מונחה עצמים



תפקיד המחלקה UploadedFileDTO:

המחלקה UploadedFileDTO משמשת כאובייקט להעברת נתונים (Data Transfer Object – DTO) בין שכבת הלוגיקה העסקית לבין שכבת ממשק המשתמש במערכת. מטרת המחלקה היא להעביר מידע מעובד ומצומצם על קבצים שהועלו למערכת, מבלי לחשוף את אובייקט הנתונים המלא (FileDocument) או את הלוגיקה הפנימית של המערכת. המחלקה משמשת בעיקר להצגת מידע בטבלאות ובמסכי הניהול של המשתמש, בצורה ידידותית וברורה. בנוסף, המחלקה מוגדרת כ-record בשפת Java, דבר המאפשר יצירת אובייקט בלתי־ניתן לשינוי (Immutable Object) עם קוד תמציתי ויעיל.

תכונות ופעולות משמעותיות

name (תכונה):

מייצגת את שם הקובץ שהועלה למערכת.

המידע משמש להצגת הקובץ למשתמש בממשק המשתמש ובטבלאות הניהול.

size (תכונה):

מייצגת את גודל הקובץ בפורמט קריא למשתמש.

בניגוד לנתון הגולמי הנשמר במסד הנתונים (Bytes), ערך זה מוצג בדרך ידידותית יותר, לדוגמה:

- KB
- MB
- GB

type (תכונה):

מייצגת את סוג הקובץ או הסיומת שלו.

שדה זה מאפשר למשתמש לזהות במהירות את סוג הקובץ שהועלה למערכת.

uploadTime (תכונה):

שומר את זמן או תאריך העלאת הקובץ בפורמט מותאם לתצוגה.

המידע משמש להצגת היסטוריית העלאות ומעקב אחר פעילות המשתמש.

מאפיינים מיוחדים של המחלקה

Record Class

המחלקה מוגדרת כ-record, מנגנון שהתווסף בשפת Java לצורך יצירת מחלקות נתונים בצורה תמציתית.

שימוש ב-record מספק באופן אוטומטי:

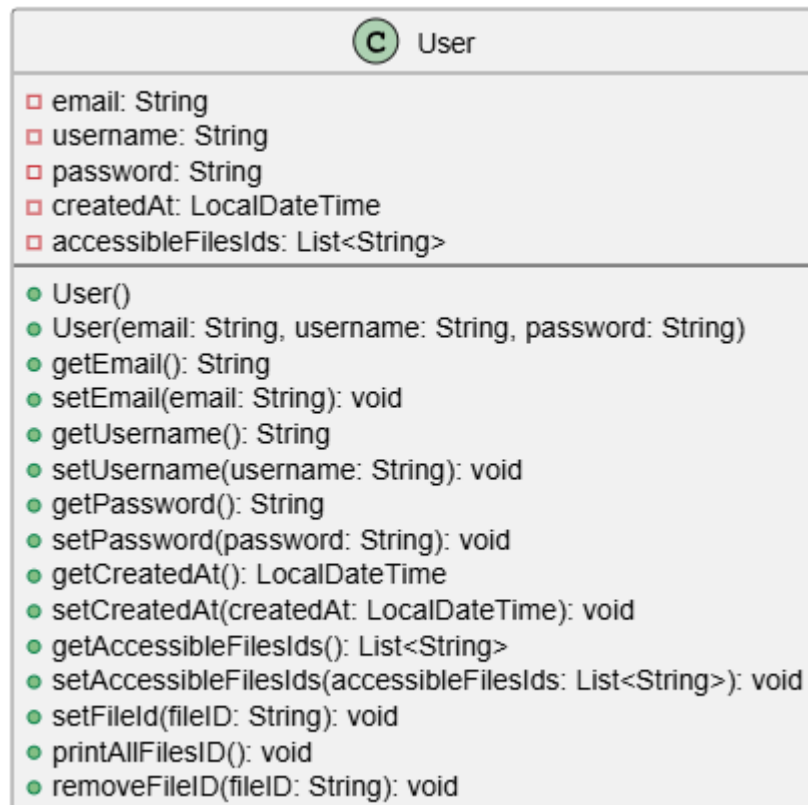
- בנאי מלא
- פעולות Getter
- פעולת toString
- פעולות equals ו hashCode

גישה זו מפחיתה כתיבת קוד מיותר ומשפרת את קריאות ותחזוקת המערכת.

מטרת השימוש ב-DTO במערכת

השימוש ב-DTO מאפשר:

- הפרדה בין שכבת הנתונים לבין שכבת התצוגה
- מניעת חשיפת מידע פנימי שאינו נדרש ל-UI
- שיפור ביצועים על ידי העברת מידע רלוונטי בלבד
- יצירת מבנה נתונים מותאם במיוחד לתצוגה בממשק המשתמש



תפקיד המחלקה User:

המחלקה User מייצגת משתמש רשום במערכת ומשמשת כיישום מרכזי לניהול חשבונות משתמשים והרשאות גישה לקבצים.

המחלקה שומרת את פרטי ההזדהות של המשתמש, מידע אישי בסיסי, ותיעוד של הקבצים המשויכים אליו במערכת.

בנוסף, המחלקה מאפשרת למערכת לזהות משתמשים, לבצע אימות התחברות, לנהל הרשאות גישה לקבצים, ולעקוב אחר פעילות המשתמש במערכת האחסון.

המחלקה נשמרת במסד הנתונים MongoDB ומשמשת כחלק מרכזי בתהליך ניהול המשתמשים והגישה למשאבי המערכת.

תכונות ופעולות משמעותיות

email (תכונה):

מייצגת את כתובת האימייל של המשתמש.

שדה זה משמש כמזהה ייחודי (Unique Identifier) עבור כל משתמש במערכת ומשמש גם בתהליך ההתחברות והאימות.

username (תכונה):

שומר את שם המשתמש המוצג במערכת.

המידע משמש להצגת פרטי המשתמש בממשק המשתמש ובפעולות שונות הקשורות לניהול החשבון.

password (תכונה):

שומר את סיסמת המשתמש לצורך אימות זהות בעת התחברות למערכת.
שדה זה משמש כחלק ממנגנון האבטחה של המערכת.

createdAt (תכונה):

שומר את תאריך ושעת יצירת חשבון המשתמש.
המערכת משתמשת במידע זה לצורך תיעוד, מעקב אחר משתמשים וניהול היסטוריית חשבונות.

accessibleFileIds (תכונה):

רשימה דינמית המכילה את מזהי הקבצים המשויכים למשתמש.
באמצעות רשימה זו המערכת מסוגלת:

- לעקוב אחר קבצים שהמשתמש העלה
- לנהל הרשאות גישה
- לשלוף קבצים השייכים למשתמש

User (פעולה):

בנאי המחלקה האחראי על יצירת אובייקט חדש מסוג User.
בעת יצירת המשתמש מתבצע אתחול ראשוני של פרטי החשבון לפני שמירתו במסד הנתונים.

setFileId (פעולה):

מוסיפה מזהה קובץ חדש לרשימת הקבצים של המשתמש.
פעולה זו מתבצעת בדרך כלל לאחר העלאת קובץ חדש למערכת.

removeFileId (פעולה):

מסירה מזהה קובץ מרשימת הקבצים של המשתמש.
המערכת משתמשת בפעולה זו בעת מחיקת קובץ או ביטול הרשאת גישה.

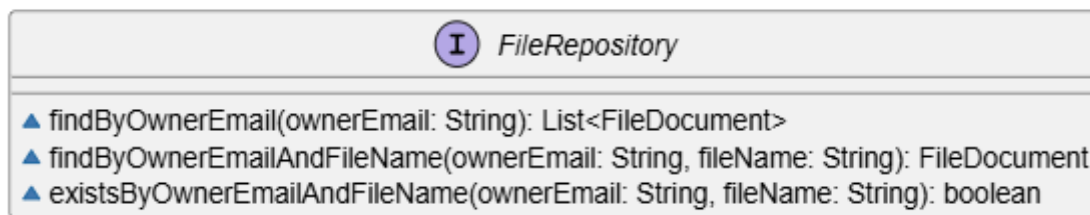
printAllFilesID (פעולה):

פעולה המשמשת להצגת כל מזהי הקבצים המשויכים למשתמש.
פעולה זו יכולה לשמש לצורכי בדיקות, ניהול מערכת או דיוג.

Getter / Setter Methods (פעולות):

המחלקה כוללת פעולות Getter ו-Setter עבור כלל השדות.
פעולות אלו מאפשרות:

- גישה מבוקרת לנתוני המשתמש
- עדכון מידע בצורה מאורגנת
- שמירה על עקרונות הכימוס (Encapsulation) בתכנות מונחה עצמים



תפקיד הממשק FileRepository:

הממשק FileRepository משמש כשכבת הגישה למסד הנתונים (Repository Layer) עבור אוסף הקבצים במערכת MongoDB. הממשק אחראי על ביצוע פעולות שליפה, חיפוש ובדיקת קיום של קבצים הנשמרים במסד הנתונים, ללא צורך בכתיבת שאילתות ידניות. הממשק מבוסס על מנגנון MongoRepository של Spring Data, המאפשר יצירה אוטומטית של פעולות גישה לנתונים בהתאם לשמות הפונקציות המוגדרות בממשק. באמצעות ממשק זה, שכבת השירותים במערכת מסוגלת לבצע פעולות ניהול קבצים בצורה פשוטה, יעילה ומאורגנת.

תכונות ופעולות משמעותיות

findByOwnerEmail (פעולה):

מחזירה רשימת קבצים השייכים למשתמש מסוים לפי כתובת האימייל שלו. המערכת משתמשת בפעולה זו לצורך:

- הצגת רשימת הקבצים האישיים של המשתמש
- טעינת נתוני הקבצים למסך הבית
- ניהול קבצים לפי משתמש

הפעולה מחזירה אוסף מסוג <List<FileDocument> המכיל את כל הקבצים המשויכים למשתמש.

findByOwnerEmailAndFileName (פעולה):

מחפשת קובץ מסוים לפי כתובת האימייל של המשתמש ושם הקובץ. המערכת משתמשת בפעולה זו כאשר נדרש לאתר קובץ מדויק לצורך:

- הורדת קובץ
- מחיקת קובץ
- בדיקות הרשאה
- שליפת נתוני מטא־דאטה

existsByOwnerEmailAndFileName (פעולה):

בודקת האם כבר קיים קובץ עם אותו שם עבור משתמש מסוים.

פעולה זו חשובה במיוחד לצורך:

- מניעת העלאה כפולה של קבצים
- מניעת דריסת מידע קיים
- שמירה על תקינות נתוני המערכת

הפעולה מחזירה ערך בוליאני (boolean) המציין האם הקובץ כבר קיים במסד הנתונים.

מאפיינים מיוחדים של הממשק

שימוש ב-Spring Data MongoDB

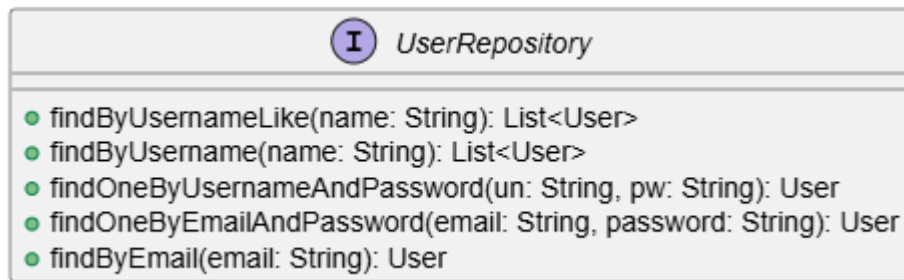
הממשק עושה שימוש ביכולות האוטומטיות של Spring Data MongoDB.

באמצעות מנגנון זה:

- Spring מייצר אוטומטית את מימוש הפונקציות
- אין צורך בכתיבת שאילתות MongoDB ידניות
- קוד המערכת נשאר קצר, נקי וקריא יותר
- חשיבות שכבת Repository במערכת
- שכבת ה-Repository מספקת הפרדה בין:
- שכבת הלוגיקה העסקית (Services)
- לבין שכבת מסד הנתונים

גישה זו משפרת:

- תחזוקת קוד
- קריאות המערכת
- הרחבה עתידית



תפקיד הממשק UserRepository:

הממשק UserRepository משמש כשכבת הגישה למסד הנתונים (Repository Layer) עבור אוסף המשתמשים במערכת MongoDB. הממשק אחראי על ביצוע פעולות הקשורות לניהול משתמשים, כגון:

- חיפוש משתמשים
- אימות התחברות
- שליפת נתוני משתמשים
- ובדיקת קיום חשבונות במערכת

הממשק מבוסס על מנגנון Spring Data MongoDB, אשר מאפשר יצירה אוטומטית של שאלות ומימוש פעולות גישה לנתונים בהתאם לשמות הפונקציות המוגדרות בממשק. באמצעות ממשק זה, שכבת השירותים במערכת יכולה לבצע פעולות על נתוני המשתמשים בצורה יעילה, מסודרת וללא כתיבת שאלות ידניות.

תכונות ופעולות משמעותיות

findByUsername (פעולה):

מחפשת משתמשים לפי שם משתמש מדויק.

המערכת משתמשת בפעולה זו לצורך:

איתור משתמשים

בדיקות זמינות שם משתמש

ניהול נתוני משתמשים

findOneByUsernameAndPassword (פעולה):

מבצעת אימות משתמש באמצעות שם משתמש וסיסמה.

המערכת משתמשת בפעולה זו במהלך תהליך ההתחברות כדי לוודא שפרטי ההזדהות תקינים.

אם נמצאה התאמה, מוחזר אובייקט מסוג User, אחרת מוחזר ערך ריק (null).

findOneByEmailAndPassword (פעולה):

מבצעת אימות משתמש באמצעות כתובת אימייל וסיסמה.

פעולה זו מספקת אפשרות התחברות נוספת המבוססת על כתובת האימייל של המשתמש.

המערכת משתמשת בפעולה זו כחלק ממנגנון האימות והכניסה לחשבון.

findByEmail (פעולה):

מחפשת משתמש לפי כתובת האימייל שלו.

פעולה זו משמשת לצורך:

- בדיקת קיום משתמש במערכת
- מניעת יצירת חשבונות כפולים
- שליפת פרטי משתמש
- ניהול ועדכון חשבונות
- מאפיינים מיוחדים של הממשק
- שימוש ב-Spring Data MongoDB

הממשק עושה שימוש במנגנון האוטומטי של Spring Data MongoDB.

באמצעות מנגנון זה:

- Spring מייצר באופן אוטומטי את מימוש הפונקציות
- אין צורך בכתיבת שאילתות MongoDB ידניות
- הקוד נשאר קצר, נקי וקל לתחזוקה
- חשיבות שכבת Repository במערכת
- שכבת ה-Repository מספקת הפרדה בין:
- שכבת הלוגיקה העסקית (Services)
- לבין שכבת מסד הנתונים

הפרדה זו מאפשרת:

- תחזוקת קוד נוחה יותר
- הרחבה עתידית של המערכת
- שיפור קריאות הקוד



תפקיד המחלקה FtpClientHandler:

המחלקה FtpClientHandler אחראית על ניהול התקשורת בין שרת ה-FTP לבין לקוח FTP בודד המחובר למערכת.

המחלקה מטפלת בכל פקודות ה-FTP הנשלחות מהלקוח, כגון:

- העלאת קבצים
- הורדת קבצים
- הצגת רשימת קבצים
- ניהול חיבורי DATA ו-CONTROL

לכל משתמש המתחבר לשרת נוצר מופע עצמאי של המחלקה, אשר פועל בתהליכון (Thread) נפרד ומאפשר טיפול במספר משתמשים במקביל.

בנוסף, המחלקה אחראית על אבטחת הגישה לקבצים ומונעת גישה לא מורשית מחוץ לתיקיית המשתמש.

תכונות ופעולות משמעותיות

controlSocket (תכונה):

מייצג את ערוץ התקשורת הראשי (Control Connection) בין שרת ה-FTP לבין הלקוח. דרך חיבור זה מועברות פקודות ה-FTP והודעות הבקרה.

rootDir (תכונה):

מייצג את תיקיית הבסיס של מערכת האחסון בשרת. כל פעולות הקריאה והכתיבה של קבצים מתבצעות ביחס לתיקייה זו.

passiveServer (תכונה):

מייצג שרת זמני המשמש ליצירת חיבור DATA במצב Passive Mode. חיבור זה משמש להעברת נתוני הקבצים עצמם בזמן העלאה, הורדה או הצגת רשימות קבצים.

writer (תכונה):

אובייקט המשמש לשליחת תגובות טקסטואליות ללקוח ה-FTP. באמצעותו השרת מחזיר:

קודי סטטוס

הודעות הצלחה

הודעות שגיאה

בהתאם לפרוטוקול FTP.

currentUserDir (תכונה):

שומר את תיקיית העבודה הנוכחית של המשתמש המחובר. שדה זה מספק שכבת אבטחה חשובה בכך שהוא מגביל את המשתמש לפעול רק בתוך הספרייה האישית שלו ומונע גישה לקבצים מחוץ לתחום המותר.

FtpClientHandler (פעולה):

בנאי המחלקה האחראי על יצירת מופע חדש עבור לקוח FTP מחובר. בעת יצירת האובייקט מתבצע אתחול של ערוצי התקשורת ונתיבי העבודה של המשתמש.

run (פעולה):

הפעולה המרכזית של המחלקה. הפעולה מאזינה לפקודות הנשלחות מהלקוח ומנתבת כל פקודה לטיפול המתאים בהתאם לפרוטוקול FTP.

הפעולה פועלת ברציפות לאורך זמן החיבור של המשתמש לשרת.

handlePasv (פעולה):

אחראית על הפעלת מצב Passive Mode בפרוטוקול FTP. הפעולה פותחת חיבור DATA ייעודי להעברת קבצים ומידע בין השרת לבין הלקוח.

handleStor (פעולה):

מטפלת בתהליך העלאת קובץ מהלקוח אל השרת. במהלך הפעולה:

- נפתח חיבור DATA
- מתקבלת זרימת נתונים מהלקוח
- והקובץ נשמר בדיסק השרת

הפעולה תומכת בהעברת קבצים גדולים באמצעות Streaming ללא טעינה מלאה לזיכרון.

handleRetr (פעולה):

מטפלת בתהליך הורדת קובץ מהשרת אל הלקוח.

הקובץ מוזרם ישירות מהדיסק אל חיבור הרשת בצורה רציפה ויעילה.

handleList (פעולה):

מחזירה ללקוח רשימת קבצים הנמצאים בתיקיית המשתמש.

פעולה זו משמשת להצגת תוכן הספרייה בממשקי FTP שונים.

getSafePath (פעולה):

מבצעת בדיקות אבטחה על נתיבי קבצים.

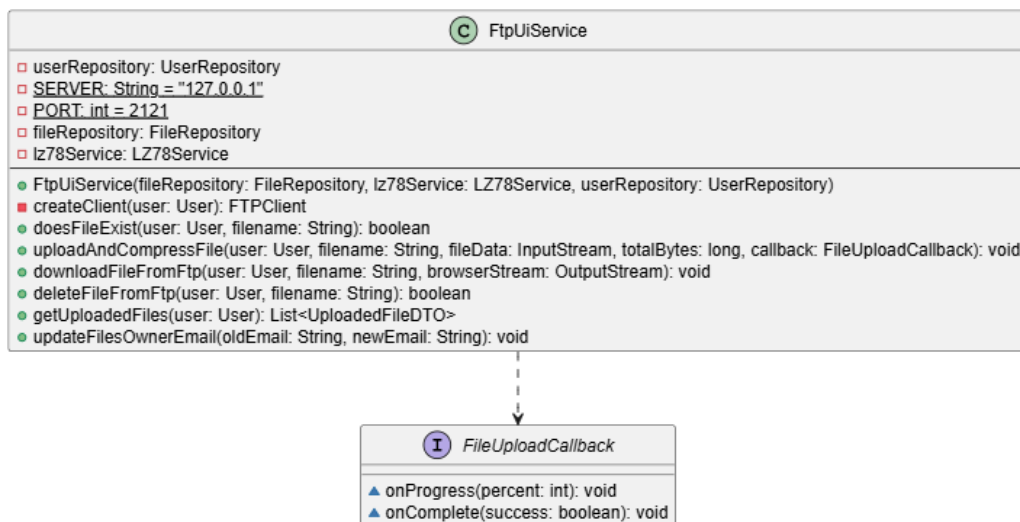
מטרת הפעולה היא למנוע ניסיונות גישה לקבצים מחוץ לתיקיית המשתמש באמצעות נתיבים זדוניים.

פעולה זו מהווה מנגנון אבטחה מרכזי במערכת.

closePassive (פעולה):

אחראית על סגירת חיבור ה-Passive Mode לאחר סיום העברת הנתונים.

הפעולה משחררת משאבי מערכת ומונעת דליפות חיבורים פתוחים.

**תפקיד המחלקה FtpUiService:**

המחלקה FtpUiService מהווה את שכבת הלוגיקה המרכזית האחראית על ניהול הקבצים במערכת.

המחלקה משמשת כרכיב גישור (Bridge) בין:

- ממשק המשתמש (UI)

- שרת ה-FTP

- מנגנון הדחיסה LZ78

- ומסד הנתונים MongoDB

באמצעות מחלקה זו המערכת מסוגלת לבצע:

- העלאת קבצים

- דחיסת קבצים

- הורדת קבצים

- מחיקת קבצים
- ניהול מטא-דאטה
- ושליפת נתוני קבצים להצגה בממשק המשתמש

המחלקה מרכזת את כל תהליכי העבודה הקשורים לניהול קבצים ומאפשרת עבודה מסודרת ויעילה מול כלל רכיבי המערכת.

תכונות ופעולות משמעותיות

userRepository (תכונה):

אובייקט גישה למסד הנתונים של המשתמשים.

באמצעותו המחלקה מבצעת:

- שליפת נתוני משתמשים
- עדכון נתוני בעלות על קבצים
- ניהול קשר בין משתמשים לקבצים

SERVER / PORT (תכונות):

מייצגות את כתובת שרת ה-FTP והפורט שעליו השרת מאזין.

נתונים אלו משמשים ליצירת חיבורי FTP מול שרת האחסון המקומי.

fileRepository (תכונה):

אובייקט גישה למסד הנתונים של הקבצים.

באמצעותו מתבצעות פעולות כגון:

- שמירת מטא-דאטה
- שליפת קבצים
- בדיקות קיום
- ועדכון נתוני קבצים

lz78Service (תכונה):

אובייקט המחובר למנוע הדחיסה LZ78 של המערכת.

המחלקה משתמשת בו לצורך:

- דחיסת קבצים בזמן העלאה
- פריסת קבצים בזמן הורדה

createClient (פעולה):

יוצרת חיבור FTP חדש מול שרת האחסון עבור משתמש מסוים.

הפעולה אחראית על אתחול ערוץ התקשורת והזדהות המשתמש מול שרת ה-FTP.

doesFileExist (פעולה):

בודקת האם כבר קיים קובץ עם אותו שם עבור המשתמש.

המערכת משתמשת בפעולה זו כדי למנוע:

- העלאה כפולה

- דריסת קבצים קיימים
- וחוסר תקינות בנתוני האחסון

uploadAndCompressFile (פעולה):

הפעולה המרכזית האחראית על העלאת קבצים למערכת.

במהלך הפעולה:

- מתקבל קובץ מהמשתמש
- הקובץ נדחס באמצעות אלגוריתם LZ78
- נוצר חיבור FTP לשרת
- הקובץ מועבר לשרת האחסון
- ונשמר מידע המטא-דאטה במסד הנתונים

הפעולה משתמשת במנגנון Streaming, ולכן הקובץ מועבר בצורה רציפה ללא טעינה מלאה לזיכרון השרת.

גישה זו משפרת ביצועים ומאפשרת עבודה עם קבצים גדולים.

downloadFileFromFtp (פעולה):

אחראית על הורדת קבצים מהשרת.

במהלך הפעולה:

- הקובץ נשלף משרת ה-FTP
- מתבצעת פריסה (Decompression) באמצעות LZ78
- והמידע מוזרם ישירות לדפדפן המשתמש

גם כאן נעשה שימוש ב-Streaming לצורך חיסכון בזיכרון ושיפור ביצועים.

deleteFileFromFtp (פעולה):

אחראית על מחיקת קבצים מהשרת וממסד הנתונים.

הפעולה מסירה:

- את הקובץ הפיזי משרת ה-FTP
- ואת רשומת המטא-דאטה ממסד הנתונים

getUploadedFiles (פעולה):

מחזירה רשימת קבצים שהועלו על ידי משתמש מסוים.

הפעולה ממירה את נתוני הקבצים לאובייקטים מסוג UploadedFileDTO המותאמים להצגה בממשק המשתמש.

updateFilesOwnerEmail (פעולה):

מעדכנת את כתובת האימייל של בעל הקבצים במערכת.

פעולה זו נדרשת כאשר משתמש משנה את כתובת האימייל שלו, על מנת לשמור על תקינות הקשר בין המשתמש לבין הקבצים שבבעלותו.

הממשק FileUploadCallback**תפקיד הממשק:**

הממשק FileUploadCallback משמש כמנגנון Callback למעקב אחר תהליך העלאת קבצים. הממשק מאפשר לעדכן את ממשק המשתמש בזמן אמת במהלך העלאת הקובץ.

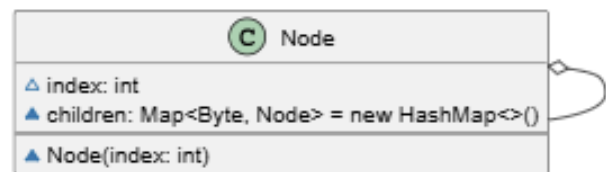
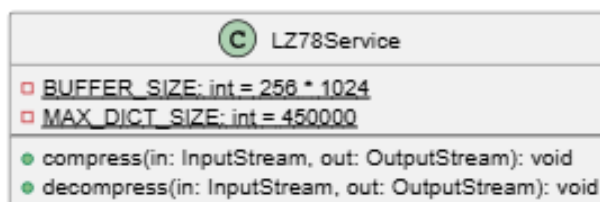
onProgress (פעולה):

מתעדכנת במהלך העלאת הקובץ ומחזירה את אחוז ההתקדמות של תהליך ההעלאה.

onComplete (פעולה):

נקראת כאשר תהליך ההעלאה מסתיים.

הפעולה מחזירה האם ההעלאה הצליחה או נכשלה.

**תפקיד המחלקה LZ78Service**

המחלקה LZ78Service אחראית על מימוש אלגוריתם הדחיסה והפריסה LZ78 במערכת. זהו רכיב קריטי בלוגיקת המערכת, שמטרתו לצמצם את נפח הקבצים בזמן העלאה ולשחזר אותם במלואם בזמן הורדה. המחלקה פועלת על זרמים (Streams), ולכן היא מסוגלת לעבד קבצים גדולים בצורה רציפה, ללא טעינה מלאה לזיכרון.

תכונות ופעולות משמעותיות**BUFFER_SIZE (תכונה סטטית):**

גודל החוצץ (Buffer) שבו משתמש האלגוריתם לקריאת נתונים מהקובץ.

ערך זה (KB256) מאפשר איזון בין ביצועים לבין שימוש בזיכרון.

MAX_DICT_SIZE (תכונה סטטית):

מגבלת הגודל של מילון הדחיסה.

כאשר המילון מגיע לגודל זה, מתבצע איפוס חלקי/מלא כדי למנוע צריכת זיכרון גבוהה מדי.

compress (פעולה):

פעולת הדחיסה המרכזית של המחלקה.

במהלך הפעולה:

- נקראים נתונים מה-InputStream
- נבנה מילון דינמי של רצפים חוזרים
- כל רצף מוחלף בקוד קצר יותר
- התוצאה נכתבת ל-OutputStream

מטרת הפעולה היא להקטין את גודל הקובץ תוך שמירה מלאה על המידע.

decompress (פעולה):

פעולת הפריסה (שחזור הקובץ המקורי).

במהלך הפעולה:

- נקראים קודי הדחיסה מהזרם
- נבנה מחדש מילון הדחיסה
- משוחזר המידע המקורי בצורה מדויקת
- נכתב ל-OutputStream
- Node (מחלקה פנימית של עץ הדחיסה)

תפקיד המחלקה:

המחלקה Node מייצגת צומת בעץ הדחיסה (Trie) של אלגוריתם LZ78. היא משמשת לבניית מבנה נתונים שמאפשר זיהוי מהיר של רצפים חוזרים בקובץ.

index (תכונה):

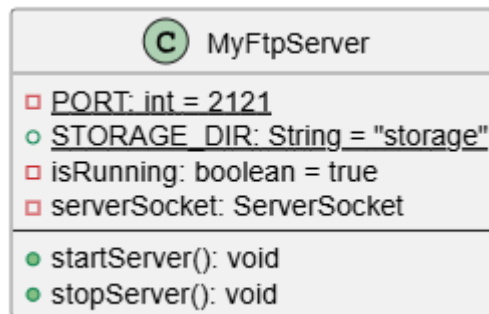
מייצג את המזהה הייחודי של הצומת במילון הדחיסה.

children (תכונה):

מפה (Map) של בתיים (Byte) לצמתים הבאים בעץ. מבנה זה מאפשר מעבר מהיר על רצפים של נתונים בזמן הדחיסה.

Node(index) (פעולה / בנאי):

- יוצר צומת חדש בעץ עם מזהה נתון.
- קשר בין הצמתים (Node o-- Node)
- הקשר מציין מבנה היררכי מסוג עץ (Trie):
- כל Node יכול להחזיק מספר צאצאים
- כל צאצא מייצג המשך של רצף נתונים
- כך נבנה מילון הדחיסה בצורה דינמית ויעילה.



תפקיד המחלקה MyFtpServer

המחלקה MyFtpServer משמשת כשרת ה-FTP המקומי של המערכת. תפקידה הוא להאזין לבקשות חיבור נכנסות מלקוחות, לנהל תקשורת קבצים, ולספק שכבת תשתית שמאפשרת העלאה, הורדה וניהול קבצים בצורה מבוקרת. זהו רכיב תשתיתי מרכזי שמפעיל את כל מנגנון ה-FTP של המערכת.

תכונות ופעולות משמעותיות

PORT (תכונה סטטית):

מגדיר את הפורט שבו שרת ה-FTP מאזין לבקשות חיבור. במקרה זה: 2121.

פורט זה משמש כנקודת כניסה לכל התקשורת בין לקוחות לשרת.

STORAGE_DIR (תכונה סטטית):

מגדיר את תיקיית האחסון המקומית של השרת. כל הקבצים שהועלו למערכת נשמרים פיזית בתוך תיקייה זו.

isRunning (תכונה):

מצב הפעולה של השרת.

true - השרת פעיל ומאזין לבקשות

false - השרת כבוי ואינו מקבל חיבורים

serverSocket (תכונה):

האובייקט האחראי על פתיחת ה-Socket הראשי של השרת. באמצעותו השרת:

- מאזין לחיבורים נכנסים
- מקבל לקוחות חדשים
- ומעביר אותם לטיפול

startServer (פעולה):

פעולה שמפעילה את השרת.

במהלך הפעולה:

- נפתח ServerSocket בפורט המוגדר
- מתחיל לולאת האזנה לחיבורים

- לכל חיבור נכנס מוקצה טיפול נפרד (Client Handler)

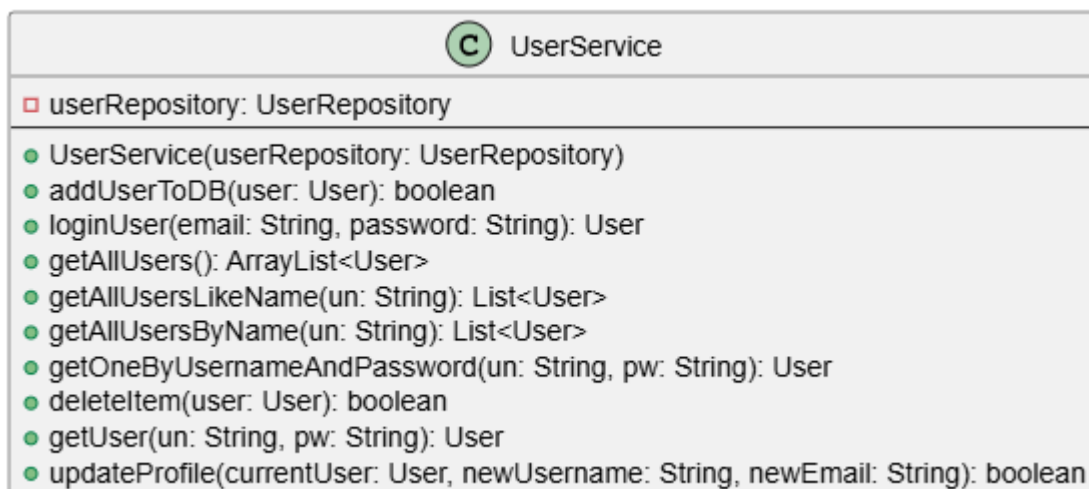
זהו למעשה שלב ההפעלה של כל מערכת ה-FTP.

stopServer (פעולה):

פעולה שמכבה את השרת בצורה מסודרת.

במהלך הפעולה:

- נעצרת קבלת חיבורים חדשים
- נסגרים סוקטיים פעילים
- והשרת עובר למצב לא פעיל (isRunning = false)



תפקיד המחלקה UserService

המחלקה UserService היא שכבת השירות המרכזית במערכת האחראית על כל הלוגיקה העסקית הקשורה למשתמשים.

היא אינה מתקשרת ישירות עם ממשק המשתמש ואינה מנהלת UI, אלא משמשת כמתווך (Service Layer) בין:

שכבת הנתונים (UserRepository)

לבין שכבות גבוהות יותר כמו Controllers ו-Views

מטרת המחלקה היא לרכז את כל הפעולות על משתמשים במקום אחד, כך שהמערכת תהיה מודולרית, נקייה וקלה לתחזוקה.

בנוסף, המחלקה אחראית על אכיפת כללי עסק (Business Rules), כמו מניעת משתמשים כפולים, אימות התחברות, ועדכון עקבי של נתוני משתמשים.

תכונות מרכזיות

userRepository (תכונה)

אובייקט גישה למסד הנתונים של המשתמשים.

באמצעותו מתבצעות כל פעולות ה-CRUD:

- יצירה של משתמשים חדשים
- קריאה ושליפה של משתמשים

- עדכון פרטי משתמש
- מחיקה של משתמשים

המחלקה למעשה לא שומרת נתונים בעצמה, אלא מסתמכת לחלוטין על ה-Repository. פעולות מרכזיות

UserService (בנאי)

פעולת אתחול המחלקה.

מקבלת את UserRepository דרך Dependency Injection, מה שמאפשר:

- הפרדה בין שכבות
- בדיקות יחידה קלות יותר
- גמישות בהחלפת מקור הנתונים בעתיד

addUserToDB (פעולה)

אחראית על הוספת משתמש חדש למערכת.

תהליך העבודה כולל:

- בדיקה אם המשתמש כבר קיים במערכת
- ולידציה בסיסית לנתונים
- שמירה במסד הנתונים

מחזירה true אם ההוספה הצליחה, או false במקרה של כישלון.

loginUser (פעולה)

מבצעת אימות משתמש מול המערכת.

הפעולה:

- מקבלת אימייל וסיסמה
- מבצעת בדיקה מול הנתונים השמורים
- מחזירה אובייקט User אם האימות הצליח
- או null אם ההתחברות נכשלה

זהו אחד הרכיבים הקריטיים באבטחת המערכת.

getAllUsers (פעולה)

מחזירה את כלל המשתמשים הקיימים במערכת.

פעולה זו משמשת בעיקר לצרכי ניהול, דוחות ובדיקות מערכת.

getOneByUsernameAndPassword (פעולה)

שליפת משתמש לפי שם משתמש וסיסמה יחד.

משמשת לאימות ישיר מול בסיס הנתונים, לעיתים כחלק מתהליך login.

deleteItem (פעולה)

אחראית על מחיקת משתמש מהמערכת.

הפעולה מסירה את המשתמש לחלוטין ממסד הנתונים, ולכן לרוב מוגבלת להרשאות מנהל.

getUser (פעולה)

פעולת עזר לשליפת משתמש לפי שם וסיסמה.

לעיתים משמשת כגרסה פנימית מקוצרת של `loginUser`.

updateProfile (פעולה)

אחראית על עדכון פרטי משתמש קיימים.

במהלך הפעולה מתבצע:

- עדכון שם משתמש
 - עדכון כתובת אימייל
 - שמירה מחדש במסד הנתונים
 - שמירה על עקביות מול קבצים קיימים של המשתמש
- זהו תהליך רגיש, במיוחד כאשר האימייל משמש כמזהה ראשי במערכת.

C WeatherService
■ <code>restTemplate: RestTemplate = new RestTemplate()</code>
● <code>fetchCurrentWeather(latitude: double, longitude: double): String</code>

תפקיד המחלקה WeatherService

המחלקה `WeatherService` אחראית על ניהול ושליפת נתוני מזג האוויר במערכת באמצעות שירות API חיצוני.

זהו רכיב חשוב בשכבת ה-`Service` של המערכת, מכיוון שהוא מחבר בין:

- המערכת
 - שירות מזג האוויר החיצוני `Open-Meteo`
 - ותצוגת הנתונים בממשק המשתמש
- באמצעות מחלקה זו המערכת יכולה:
- למשוך נתוני טמפרטורה בזמן אמת
 - להציג מידע דינמי למשתמש
 - לבצע תקשורת HTTP מול שירות חיצוני
 - לעבד נתוני JSON המתקבלים מהרשת

תכונות מרכזיות

`restTemplate` (תכונה)

רכיב התקשורת המרכזי של המחלקה.

האובייקט `RestTemplate` משמש לביצוע בקשות HTTP אל שירותי רשת חיצוניים.

באמצעות `WeatherService` מבצע:

שליחת בקשות GET ל-`Open-Meteo`

קבלת תשובות מהשרת

משיכת נתוני מזג אוויר בזמן אמת

תקשורת מול REST API חיצוני

למעשה, זהו הגשר בין המערכת לבין שירות מזג האוויר באינטרנט.

פעולות מרכזיות

fetchCurrentWeather (פעולה)

הפעולה המרכזית של המחלקה.

הפעולה מקבלת:

latitude - קו רוחב

longitude - קו אורך

באמצעות נתונים אלו נבנית כתובת URL דינמית עבור שירות Open-Meteo.

במהלך הפעולה:

- נוצרת בקשת HTTP
- נשלחת פנייה לשרת החיצוני
- מתקבלת תשובת JSON
- מתבצע חילוץ של הטמפרטורה מתוך הנתונים
- הערך מוחזר כמחרוזת מוכנה להצגה

לדוגמה: 22.5°C

הפעולה מאפשרת למערכת לקבל מידע אקלימי עדכני בצורה דינמית וללא צורך במסד נתונים מקומי.

עיבוד נתוני JSON

לאחר קבלת תשובת השרת, המחלקה משתמשת בספריית org.json לצורך פענוח המידע.

בשלב זה:

- נוצר אובייקט JSONObject
 - מתבצע חילוץ של האובייקט current
 - נשלף הערך temperature_2m
 - הנתון מומר לפורמט טקסטואלי להצגה בממשק
- כך המערכת מצליחה להמיר מידע גולמי מהרשת לנתון קריא וברור עבור המשתמש.

טיפול בשגיאות

המחלקה כוללת מנגנון try-catch לצורך טיפול בתקלות תקשורת ועיבוד מידע.

במידה שמתרחשת שגיאה:

- כשל בתקשורת אינטרנט
- שרת API לא זמין
- תשובת JSON לא תקינה
- המערכת מונעת קריסה של התהליך ומחזירה ערך ברירת מחדל: $-\text{C}^{\circ}$

גישה זו משפרת את יציבות המערכת ואת חוויית המשתמש.

תרומת המחלקה למערכת

מחלקת WeatherService תורמת למערכת במספר היבטים:

- הצגת מידע דינמי בזמן אמת
- אינטגרציה עם שירותי ענן חיצוניים
- שיפור חוויית המשתמש באמצעות נתונים חיים
- הפרדה נכונה בין שכבת ה-UI לבין שכבת התקשורת
- שימוש בתקשורת HTTP מודרנית ומבוססת REST API
- פישוט תהליך שליפת ועיבוד נתוני מזג האוויר במערכת

C AppNavbarLayout
■ navigationContainer: HorizontalLayout = new HorizontalLayout() ■ rightSectionContainer: HorizontalLayout = new HorizontalLayout()
● AppNavbarLayout() ■ createNavbar(): void ◆ onAttach(attachEvent: com.vaadin.flow.component.AttachEvent): void ● refreshNavbarContent(): void ● <u>refreshCurrentNavbar(): void</u> ● profile(): void ● logout(): void

תפקיד המחלקה AppNavbarLayout

המחלקה AppNavbarLayout אחראית על בניית וניהול סרגל הניווט העליון (Navbar) של מערכת ה-UI.

היא משמשת כרכיב תשתיתי קבוע שמופיע כמעט בכל המסכים באפליקציה, ומספקת למשתמש גישה מהירה לפעולות מרכזיות כמו:

- מעבר לפרופיל המשתמש
- התנתקות מהמערכת
- ועדכון מצב התצוגה בהתאם למצב ההתחברות

navigationContainer (תכונה)

מכיל את כל רכיבי הניווט בצד השמאלי של ה-Navbar. בתוך רכיב זה מוצגים לרוב:

- כפתורי ניווט למסכים שונים
- לוגו או שם המערכת
- קישורים מרכזיים

הוא בנוי כ-`HorizontalLayout` כדי לאפשר סידור אופקי של האלמנטים.

rightSectionContainer (תכונה)

מכיל את החלק הימני של סרגל הניווט.

בדרך כלל כולל:

- שם המשתמש המחובר
- כפתור פרופיל
- כפתור התנתקות

גם הוא מבוסס על `HorizontalLayout` לצורך יישור רכיבים בצורה נקייה ואחידה.

פעולות מרכזיות**AppNavbarLayout (בנאי)**

פעולת יצירת הרכיב.

במהלך הבנאי מתבצע:

- יצירת מבנה ה-`Navbar`
- אתחול הקונטיינרים
- קריאה לפונקציה `createNavbar` לבניית התוכן בפועל

זהו שלב האתחול הראשוני של רכיב ה-UI.

createNavbar (פעולה)

אחראית על בניית כל רכיבי ה-`Navbar` בפועל.

במהלך הפעולה:

- נוצרים כפתורי ניווט
 - מוגדרים רכיבי ימין ושמאל
 - מתבצע עיצוב ראשוני של הסרגל
- זו הפעולה שבונה את המבנה הוויזואלי של הסרגל.

onAttach (פעולה מוגנת)

פעולה שמופעלת כאשר הרכיב נטען לתוך המסך (`Lifecycle` של `Vaadin`).

בשלב זה ניתן:

- לרענן נתונים דינמיים
- לבדוק אם משתמש מחובר
- לעדכן תצוגת `Navbar` בהתאם למצב המערכת

refreshNavbarContent (פעולה)

אחראית על רענון תוכן ה-`Navbar` בזמן ריצה.

לדוגמה:

- שינוי בין משתמש אורח למשתמש מחובר
- עדכון שם משתמש
- שינוי כפתורים בהתאם להרשאות

refreshCurrentNavbar (פעולה סטטית)

מאפשרת לרענן את ה-Navbar מכל מקום במערכת.

זהו מנגנון חשוב כאשר יש שינוי מצב גלובלי, למשל:

- התחברות
- התנתקות
- עדכון פרופיל

profile (פעולה)

מעבירה את המשתמש למסך הפרופיל שלו.

פותרת:

דיאלוג פרופיל

logOut (פעולה)

מבצעת התנתקות מהמערכת.

במהלך הפעולה:

- מנקים Session של המשתמש
- מאפסים מצב התחברות
- מחזירים למסך התחברות או דף ראשי

C	HomeView
□	ftpService: FtpUiService
□	currentUI: UI
■	grid: Grid<UploadedFileDTO> = new Grid<>(UploadedFileDTO.class, false)
□	activeUser: User
●	HomeView(ftpService: FtpUiService)
●	beforeEnter(event: BeforeEnterEvent): void
■	buildUI(): void
■	refreshGrid(): void

תפקיד המחלקה HomeView

המחלקה HomeView היא המסך המרכזי של המערכת (Main Dashboard), והיא אחראית על ניהול והצגת כל הקבצים של המשתמש.

זהו אחד הרכיבים החשובים ביותר בשכבת ה-UI, מכיוון שהוא מחבר בין:

המשתמש

שירות הקבצים (FtpUiService)

ותצוגת הנתונים בפועל

במסך זה המשתמש יכול:

- לראות את כל הקבצים שהעלה
- לרענן את רשימת הקבצים
- ולבצע פעולות כמו העלאה, הורדה ומחיקה (דרך שירותים חיצוניים)

תכונות מרכזיות

ftpService (תכונה)

שירות הליבה של המערכת לניהול קבצים.

באמצעות HomeView מבצע:

שליפת קבצים מהשרת

הורדת קבצים

מחיקת קבצים

קבלת נתונים להצגה בטבלה

למעשה, זה הגשר בין ה-UI לבין כל שכבת הקבצים וה-FTP.

currentUI (תכונה)

מייצג את מופע ה-UI הנוכחי של Vaadin.

משמש עבור:

עדכוני מסך

פעולות UI דינמיות

סנכרון מצב המשתמש מול הממשק

grid (תכונה)

טבלת הנתונים המרכזית של המסך.

ה-Grid מציג רשימת קבצים מסוג:

- UploadedFileDTO

- בתצוגה זו המשתמש רואה:

- שם קובץ

- סוג קובץ

- גודל

- זמן העלאה

זהו הרכיב הוויזואלי המרכזי של המסך.

activeUser (תכונה)

מייצג את המשתמש המחובר הנוכחי.

באמצעות המערכת יודעת:

- איזה קבצים להציג

- אילו פעולות מותרות

- ואיזה נתונים לשלוח מהשרת

פעולות מרכזיות**HomeView (בנאי)**

פעולת יצירת המסך.

במהלך הבנאי:

- מוזרק ftpService
- מתבצע אתחול רכיבי UI
- נבנה המבנה הראשי של הדף

זהו שלב יצירת עמוד הבית.

beforeEnter (פעולה)

פעולה שמופעלת לפני כניסה למסך (Vaadin Navigation Guard).

תפקידה:

- לבדוק אם המשתמש מחובר
 - לאמת הרשאות גישה
 - למנוע גישה ממשתמשים לא מורשים
- אם המשתמש לא מחובר - הוא מנותב למסך התחברות.

buildUI (פעולה)

אחראית על בניית ממשק המשתמש בפועל.

במהלך הפעולה:

נוצר ה-Grid**מתווספים כפתורים (Upload / Refresh / Delete)**

מוגדרים Layoutים וסידור המסך

זו הפעולה שבונה את כל ה-UI הוויזואלי.

refreshGrid (פעולה)

אחראית על רענון הנתונים בטבלה.

במהלך הפעולה:

- נשלפים מחדש הקבצים מהשרת דרך ftpService
- הנתונים מומרות ל-UploadedFileDTO
- וה-Grid מתעדכן בזמן אמת

פעולה זו מאפשרת לראות שינויים מיד לאחר העלאה/מחיקה.



תפקיד המחלקה LoginView

המחלקה LoginView מייצגת את מסך ההתחברות של המערכת (Login Screen). זוהי מחלקת UI האחראית על תהליך הזדהות המשתמש מול המערכת. המחלקה מאפשרת למשתמש:

- להזין שם משתמש וסיסמה
- לבצע התחברות
- לקבל גישה למערכת לאחר אימות מוצלח

מחלקה זו מהווה נקודת הכניסה הראשית למערכת, ולכן היא חלק חשוב מאוד בשכבת ה־UI והאבטחה. ה־LoginView פועל מול שירות המשתמשים (UserService) לצורך:

- אימות פרטי התחברות
- שליפת נתוני משתמש
- בדיקת הרשאות וגישה

תכונות מרכזיות

למחלקה זו לא מוצגות תכונות ב־UML, אך מתוך הבנאי ניתן להבין שקיים שימוש ב־UserService. שירות userService (שירות) האחראי על ניהול משתמשים ואימות התחברות. באמצעותו המחלקה יכולה:

- לבדוק אם שם המשתמש והסיסמה תקינים
- לאתר משתמש במסד הנתונים
- לטעון מידע על המשתמש המחובר
- להתחיל Session עבור המשתמש

שירות זה מהווה את החיבור בין מסך ההתחברות לבין שכבת הלוגיקה העסקית של המערכת.

פעולות מרכזיות

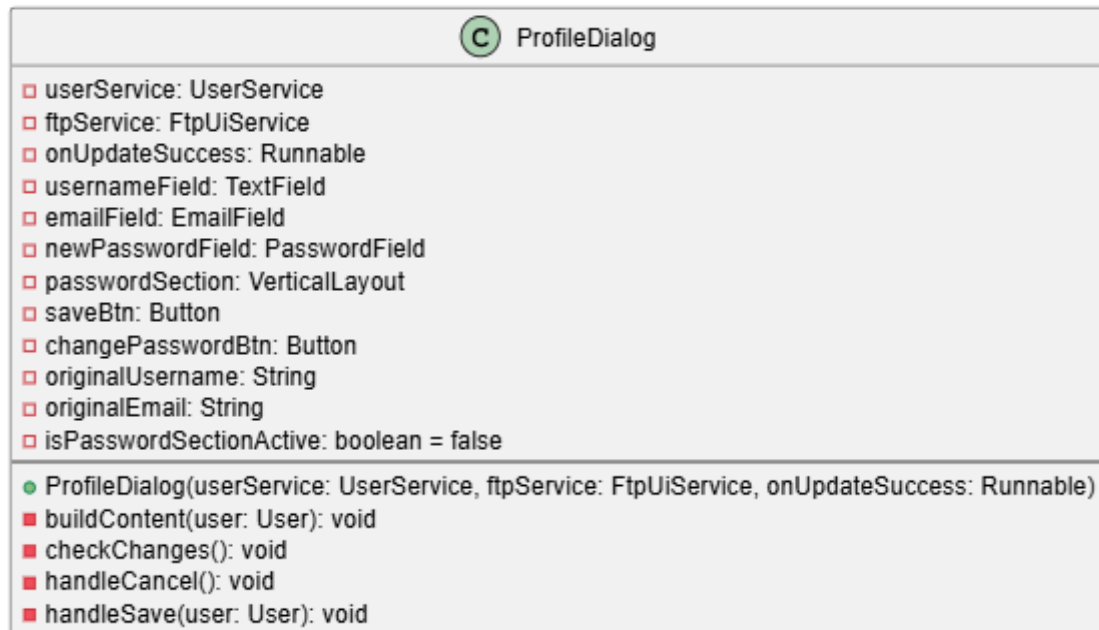
LoginView (בנאי)

פעולת יצירת מסך ההתחברות. במהלך הבנאי:

- מוזרק UserService
- נבנים רכיבי ה־UI של מסך ההתחברות
- נוצרים שדות קלט עבור שם משתמש וסיסמה
- מוגדר כפתור Login
- מוגדרים מאזיני אירועים (Listeners) ללחיצה על התחברות

בנוסף, הבנאי אחראי על הכנת כל מבנה המסך והצגתו למשתמש.
בעת ניסיון התחברות:

- הנתונים נשלחים ל-UserService
 - מתבצע אימות משתמש
 - במקרה הצלחה המשתמש מועבר למסך הראשי
 - במקרה כישלון מוצגת הודעת שגיאה מתאימה
- מחלקה זו אחראית על חוויית ההתחברות הראשונית ועל אבטחת הכניסה למערכת.



תפקיד המחלקה ProfileDialog

המחלקה ProfileDialog מייצגת חלון דיאלוג (Dialog Window) לניהול ועדכון פרטי המשתמש במערכת.

מדובר ברכיב UI אינטראקטיבי המאפשר למשתמש לערוך את פרטי החשבון האישי שלו בצורה מאובטחת ונוחה.

- המחלקה אחראית על:
 - הצגת פרטי המשתמש הנוכחיים
 - עריכת שם משתמש ואימייל
 - שינוי סיסמה
 - שמירת השינויים במערכת
 - בדיקת תקינות ועדכון המידע
- בנוסף, המחלקה פועלת מול שירותי המערכת (UserService ו-FtpUiService) ליצור עדכון הנתונים וסנכרון המידע לאחר השינוי.
- זהו רכיב חשוב בשכבת ה-UI והניהול האישי של המשתמש.

תכונות מרכזיות**userService (תכונה)**

שירות ניהול המשתמשים של המערכת.

באמצעותו המחלקה מבצעת:

- עדכון פרטי משתמש
- שינוי סיסמה
- אימות נתונים
- שמירת שינויים במסד הנתונים

זהו השירות המרכזי לטיפול בכל פעולות המשתמש.

ftpService (תכונה)

שירות ניהול הקבצים של המערכת.

- באמצעותו ניתן:
 - לסנכרן שינויים הקשורים למשתמש
 - לעדכן נתוני קבצים בהתאם לשינויי משתמש
 - לנהל מידע הקשור לחשבון המשתמש במערכת הקבצים
- שירות זה מחבר בין ניהול המשתמש לבין שכבת הקבצים.

onUpdateSuccess (תכונה)

פונקציית Callback מסוג Runnable.

תפקידה:

- להפעיל פעולה לאחר עדכון מוצלח של פרטי המשתמש
- לרענן מסכים
- לעדכן מידע ב-UI
- לסנכרן נתונים בממשק

כך ניתן לבצע עדכון דינמי של המערכת לאחר שמירת שינויים.

usernameField (תכונה)

שדה קלט עבור שם המשתמש.

מאפשר:

- צפייה בשם המשתמש הנוכחי
- עריכת השם
- שליחת ערך חדש לעדכון

emailField (תכונה)

שדה קלט מסוג EmailField.

מאפשר למשתמש:

לעדכן כתובת אימייל

לבצע ולידציה בסיסית של מבנה האימייל

זהו רכיב חשוב לשמירה על תקינות נתוני המשתמש.

newPasswordField (תכונה)

שדה קלט לסיסמה חדשה.

משמש עבור:

- שינוי סיסמת המשתמש
- קבלת קלט מאובטח (מוסתר)
- שליחת הסיסמה לעדכון

passwordSection (תכונה)

Layout המכיל את אזור שינוי הסיסמה.

באמצעותו ניתן:

- להציג או להסתיר את אזור שינוי הסיסמה
- לארגן את רכיבי שינוי הסיסמה בצורה ויזואלית

saveBtn (תכונה)

כפתור שמירת השינויים.

בלחיצה עליו:

- מתבצעת בדיקת תקינות
- הנתונים נשמרים
- נשלחת בקשת עדכון לשירותי המערכת

changePasswordBtn (תכונה)

כפתור להפעלת מצב שינוי סיסמה.

תפקידו:

- לפתוח את אזור שינוי הסיסמה
- לאפשר למשתמש לעדכן סיסמה רק בעת הצורך

originalUsername / originalEmail (תכונה)

שומרים את הערכים המקוריים של המשתמש לפני העריכה.

באמצעותם ניתן:

- לזהות אם בוצעו שינויים
- למנוע שמירה מיותרת
- לבצע השוואה מול הנתונים החדשים

isPasswordSectionActive (תכונה)

משתנה בוליאני המציין האם אזור שינוי הסיסמה פעיל.

באמצעותו המערכת יודעת:

- האם להציג את שדות הסיסמה
- האם לבצע בדיקות הקשורות לסיסמה
- האם המשתמש בחר לשנות סיסמה

פעולות מרכזיות**ProfileDialog (בנאי)**

פעולת יצירת חלון ניהול הפרופיל.

במהלך הבנאי:

- מוזרקים השירותים הדרושים
 - נטענים נתוני המשתמש
 - נבנים רכיבי ה-UI
 - מוגדרים מאזיני אירועים לכפתורים
 - מוגדר מצב התחלתי של החלון
- הבנאי אחראי להכנת כל ממשק ניהול הפרופיל.

buildContent (פעולה)

אחראית על בניית תוכן חלון הדיאלוג.

במהלך הפעולה:

- נטענים נתוני המשתמש
 - נוצרים שדות הקלט
 - מוגדרים Layoutים
 - נבנים כפתורי הפעולה
 - מוצג אזור שינוי הסיסמה במידת הצורך
- זוהי הפעולה שבונה את כל ה-UI הוויזואלי של חלון הפרופיל.

checkChanges (פעולה)

אחראית על בדיקת שינויים שביצע המשתמש.

הפעולה בודקת:

- האם שם המשתמש השתנה
- האם האימייל השתנה
- האם הוזנה סיסמה חדשה

באמצעות פעולה זו ניתן:

- להפעיל/לנטרל את כפתור השמירה
- למנוע שמירה מיותרת
- לשפר ביצועים וחווית משתמש

handleCancel (פעולה)

אחראית על ביטול פעולת העריכה.

במהלך הפעולה:

- החלון נסגר
- השינויים שלא נשמרו מבוטלים
- המערכת חוזרת למצב הקודם
- פעולה זו מאפשרת יציאה בטוחה מעריכת הפרופיל.

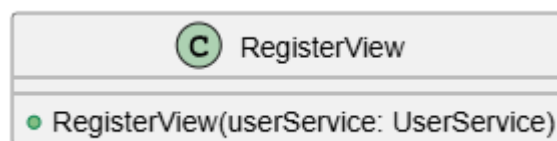
handleSave (פעולה)

אחראית על שמירת השינויים שביצע המשתמש.

במהלך הפעולה:

- מתבצעת ולידציה לנתונים
- נבדקת תקינות המידע
- מתבצע עדכון דרך UserService
- במידת הצורך מתבצע שינוי סיסמה
- מופעל onSuccessUpdate
- המידע בממשק מתרענן

זוהי הפעולה המרכזית של המחלקה, מכיוון שהיא מבצעת בפועל את עדכון פרטי המשתמש במערכת.



תפקיד המחלקה RegisterView

המחלקה RegisterView מייצגת את מסך ההרשמה של המערכת (Registration Screen). מחלקה זו אחראית על יצירת משתמשים חדשים ומתן אפשרות לפתיחת חשבון במערכת. זהו רכיב מרכזי בשכבת ה-UI, מכיוון שהוא מאפשר למשתמשים חדשים:

- להזין פרטי הרשמה
- ליצור חשבון חדש
- לבצע אימות נתונים
- להצטרף למערכת בצורה מאובטחת

המחלקה פועלת מול UserService לצורך:

- יצירת משתמש חדש
- בדיקת תקינות נתונים
- מניעת כפילויות משתמשים
- שמירת המשתמש במסד הנתונים

מסך זה מהווה חלק חשוב ממערכת ניהול המשתמשים והאבטחה של המערכת.

תכונות מרכזיות

userService (שירות)

שירות ניהול המשתמשים של המערכת.

באמצעותו המחלקה מבצעת:

- יצירת משתמש חדש
- שמירת משתמש במסד הנתונים
- בדיקת קיום משתמש קיים
- אימות נתוני הרשמה
- ניהול תהליך ההרשמה

שירות זה מהווה את החיבור בין מסך ההרשמה לבין הלוגיקה העסקית של המערכת.

פעולות מרכזיות

RegisterView (בנאי)

פעולת יצירת מסך ההרשמה.

במהלך הבנאי:

מוזרק UserService

נבנים רכיבי ה-UI של טופס ההרשמה

נוצרים שדות קלט עבור:

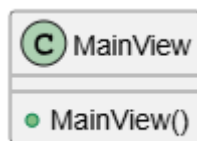
- שם משתמש
- אימייל
- סיסמה
- מוגדר כפתור הרשמה
- מוגדרים מאזיני אירועים (Listeners)
- מוגדרות בדיקות תקינות בסיסיות

בנוסף, הבנאי אחראי על עיצוב וסידור ממשק המשתמש של מסך ההרשמה.

בעת שליחת טופס ההרשמה:

- הנתונים נשלחים ל-UserService
- מתבצעת בדיקת תקינות
- נבדק האם המשתמש כבר קיים
- במקרה הצלחה נוצר חשבון חדש
- במקרה כישלון מוצגת הודעת שגיאה מתאימה

מחלקה זו אחראית על תהליך יצירת החשבון ועל הכנסת משתמשים חדשים למערכת בצורה תקינה ומאובטחת.



תפקיד המחלקה MainView

המחלקה **MainView** מייצגת את מסך הפתיחה הראשי של המערכת (Landing Page / Welcome Screen).

זהו המסך הראשוני שמוצג למשתמש בעת כניסה לאפליקציה, והוא אחראי על הצגת המערכת, היכולות שלה והניווט הראשוני.

המחלקה שייכת לשכבת ה־UI ותפקידה העיקרי הוא:

- להציג מידע ראשוני על המערכת
 - לשפר את חוויית המשתמש הראשונית
 - להציג פיצ'רים ויכולות מרכזיות
 - לספק ממשק ניווט ברור למסכי התחברות והרשמה
- מסך זה משמש כעמוד כניסה שיווקי ותפעולי של המערכת.

MainView (בנאי)

פעולת יצירת מסך הפתיחה.

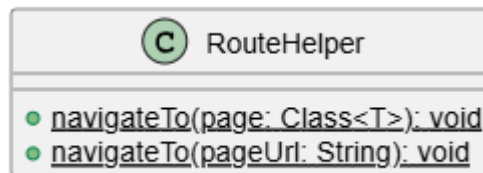
במהלך הבנאי:

- נבנה מבנה העמוד הראשי
- נוצרים Layoutים של המסך
- מוגדרים אזורי תוכן
- מוצגים פיצ'רים מרכזיים של המערכת
- מוגדר עיצוב ויזואלי ראשוני

בנוסף, הבנאי אחראי על בניית חוויית המשתמש הראשונית ועל הצגת המידע המרכזי בצורה ברורה ומסודרת.

מסך זה כולל:

- כותרת מערכת
- תמונה של הפרויקט
- תיאור קצר



תפקיד המחלקה RouteHelper

המחלקה RouteHelper היא מחלקת עזר (Utility Class) האחראית על ניהול וניווט בין עמודים במערכת.

מחלקה זו מרכזת את פעולות המעבר בין מסכים שונים באפליקציית ה-UI ומאפשרת ביצוע ניווט בצורה פשוטה ואחידה.

המחלקה שייכת לשכבת ה-UI והתשתית, ותפקידה:

- לבצע מעבר בין Views שונים
 - לנהל ניווט לפי מחלקה או URL
 - להפחית שכפול קוד
 - לפשט את השימוש במנגנון הניווט של Vaadin
- במקום שכל מסך יבצע ניווט בצורה ידנית, המחלקה מספקת ממשק מרכזי ואחיד לכל פעולות הניווט במערכת.

פעולות מרכזיות

navigateTo(page: Class<T>) (פעולה סטטית)

פעולה האחראית על מעבר למסך לפי מחלקת ה-View.

הפעולה מקבלת:

מחלקת יעד (<Class<T>)

ומבצעת:

- ניווט למסך המתאים במערכת
 - מעבר View בתוך מנגנון הניווט של Vaadin
- שימוש בגישה זו מאפשר:
- ניווט בטוח יותר מבחינת טיפוסים (Type Safe)
 - מניעת טעויות בכתיבת URLs
 - קישור ישיר בין מחלקות המסכים

לדוגמה:

מעבר ל-HomeView

מעבר ל-LoginView

מעבר ל-RegisterView

פעולה זו משמשת בעיקר כאשר עובדים ישירות מול מחלקות ה-UI של המערכת.

navigateTo(pageUrl: String) (פעולה סטטית)

פעולה האחראית על מעבר למסך לפי כתובת URL.

הפעולה מקבלת:

מחרוזת URL

ומבצעת:

- ניווט לעמוד המתאים במערכת

- מעבר דינמי לפי נתיב

גישה זו מאפשרת:

- ניווט גמיש ודינמי

- עבודה עם Routes דינמיים

- שימוש בכתובות URL מותאמות

פעולה זו שימושית במיוחד כאשר:

- הנתיב נבנה בזמן ריצה

- מתבצע Redirect

- יש צורך בניווט דינמי מתוך נתונים חיצוניים

חשיבות המחלקה במערכת

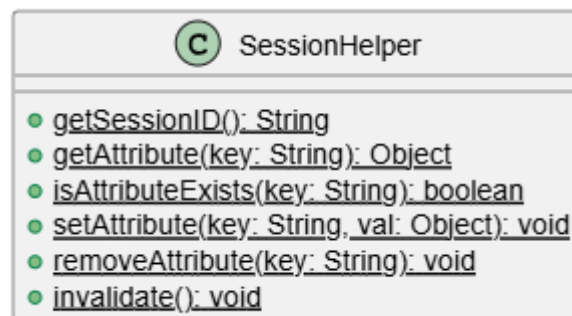
המחלקה RouteHelper משפרת את ארכיטקטורת המערכת בכך שהיא:

- מרכזת את כל פעולות הניווט

- מפחיתה שכפול קוד במסכים שונים

- משפרת קריאות ותחזוקת קוד

- מספקת שכבת הפשטה מעל מנגנון הניווט של Vaadin



תפקיד המחלקה SessionHelper

המחלקה SessionHelper היא מחלקת עזר (Utility Class) האחראית על ניהול ה־Session של המשתמש במערכת. מחלקה זו מרכזת את כל פעולות העבודה מול ה־Session ומספקת ממשק פשוט ואחיד לניהול מידע זמני של המשתמש המחובר.

המחלקה שייכת לשכבת התשתית והאבטחה של המערכת, ותפקידה:

- לשמור מידע על המשתמש המחובר
- לנהל נתוני Session
- לאפשר גישה נוחה לנתונים זמניים
- לנהל התחברות והתנתקות
- לבצע ניקוי Session בעת Logout

במקום לגשת ישירות למנגנון ה־Session של Vaadin או השרת, המחלקה מספקת שכבת הפשטה מסודרת ונוחה לשימוש.

פעולות מרכזיות

getSessionID (פעולה סטטית)

אחראית על שליפת מזהה ה־Session הנוכחי של המשתמש. הפעולה מחזירה:

- String המכיל את מזהה ה־Session
- מזהה זה מאפשר:
- זיהוי ייחודי של המשתמש המחובר
- ניהול אבטחה
- מעקב אחר Session פעיל
- בדיקות התחברות

פעולה זו שימושית במיוחד עבור מנגנוני אבטחה ולוגים.

getAttribute (פעולה סטטית)

אחראית על שליפת ערך מתוך ה-Session.

הפעולה מקבלת:

key - מפתח של הנתון המבוקש

ומחזירה:

אובייקט (Object) המאוחסן תחת אותו מפתח

באמצעות פעולה זו ניתן לשלוף:

- משתמש מחובר
- הרשאות
- נתוני UI זמניים
- מידע זמני של המערכת

isAttributeExists (פעולה סטטית)

בודקת האם קיים ערך מסוים בתוך ה-Session.

הפעולה מקבלת:

key - מפתח לבדיקה

ומחזירה: boolean

באמצעות פעולה זו ניתן:

- לבדוק אם משתמש מחובר
- לבדוק קיום נתונים זמניים
- למנוע גישה לנתונים לא קיימים

פעולה זו מסייעת במניעת שגיאות וגישה לא תקינה ל-Session.

setAttribute (פעולה סטטית)

אחראית על שמירת נתון בתוך ה-Session.

הפעולה מקבלת:

key - שם הנתון

val - הערך לשמירה

באמצעות פעולה זו ניתן לשמור:

- משתמש מחובר
- Token
- הרשאות
- מידע זמני עבור ה-UI

הפעולה מאפשרת שיתוף מידע בין מסכים שונים במהלך העבודה של המשתמש.

removeAttribute (פעולה סטטית)

אחראית על מחיקת נתון מתוך ה־Session.

הפעולה מקבלת:

key - המפתח של הנתון למחיקה

תפקידה:

- לנקות נתונים זמניים
 - להסיר מידע ישן
 - למחוק מידע שאינו דרוש יותר
- פעולה זו מסייעת בניהול זיכרון ובשמירה על Session נקי.

invalidate (פעולה סטטית)

אחראית על סיום ה־Session הנוכחי של המשתמש.

במהלך הפעולה:

- כל נתוני ה־Session נמחקים
 - המשתמש מתנתק מהמערכת
 - ה־Session נהרס
- פעולה זו משמשת בעיקר עבור:

- Logout
- סיום התחברות
- ניקוי מידע רגיש
- אבטחת המערכת

זוהי אחת הפעולות החשובות ביותר במנגנון ניהול המשתמשים והאבטחה.

חשיבות המחלקה במערכת

המחלקה SessionHelper משפרת את מבנה המערכת בכך שהיא:

- מרכזת את כל פעולות ה־Session במקום אחד
- מפשטת עבודה מול מנגנון ה־Session
- משפרת קריאות ותחזוקת קוד
- מפחיתה שכפול קוד
- משפרת אבטחה וניהול משתמשים

מדובר במחלקת תשתית חשובה מאוד לניהול משתמשים, התחברות ואבטחת מידע במערכת.

12.3 מבני נתונים בהם נעשה שימוש

במהלך פיתוח המערכת שולבו מספר מבני נתונים מתקדמים ובסיסיים, אשר נבחרו בקפידה כדי לספק זמני גישה מהירים ($O(1)$) ולתמוך בהזרמת הנתונים הכבדה של שרת ה-FTP. להלן מבני הנתונים העיקריים:

1. מפת גיבוב (HashMap) ועץ תחיליות (Trie)

סוג מבנה הנתונים: אוסף מסוג מפתח-ערך (Key-Value Dictionary) הבנוי כמבנה עצי מבוסס תחיליות.

איפה השתמשנו בו בקוד: במחלקת LZ78Service (בפונקציה compress), שם הוגדרה מחלקה פנימית Node המחזיקה children Map<Byte, Node>.

לאיזה צורך הוא משמש: מבנה זה משמש לבניית המילון הדינמי בזמן ריצה עבור אלגוריתם הדחיסה LZ78. כאשר המערכת קוראת קובץ, היא צריכה לבדוק האם רצף הבתים (Bytes) הנוכחי כבר הופיע בעבר. השימוש בעץ מבוסס HashMap מאפשר לבדוק האם תו מסוים הוא "ילד" של הרצף הנוכחי בסיבוכיות זמן של קבועה $O(1)$. זהו המפתח לביצועי דחיסה בזמן אמת (On-the-fly) שאינם תוקעים את תהליך ההעלאה של המשתמש.

2. מערך דינמי (ArrayList) כרשימת מילון

סוג מבנה הנתונים: מערך דינמי (Dynamic Array).

איפה השתמשנו בו בקוד: במחלקת LZ78Service (בפונקציה decompress), הוגדר כ-
`dict = new ArrayList<List<byte[]>>()`.

לאיזה צורך הוא משמש: מבנה זה מאחסן את מילון התחיליות בזמן פריסת הקובץ חזרה לפורמט המקורי. להבדיל מדחיסה, בפריסה אנחנו מקבלים מספר שלם (Index) וצריכים לשלוף באופן מיידי את מחרוזת הבתים התואמת לו. מכיוון ש-ArrayList שומר את איבריו בבלוק רציף בזיכרון, פעולת השליפה לפי אינדקס (`dict.get(index)`) מבוצעת בסיבוכיות זמן $O(1)$. זה קריטי כדי שהקובץ ישוחזר במהירות המקסימלית ללא השתיות בהורדה.

3. רשימות דינמיות מקושרות/מערכים (List של אובייקטים)

סוג מבנה הנתונים: רשימה (List).

איפה השתמשנו בו בקוד: 1. במחלקה User כרשימת מחרוזות: `List<String> accessibleFileIds`.

2. במחלקה FtpUiService להחזרת נתוני התצוגה: `<List<UploadedFileDTO>`.

לאיזה צורך הוא משמש: * ברמת מסד הנתונים (מחלקה User), הרשימה שומרת באופן גמיש את כל מזהי הקבצים (IDs) השייכים ללקוח. מכיוון שכמות הקבצים שמשתמש מעלה משתנה ואינה ידועה מראש, מערך סטטי לא יתאים כאן. ה-List מאפשר הוספה והסרה של קבצים (Delete) בצורה דינמית.

ברמת הממשק (UI), הרשימה אוספת את כל המטא-דאטה מ-MongoDB ומזינה אותו ישירות אל רכיב הטבלה (Grid) של חזית המערכת (Vaadin), אשר מצפה לקבל אוסף איטרטיבי (Iterable) שניתן לרוץ עליו בלולאה ולהציגו שורה אחר שורה.

4. מערכי בתים סטטיים (Static Byte Arrays)

סוג מבנה הנתונים: מערך פרימיטיבי רציף (byte[]).

איפה השתמשנו בו בקוד: בתוך מחלקת LZ78Service בכל פעם שמאתרים רצף חדש, למשל

בשורה: `byte[] entry = new byte[prefix.length + 1];`

לאיזה צורך הוא משמש: טיפול בזרם הנתונים הגולמי (Raw Data) ברמה הנמוכה ביותר של הזיכרון. כאשר האלגוריתם שלנו יוצר רצף חדש במילון (תחילית קיימת + בית חדש), השימוש במערך בתים פשוט מאפשר לנו לנצל את הפקודה המובנית של ג'אווה `System.arraycopy`. פקודה זו מבצעת העתקה של בלוק זיכרון ברמת החומרה (Native), מה שמקטין את עומס העיבוד (CPU) של השרת למינימום ומאפשר שחזור קבצים מהיר ביותר.

12.4 קוד התוכנית לפעולות ואלגוריתמים עיקריים וחשובים

בסעיף זה יוצגו 5 פונקציות ואלגוריתמים מרכזיים המהווים את ליבת המערכת - החל ממנוע הדחיסה, דרך הזרמת הנתונים בשרת ה-FTP המקומי, ועד לעדכון זהויות מול מסד הנתונים.

1. אלגוריתם הדחיסה compress (מתוך LZ78Service)

פונקציה זו היא הלב האלגוריתמי של הפרויקט, המיישמת דחיסת נתונים בזמן אמת בשיטת LZ78 באמצעות מבנה נתונים של עץ תחיליות (Trie).

```
public void compress(InputStream in, OutputStream out) throws IOException {
    DataOutputStream dataOut = new DataOutputStream(new BufferedOutputStream(out, 256 *
1024));
    InputStream bufferedIn = (in instanceof BufferedInputStream) ? in : new
BufferedInputStream(in, 256 * 1024);
    Node root = new Node(0);
    Node current = root;
    int dictSize = 1;
    int read;
    while ((read = bufferedIn.read()) != -1) {
        byte b = (byte) read;
        Node next = current.children.get(b);
        if (next != null) {
            current = next;
        } else {
            dataOut.writeInt(current.index);
            dataOut.writeBoolean(true);
            dataOut.writeByte(b);

            if (dictSize < 450000) {
                current.children.put(b, new Node(dictSize++));
            } else {
                root = new Node(0);
                dictSize = 1;
            }
            current = root;
        }
    }
    if (current != root) {
        dataOut.writeInt(current.index);
        dataOut.writeBoolean(false);
    }
    dataOut.flush();
}
```

טענת כניסה (Parameters): מקבלת in (זרם קלט גולמי לקריאה) ו-out (זרם פלט לכתיבת הנתונים הדחוסים).

טענת יציאה (Return Value): לא מחזירה ערך (void). הנתונים הדחוסים נכתבים ישירות לזרם הפלט. תזרוק IOException במקרה של שגיאת קריאה/כתיבה.

מה הפונקציה מבצעת: דחיסת קובץ באמצעות בניית מילון תחליות דינמי ואיתור רצפים חוזרים. אופן פעולתה המפורט: הפונקציה קוראת את המידע בית אחר בית אל תוך חוצץ. היא מחפשת את הרצף הנוכחי במילון. כאשר היא נתקלת ברצף חדש, היא כותבת לפלט את אינדקס הרצף המוכר האחרון יחד עם התו החדש, ומוסיפה את הרצף החדש לעץ. במידה והמילון חורג מהגודל המקסימלי שנקבע (למניעת זליגת זיכרון), הוא מתאפס ומתחיל מחדש.

סיבוכיות זמן ריצה: $O(N)$, כאשר N הוא מספר הבתים (Bytes) בקובץ המקור. המעבר על הקובץ מתבצע פעם אחת בלבד (Single Pass), ופעולות העץ מתבצעות בסיבוכיות קבועה.

2. אלגוריתם פריסת הנתונים decompress (מתוך LZ78Service)

פונקציה זו משחזרת את המידע שנדחס (LZ78) חזרה לפורמט המקורי ללא אובדן מידע (Lossless). כולל ניהול אקטיבי של ניתוקי רשת מצד דפדפן הלקוח.

```
public void decompress(InputStream in, OutputStream out) throws IOException {
    DataInputStream dataIn = new DataInputStream(new BufferedInputStream(in, 256 *
1024));
    BufferedOutputStream dataOut = new BufferedOutputStream(out, 256 * 1024);
    List<byte[]> dict = new ArrayList<>(450000);
    dict.add(new byte[0]);
    try {
        while (true) {
            int index = dataIn.readInt();
            boolean hasValue = dataIn.readBoolean();
            if (index < 0 || index >= dict.size())
                throw new IOException("Desynced dictionary.");
            byte[] prefix = dict.get(index);
            byte[] entry;
            if (hasValue) {
                byte value = dataIn.readByte();
                entry = new byte[prefix.length + 1];
                System.arraycopy(prefix, 0, entry, 0, prefix.length);
                entry[entry.length - 1] = value;

                if (dict.size() < 450000)
                    dict.add(entry);
                else {
                    dict.clear();
                    dict.add(new byte[0]);
                }
            } else {
                entry = prefix;
            }
        }
    }
}
```

```

        try {
            dataOut.write(entry);
        } catch (IOException e) {
            break; // ניתוק יזום במקרה שהלקוח ביטל הורדה
        }
    }
} catch (EOFException e) {
}
try {
    dataOut.flush();
} catch (IOException e) {
}
}

```

טענת כניסה in: (Parameters) (זרם המידע הדחוס שנקרא משרת ה-FTP) ו-out (זרם התגובה המקושר לדפדפן הלקוח).

טענת יציאה void: (Return Value). הנתונים המשוחזרים מועברים ישירות לדפדפן למטרת הורדה. מה הפונקציה מבצעת: קריאת רשומות LZ78 דחוסות (אינדקס + תו) ושחזורן לנתונים המקוריים במלואם.

אופן פעולתה המפורט: הפונקציה קוראת צמידים, שולפת מהמילון (המנוהל כ-ArrayList) את התחילית השמורה באינדקס, משרשרת אליה את התו החדש, ומזרימה את התוצאה לדפדפן. במקביל, היא מוסיפה את הרצף החדש למילון כדי לשמור על סנכרון עם הדחוס. הפונקציה עוטפת את שלבי ה-write וה-flush ב-try-catch כדי לעצור את הפעולה מיד אם הלקוח מבטל את ההורדה ולחסוך משאבי שרת. סיבוכיות זמן ריצה: $O(N)$, כאשר N הוא מספר רשומות הקידוד בקובץ הדחוס. שליפת המידע מהמערך דורשת זמן קבוע $O(1)$.

3. אלגוריתם הזרמת העלאה (FtpUiService מתוך) uploadAndCompressFile

פונקציה זו מהווה את הגישור ושכבת התיווך (Piping) בין ממשק המשתמש (UI), מנוע הדחיסה, שרת ה-FTP ומסד הנתונים.

```
public void uploadAndCompressFile(User user, String filename, InputStream fileData) throws
IOException {
    FTPClient ftp = createClient(user);
    String remoteName = filename.endsWith(".lz78") ? filename : filename + ".lz78";
    try (OutputStream ftpOut = ftp.storeFileStream(remoteName)) {
        if (ftpOut == null)
            throw new IOException("Failed to open FTP data connection.");
        lz78Service.compress(fileData, ftpOut);
    }
    if (ftp.completePendingCommand()) {
        FTPFile[] files = ftp.listFiles(remoteName);
        long compressedSize = (files != null && files.length > 0) ? files[0].getSize() :
0;
        FileDocument doc = fileRepository.findByOwnerEmailAndFileName(user.getEmail(),
filename);
        if (doc == null)
            doc = new FileDocument(user.getEmail(), filename, 0);
        doc.setCompressedSize(compressedSize);
        fileRepository.save(doc);
        user.setFileId(doc.getFileId());
        userRepository.save(user);
    }
    ftp.disconnect();
}
```

טענת כניסה (Parameters): אובייקט המשתמש (user), שם הקובץ הרצוי (filename), וזרם הנתונים המגיע ישירות מרכיב ההעלאה של הדפדפן (fileData).

טענת יציאה (Return Value): void. מתעדכנת רשומה ב-MongoDB והקובץ נשמר בדיסק. מה הפונקציה מבצעת: הזרמה חיה של נתונים (Zero-RAM), דחיסתם, ושמירת מטא-דאטה אודותם. אופן פעולתה המפורט: מבוססת על חיבור FTP מאובטח תחת פרטי הלקוח. הפונקציה פותחת צינור פלט לשרת ה-FTP (באמצעות storeFileStream) ומנתבת את קלט הדפדפן ישירות לפונקציית ה-compress (ללא אגירה בזיכרון המרכזי). בסיום התהליך המוצלח, היא מודדת את הגודל החדש (הדחוס) בשרת, רושמת את הנתון במסד הנתונים כ-FileDocument, ומקשרת את מזהה הקובץ לחשבון הלקוח.

סיבוכיות זמן ריצה: $O(N)$. הפעולה תלויה לינארית בזמן הדחיסה והעברת הנתונים על פני הרשת.

4. טיפול בבקשת שמירה handleStor (מתוך FtpClientHandler)

פונקציה פנימית בשרת ה-FTP המותאם אישית של המערכת, האחראית על קבלת הנתונים בפרוטוקול FTP ושמירתם בדיסק.

```
private void handleStor(String filename) {
    if (passiveServer == null) {
        writer.println("425 Use PASV first");
        return;
    }
    if (currentUserDir.isEmpty()) {
        writer.println("530 Please log in first");
        return;
    }
    File file = new File(rootDir + currentUserDir + getSafePath(filename));
    file.getParentFile().mkdirs();
    try {
        writer.println("150 Sending data");
        try (Socket ds = passiveServer.accept();
            InputStream in = ds.getInputStream();
            OutputStream out = new FileOutputStream(file)) {
            in.transferTo(out);
            out.flush();
        }
        writer.println("226 Transfer complete");
    } catch (Exception e) {
        writer.println("550 Error");
    } finally {
        closePassive();
    }
}
```

טענת כניסה (Parameters): מחרוזת המייצגת את שם הקובץ (filename), כפי שהתקבל כארגומנט לפקודת STOR בשרת.

טענת יציאה void (Return Value): תקשורת פרוטוקולית (כגון קודי 150 ו-226) נשלחת ללקוח חזרה בערוץ הבקרה.

מה הפונקציה מבצעת: מאזינה לחיבור נתונים, שואבת את המידע וכותבת אותו לקובץ על הכונן המקומי. אופן פעולתה המפורט: הפונקציה מוודאת שהלקוח במצב התחברות תקין והגדיר פורט פאסיבי (PASV). היא מרכיבה את נתיב השמירה תוך שימוש בנתיב המבודד של המשתמש (currentUserDir) בלבד (למניעת פריצות חציית-ספרייה). לבסוף, היא פותחת Socket להאזנה, משתמשת בפונקציה היעילה transferTo להזרמת הנתונים, ושולחת אישור סיום ללקוח. סיבוכיות זמן ריצה: $O(N)$, כאשר N הוא גודל הקובץ.

5. אלגוריתם עדכון זהות משתמש updateProfile (מתוך UserService)

מנהל את הלוגיקה העסקית לעדכון פרטי חשבון מול MongoDB, בדגש על ניהול שינויי מפתח ראשי בצורה בטוחה.

```
public boolean updateProfile(User currentUser, String newUsername, String newEmail) {
    String oldEmail = currentUser.getEmail();
    boolean isEmailChanged = !oldEmail.equalsIgnoreCase(newEmail);
    if (isEmailChanged) {
        if (userRepository.existsById(newEmail)) {
            return false;
        }
    }
    currentUser.setUsername(newUsername);
    if (isEmailChanged) {
        userRepository.deleteById(oldEmail);
        currentUser.setEmail(newEmail);
        userRepository.save(currentUser);
    } else {
        userRepository.save(currentUser);
    }
    return true;
}
```

טענת כניסה (Parameters):

currentUser – יישות המשתמש הקיים במערכת (מתוך הסשן).

newUsername – שם התצוגה המעודכן המבוקש.

newEmail – כתובת האימייל המעודכנת המבוקשת.

טענת יציאה boolean (Return Value): (בוליאני). יוחזר true אם העדכון בוצע, ו-false אם האימייל החדש תפוס.

מה הפונקציה מבצעת: עריכת נתוני הפרופיל של הלקוח תוך מניעת שגיאות דריסה של מפתחות ראשיים (Primary Keys).

אופן פעולתה המפורט: מכיוון שמפתח ראשי (@Id) ב-MongoDB אינו ניתן לעריכה ישירה, הפונקציה בודקת תחילה האם בוצע שינוי באימייל. אם כן, היא מריצה שאילתת תפוסה (existsById). אם האימייל פנוי, הפונקציה מוחקת את הרשומה הקיימת מהדיסק, משנה את מזהה האובייקט בזיכרון, ומבצעת שמירה (save) היוצרת רשומה חדשה וחוקית. שינוי שם משתמש בלבד מטופל כעדכון נתונים סטנדרטי. סיבוכיות זמן ריצה: O(1). פעולות שאילתה בסיסיות (Find, Delete, Save) מול מסד נתונים מבוסס מפתחות מתבצעות בסיבוכיות זמן קבועה.

13. תיאור מסד הנתונים

המערכת עושה שימוש במסד נתונים מסוג NoSQL המאוחסן בסביבת ענן (כגון: MongoDB, Firebase) הבחירה במסד נתונים מסוג זה נובעת מהצורך בגמישות מבנה הנתונים, יכולת הרחבה (Scalability) והתאמה למערכות מודרניות מבוססות רשת. מסד נתונים NoSQL הוא מסד נתונים שאינו מבוסס על מבנה טבלאי קשיח ואינו מחייב סכימה קבועה מראש. מסדים אלו מאפשרים אחסון מידע במבנים דינמיים כגון מסמכים, אובייקטים או מפתחות-ערכים, ומתאימים במיוחד למערכות שבהן סוגי הנתונים, שדות המידע או נפח הנתונים עשויים להשתנות עם הזמן. גישה זו מקלה על פיתוח ותחזוקה של מערכות מורכבות, ומאפשרת התאמה מהירה לדרישות חדשות ללא שינוי מבני משמעותי במסד הנתונים.

מסד נתונים בענן הוא מסד נתונים המאוחסן ומנוהל על גבי תשתית מרוחקת, הנגישה דרך רשת האינטרנט. שימוש במסד נתונים בענן מאפשר זמינות גבוהה, שרידות נתונים, גיבויים אוטומטיים וניהול נוח של ההרשאות והמשאבים, ללא צורך בתחזוקת שרתים פיזיים בצד המפתח. בנוסף, פתרונות ענן מאפשרים הגדלה או הקטנה של משאבי האחסון והביצועים בהתאם לצורכי המערכת.

אוספים העיקריים במערכת:

• משתמשים (Users)

אוסף זה משמש לאחסון מידע על משתמשי המערכת.

שדות עיקריים:

- מזהה משתמש ייחודי
- שם משתמש
- סיסמה מוצפנת
- תאריך יצירת חשבון

מידע זה מאפשר ניהול משתמשים, אימות זהות ובקרת גישה למערכת.

• קבצים (Files)

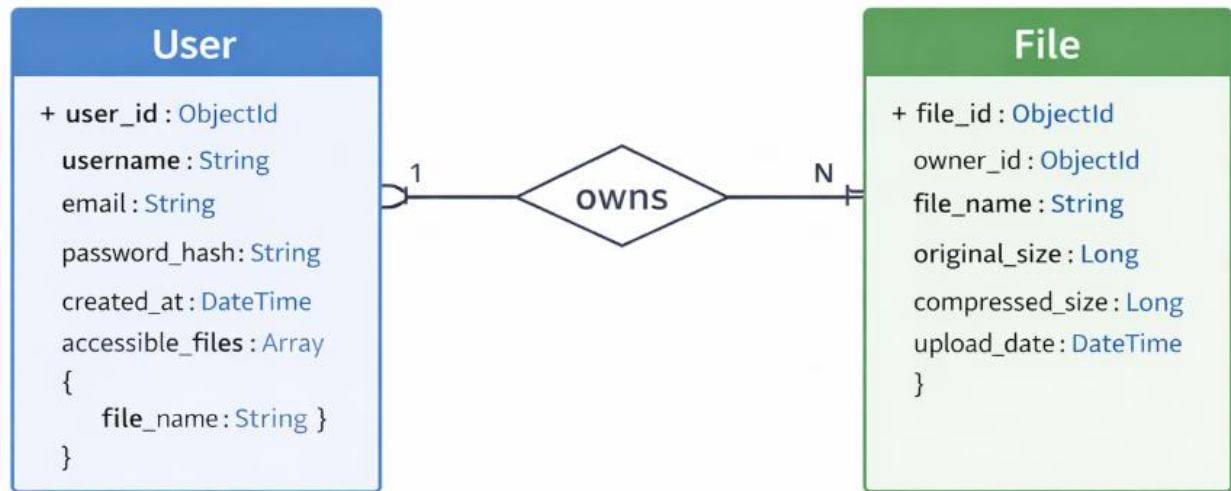
אוסף זה משמש לאחסון מידע נלווה על הקבצים המאוחסנים בשרת.

שדות עיקריים:

- מזהה קובץ
- מזהה המשתמש בעל הקובץ
- שם הקובץ
- גודל הקובץ (לפני ואחרי דחיסה)
- תאריך העלאה

הקבצים עצמם מאוחסנים במערכת האחסון של השרת, בעוד שמסד הנתונים שומר את המידע הדרוש לניהול ושליפה.

להלן תרשים ERD:



מסד הנתונים משמש כרכיב ניהולי התומך בפעולת המערכת. הוא מאפשר זיהוי משתמשים, שיוך קבצים לבעליהם, ניהול הרשאות ושליפת מידע בצורה יעילה. הפרדה זו בין אחסון הקבצים לבין אחסון המידע הנלווה תורמת לסדר הנדסי, תחזוקה נוחה והרחבה עתידית של המערכת. הקשר בין הישויות הוא קשר של אחד-לרבים (One-to-Many) שבו משתמש אחד יכול להיות משויך למספר קבצים, בעוד שכל קובץ משויך למשתמש אחד בלבד.

14. מדריך למשתמש

1. מסך הפתיחה

כאשר נכנסים לאתר בפעם הראשונה, מגיעים למסך הפתיחה הראשי של המערכת.

במסך זה תוכלו לקרוא מידע קצר על הפרויקט ועל היכולות של המערכת.

בחלק העליון של המסך מופיע התפריט הראשי.

כל עוד לא התחברתם לחשבון, המערכת תציג אתכם כ-Guest (אורח).

מתוך המסך הראשי ניתן לעבור למסך ההתחברות או למסך ההרשמה.

2. יצירת חשבון חדש (Register)

כדי להשתמש במערכת ולשמור קבצים אישיים, יש ליצור חשבון משתמש.

לשם כך:

לחצו על כפתור Register בתפריט העליון.

יופיע טופס הרשמה שבו תצטרכו למלא מספר פרטים:

שם משתמש

השם שיופיע בחשבון שלכם.

יש להזין לפחות 3 תווים.

כתובת אימייל

כתובת הדואר האלקטרוני שלכם.

כתובת זו תשמש אתכם בהמשך לצורך התחברות למערכת.

סיסמה

בחרו סיסמה באורך של לפחות 6 תווים.

במהלך מילוי הטופס המערכת בודקת שהנתונים תקינים.

כל עוד אחד מהשדות אינו תקין, כפתור ההרשמה יישאר כבוי.

לאחר שכל הפרטים ימולאו בצורה תקינה:

- כפתור ההרשמה יהפוך לזמין
- ניתן יהיה להשלים את יצירת החשבון

בסיום ההרשמה תועברו אוטומטית למסך ההתחברות.

3. התחברות למערכת (Login)

אם כבר קיים ברשותכם חשבון, ניתן להתחבר למערכת באמצעות כפתור Login שבתפריט העליון.

במסך ההתחברות יש להזין:

- כתובת אימייל
- סיסמה
- לאחר לחיצה על כפתור ההתחברות:

- המערכת תבדוק את פרטי המשתמש
- אם הפרטים נכונים, תועברו לאזור האישי שלכם

4. האזור האישי - My Cloud

לאחר התחברות מוצלחת תעברו לאזור הקבצים האישי שלכם.

זהו האזור המרכזי במערכת, ובו ניתן:

- להעלות קבצים
- להוריד קבצים
- למחוק קבצים
- לצפות בכל הקבצים שנשמרו בחשבון

בחלק העליון של המסך המערכת תציג את שם המשתמש המחובר.

5. העלאת קבצים

בחלק העליון של המסך מופיע אזור העלאת קבצים.

ניתן להעלות קובץ בשתי דרכים:

לחיצה על אזור ההעלאה ובחירת קובץ מהמחשב

גרירת קובץ ישירות אל אזור ההעלאה

לאחר בחירת הקובץ:

תהליך ההעלאה יתחיל באופן אוטומטי

יוצג פס התקדמות המציג את מצב ההעלאה

המערכת תומכת בהעלאת מספר קבצים במקביל.

במהלך ההעלאה הקבצים עוברים תהליך דחיסה, שמטרתו לחסוך מקום אחסון ולשפר את יעילות המערכת.

6. צפייה בקבצים

כל הקבצים שהועלו למערכת מוצגים בטבלה המרכזית במסך.

עבור כל קובץ ניתן לראות:

- שם הקובץ
- סוג הקובץ
- תאריך העלאה
- גודל הקובץ לאחר הדחיסה

הטבלה מתעדכנת באופן אוטומטי לאחר כל העלאה או מחיקה של קובץ.

7. הורדת קבצים

ליד כל קובץ בטבלה מופיע כפתור Download.

לחיצה על הכפתור תוריד את הקובץ בחזרה למחשב שלכם.

המערכת מבצעת את פתיחת הדחיסה באופן אוטומטי, כך שהקובץ יורד בדיוק כפי שהועלה במקור.

8. מחיקת קבצים

אם ברצונכם למחוק קובץ, ניתן לעשות זאת באמצעות כפתור Delete המופיע ליד כל קובץ בטבלה.

לאחר המחיקה:

- הקובץ יוסר מהמערכת
- לא יהיה ניתן לשחזר אותו

9. עריכת פרטי משתמש

ניתן לעדכן את פרטי החשבון דרך כפתור Profile שבתפריט העליון.

בחלון הפרופיל ניתן:

- לשנות שם משתמש
- לעדכן כתובת אימייל
- להחליף סיסמה

אם אינכם מעוניינים לשנות את הסיסמה, ניתן להשאיר את שדה הסיסמה ריק.

לאחר ביצוע השינויים יש ללחוץ על כפתור השמירה.

במקרה של שינוי כתובת האימייל:

- המערכת תעדכן את פרטי החשבון
- המשתמש יתבקש להתחבר מחדש באמצעות האימייל החדש

כל הקבצים והמידע האישי יישמרו בחשבון גם לאחר שינוי האימייל.

10. התנתקות מהמערכת (Logout)

בסיום השימוש במערכת מומלץ לבצע התנתקות מהחשבון, במיוחד כאשר משתמשים במחשב ציבורי או משותף.

כדי להתנתק:

לחצו על כפתור Logout שבתפריט העליון

לאחר ההתנתקות:

- תועברו בחזרה למסך הפתיחה של המערכת
- רק התחברות מחדש תאפשר גישה לקבצים האישיים שלכם

15. בדיקות והערכה

במהלך פיתוח המערכת ולאחר השלמתה, בוצעו בדיקות תוכנה מקיפות (Testing) במטרה לוודא את אמינות המערכת, יציבותה, אבטחתה ויכולת ההתמודדות שלה עם תרחישי שימוש שונים.

הבדיקות כללו בדיקות פונקציונליות של ממשק המשתמש ומסד הנתונים, בדיקות אלגוריתמיות למנגנוני הדחיסה והפריסה, וכן בדיקות עומסים ומקרי קצה שנועדו לבחון את עמידות המערכת בתנאי אמת.

אחת הבדיקות הראשונות שבוצעו הייתה בדיקת הרשמה עם כתובת אימייל שכבר קיימת במסד הנתונים. מטרת הבדיקה הייתה לוודא שהמערכת אינה מאפשרת יצירת משתמשים כפולים. התוצאה המצופה הייתה חסימת ההרשמה והצגת הודעת שגיאה מתאימה למשתמש. בפועל, המערכת זיהתה שהאימייל כבר קיים, חסמה את הפעולה והציגה הודעה ברורה למשתמש, ולכן הבדיקה עברה בהצלחה.

בנוסף, נבדקה אפשרות של העלאת קובץ כפול – כלומר ניסיון להעלות קובץ בעל שם זהה לקובץ שכבר נמצא בתיקיית המשתמש בשרת. מטרת הבדיקה הייתה למנוע מצב של דריסת מידע קיים. המערכת הייתה אמורה לחסום את ההעלאה ולהתריע בפני המשתמש. בפועל, ההעלאה נעצרה בהצלחה ולא נדרס אף קובץ בשרת ה-FTP, ולכן גם בדיקה זו הוגדרה כמוצלחת.

בהיבט האבטחתי בוצעה בדיקת Directory Traversal, שבה נעשה ניסיון להזין נתיב זדוני כגון other_user/.. במטרה לגשת לתיקיות של משתמשים אחרים. התוצאה המצופה הייתה שהשרת ינטרל את הנתיב וישמור את הקובץ רק בתיקייה המורשית של המשתמש המחובר. בפועל, המערכת ניתנה את הנתיב הזדוני, מנעה חריגה מהתיקייה המורשית ושמרה את הקובץ בצורה בטוחה, ולכן הבדיקה עברה בהצלחה.

בדיקה נוספת עסקה בהורדת קבצים בעלי שמות בעברית או שמות המכילים רווחים ותווים מיוחדים. המטרה הייתה לוודא שהדפדפן יקבל את שם הקובץ בצורה תקינה בעת ההורדה. לצורך כך נעשה שימוש במנגנון ContentDisposition. בפועל, הקבצים ירדו בהצלחה עם שמם המקורי ללא פגיעה בקידוד או בשם הקובץ.

כמו כן, נבדק תרחיש של ביטול הורדה באמצע התהליך (Client Abort). בתרחיש זה המשתמש התחיל להוריד קובץ גדול ולאחר מכן ביטל את ההורדה דרך הדפדפן. התוצאה המצופה הייתה שהשרת יזהה את סגירת החיבור, יעצור את תהליך הפריסה של LZ78 ולא יקרוס. בפועל, המערכת עצרה את הלולאה בצורה תקינה באמצעות מנגנון try-catch, שחררה את משאבי השרת ולא נרשמו שגיאות קריטיות ביומן המערכת.

בנוסף לכך, בוצעה בדיקת עומס באמצעות העלאת קובץ גדול במיוחד, בנפח של מעל GB1, במטרה לבדוק את מנגנון ה-Zero-RAM Streaming. המטרה הייתה לוודא שהמערכת מסוגלת לטפל בקבצים גדולים מבלי לגרום לשגיאת OutOfMemory. במהלך הבדיקה נמצא כי הדחיסה וההעברה בוצעו באמצעות Buffers בגודל KB256 בלבד, וצריכת הזיכרון של השרת נשארה יציבה לאורך כל התהליך.

לאחר ביצוע כלל הבדיקות ניתן להסיק כי המערכת עומדת בדרישות היציבות והביצועים שהוגדרו עבורה. מנוע הדחיסה מסוג LZ78 הציג תוצאות טובות במיוחד עבור קבצי טקסט, קוד מקור ונתוני XML/JSON, שבהם קיימים רצפים חוזרים רבים. לעומת זאת, עבור קבצי מדיה שכבר עברו דחיסה מוקדמת, כגון JPEG ו-MP4, יחס הדחיסה היה נמוך יותר – תוצאה צפויה בהתאם לעקרונות תורת האינפורמציה.

עוד נמצא כי הגבלת גודל המילון ל-450,000 רשומות אפשרה לשמור על יציבות המערכת ולמנוע זליגות זיכרון, מבלי לפגוע בצורה משמעותית ביעילות הדחיסה. בנוסף, ארכיטקטורת ה-Zero-RAM Streaming הוכיחה את עצמה כיעילה במיוחד, משום שהמערכת אינה טוענת את כל הקובץ לזיכרון הראשי אלא מעבדת אותו בהזרמה רציפה. בצורה זו, בכל רגע נתון נשמרים בזיכרון רק Buffers קטנים, דבר המאפשר למערכת להתמודד גם עם קבצים גדולים במיוחד.

לבסוף, בדיקות העומס שבוצעו על שרת ה-FTP המקומי הראו כי השימוש במנגנון Virtual Threads מאפשר למערכת להתמודד עם עשרות חיבורים במקביל ללא ירידה משמעותית בביצועים או בזמן

התגובה. תוצאה זו מעידה על רמת Scalability גבוהה ועל יכולת המערכת להתרחב ולהמשיך לפעול בצורה יציבה גם תחת עומסי עבודה גדולים.

16. מסקנות

במהלך הפרויקט פותחה מערכת אחסון ענן המאפשרת למשתמשים להעלות, להוריד ולנהל קבצים בצורה נוחה ומאובטחת.

המערכת שילבה בין ממשק משתמש, צד שרת, מסד נתונים ואלגוריתם דחיסה, ובכך יצרה סביבת עבודה מלאה ויעילה.

הפרויקט תרם להבנה מעשית של תהליך פיתוח תוכנה מלא – החל משלב התכנון, דרך הפיתוח והבדיקות, ועד הפעלת המערכת בפועל.

בנוסף, העבודה על הפרויקט חיזקה את הידע בטכנולוגיות כגון:

- Spring Boot
- MongoDB
- FTP
- Java
- ואלגוריתם הדחיסה LZ78

אחת המסקנות המרכזיות היא שתכנון נכון של מבנה המערכת וחלוקה מסודרת בין שכבות הקוד הופכים את המערכת ליציבה, ברורה וקלה יותר לתחזוקה ולהרחבה בעתיד.

לסיכום, המערכת עמדה במטרות שהוגדרו בתחילת הפרויקט וסיפקה פתרון יעיל לניהול ואחסון קבצים.

17. פיתוחים עתידיים

1. שיפור חוויית המשתמש בממשק הוויזואלי- פיתוח ממשק נוח יותר, כולל גרירה ושחרור ותצוגת קבצים מקדימה(Preview).
2. סינון תוכן וקבצים- מנגנון למניעת העלאה של קבצים מזיקים או לא מורשים, לשיפור האבטחה.
3. אימות דו שלבי(Two Factor Authentication)- הוספת שכבת אבטחה נוספת עבור כניסה למערכת באמצעות קוד חד פעמי.

18. ביבליוגרפיה

הסבר על אלגוריתם LZ78:

<https://www.youtube.com/watch?v=LPFtaEy3gMA>

הסברים על LZ77 ו-LZ78:

[https://www.scribd.com/document/337943838/Study-of-LZ77-and-LZ78-Data-](https://www.scribd.com/document/337943838/Study-of-LZ77-and-LZ78-Data-Compression)

Compression

https://en.wikipedia.org/wiki/LZ77_and_LZ78

הסבר על קידוד מילון:

https://en.wikipedia.org/wiki/Dictionary_coder

הסבר על קוד הופמן:

<https://web.mit.edu/6.02/www/f2011/handouts/3.pdf>

הסבר על דחיסת נתונים:

<https://apps.dtic.mil/sti/tr/pdf/ADA295369.pdf>

הסברים על FTP:

https://en.wikipedia.org/wiki/File_Transfer_Protocol

<https://www.fortinet.com/uk/resources/cyberglossary/file-transfer-protocol-ftp-meaning>

<https://www.geeksforgeeks.org/computer-networks/file-transfer-protocol-ftp-in-application-layer>